



Project Documentation Guide

Creating, organizing and maintaining project docs in Markdown, for developers, tech leads, analysts, technical writers and AI coding agents.

Kyrylo Butskyi

Project Documentation Guide

A practical handbook for creating, organizing and maintaining project documentation

WRITTEN BY

Kyrylo Butskyi

Full-stack developer focused on data-heavy products: Laravel APIs, queue-driven pipelines, React SPAs, and Docker-based deployment.

kyrylo.butskyi@gmail.com

Source: github.com/kiril4ik/project-docs-guide. This book is a typeset edition of the repository; the Markdown files remain the source of truth.

Diagrams marked "rendered" were drawn from the Mermaid sources in the repository. The additional figures were derived from tables in the guide itself.

Set in Source Serif 4, Inter and JetBrains Mono. October 2026.

Contents

	Introduction	6
--	---------------------------	----------

PART I	Foundations	11
	Overview	12
1.1	Principles of Good Documentation	13
1.2	Documentation Audiences	16
1.3	Markdown Essentials	19
1.4	Glossary	23

PART II	Document Types.....	25
	Overview	26
2.1	Developer Docs	29
2.2	API Docs.....	34
2.3	Business & Requirements Docs.....	38
2.4	Architecture Docs	44
2.5	Operations Docs	48
2.6	User Docs	51
2.7	Process & Team Docs.....	54
2.8	AI Agent Docs	56

PART III	Organizing Docs	59
	Overview	60
3.1	The Six Dimensions of Doc Organization.....	63
3.2	Organizing Models.....	67
3.3	Model: Organize by Doc Type	68
3.4	Model: Organize by Audience	70
3.5	Model: Organize by Domain / Feature.....	73
3.6	Model: Diátaxis (by User Need)	76
3.7	Model: Organize by Lifecycle / Status	79
3.8	Model: Co-located with Code	82
3.9	Model: Per-Service Repos + Central Portal	85
3.10	Model: Sequential & Dated Records.....	88
3.11	Model: Organize by Version	91
3.12	Model: Flat Structure + Metadata (Wiki-style).....	93
3.13	Model: Hybrid (Combining Models)	96
3.14	Storage Locations.....	99
3.15	Internal vs Public Docs	102
3.16	Folder Structures: Ready-Made Trees	104
3.17	Naming Conventions	110
3.18	Navigation & Metadata	113
3.19	Organization Smells & Fixes	117

PART IV	Choosing a Structure	119
	Overview	120
4.1	Decision Guide.....	121

4.2	Scenarios: Recommended Setups	126
4.3	Evolving the Structure	129

PART V	Writing.....	132
	Overview	133
5.1	Writing Process.....	134
5.2	Style Guide.....	137
5.3	Page Patterns.....	141
5.4	Diagrams & Visuals	146
5.5	Writing for AI Agents.....	149
5.6	Maintenance	152

PART VI	Templates	155
	Overview	156
	Root README	159
	Module / package / service README	161
	CONTRIBUTING.....	162
	CHANGELOG	164
	Docs root index	165
	Architecture Decision Record (Nygard style + alternatives)	166
	RFC / design doc.....	167
	Business Requirements Document	169
	Product Requirements Document	171
	User story with acceptance criteria.....	173
	Use case	175
	Business rules catalog for one domain	176
	Hand-written API endpoint reference.....	177
	Runbook.....	179
	Blameless incident postmortem.....	181
	Tutorial (learning-oriented)	183
	How-to guide (task-oriented)	185
	AGENTS	186

A	Further Reading.....	188
----------	-----------------------------	------------

BEFORE YOU START

The guide at a glance



How the content is distributed

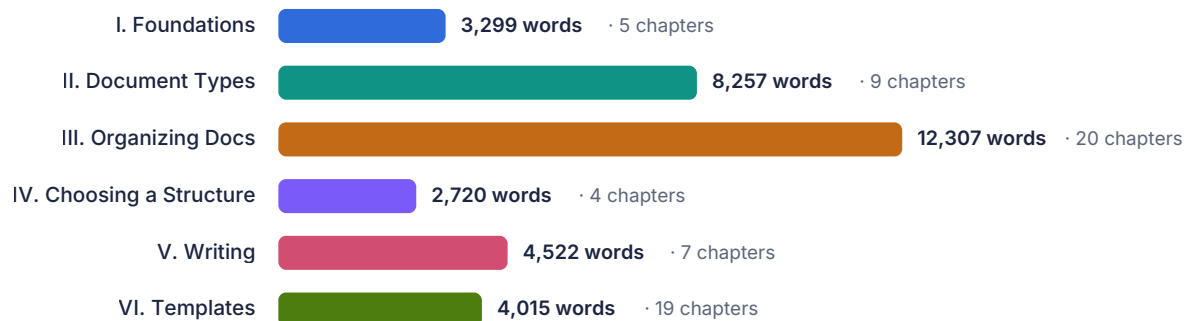


Figure 0.1 Word count of the source Markdown in each part. Templates are counted as written; organizing is the core of the guide.

Seven rules, if you read nothing else

- 1. Know who you're writing for.** One main audience, one job per document.
- 2. Pick the doc type before you write.**
- 3. Pick an organizing model on purpose,** and write the choice down.
- 4. Keep docs close to what they describe,** reviewed in the same PR as the code.
- 5. Write for scanning:** summary first, clear headings, tables, real examples.
- 6. Make it navigable:** a README index in every folder.
- 7. Maintain it:** owner, status, last-reviewed date. Wrong docs are worse than no docs.

Introduction

A practical handbook for creating, organizing and maintaining project documentation

What: A practical handbook for creating, organizing and maintaining project documentation, mostly in Markdown. **For:** developers, tech leads, product/business analysts, technical writers and AI coding agents. **Start here:** read this page (5 min), then jump to the section you need.

TL;DR

1. **Know who you're writing for.** Every document has one main audience and one job.
2. **Pick the doc type before you write.** A tutorial, an API reference and a business requirement are different things with different shapes.
3. **Pick an organizing model on purpose.** Group docs by audience, by type, by domain, or a mix of these, and write the choice down.
4. **Keep docs close to what they describe.** Docs that live in the repo get reviewed in the same PR as the code.
5. **Write for scanning.** Lead with a summary, use clear headings, short paragraphs, tables and real examples.
6. **Make it navigable.** Every folder has a `README.md` index, and every page links up to its parent and across to related pages.
7. **Maintain it.** Each doc needs an owner, a status and a "last reviewed" date. Wrong docs are worse than no docs.

■ How this guide is organized

The guide follows the order in which you actually make documentation decisions:



Figure 0.2 The six parts of the guide follow the order in which documentation decisions are made.

```

project-docs-guide/
├── README.md
├── further-reading.md

```

← you are here: basics, doc types, map
curated articles, specs, books and example docs by topic

foundations/	WHY & BASICS
README.md	
principles.md	core rules of good documentation
audiences.md	who reads docs and what they need
markdown-essentials.md	Markdown syntax, GFM, front matter, Mermaid
glossary.md	terms used in this guide
doc-types/	WHAT TO WRITE
README.md	catalog of all doc types (master table)
developer-docs.md	README, CONTRIBUTING, setup, conventions, CHANGELOG
api-docs.md	REST/GraphQL/gRPC/events, OpenAPI, reference vs guides
business-requirements.md	BRD, PRD, SRS, user stories, use cases, business rules
architecture-docs.md	C4, ADRs, RFCs / design docs
operations-docs.md	runbooks, deployment, incidents, postmortems
user-docs.md	end-user guides, FAQ, release notes
process-docs.md	onboarding, roadmap, meetings, policies
ai-agent-docs.md	AGENTS.md, CLAUDE.md, llms.txt, agent rules
organizing/	WHERE & HOW TO STORE IT (core section)
README.md	comparison matrix of all models
dimensions.md	6 axes: type, audience, domain, lifecycle, version, visibility
models/	one page per organizing model
by-doc-type.md	by-audience.md by-domain.md diataxis.md
by-lifecycle.md	co-located.md per-service-repo.md
sequential-and-dated.md	by-version.md flat-with-metadata.md
hybrid.md	combining models, one dimension per level
storage-locations.md	repo root, docs folder, next to code, separate repo, wiki, site
internal-vs-public.md	keeping published and internal docs apart
folder-structures.md	8 ready-made trees for common project shapes
naming-conventions.md	files, folders, numbering, dates, IDs, docs-root name
navigation-and-metadata.md	indexes, cross-links, front matter, status
smells-and-fixes.md	symptoms of bad organization and fixes
choosing-structure/	HOW TO PICK THE RIGHT SETUP
README.md	
decision-guide.md	questions, decision tree, scoring matrix
scenarios.md	recommended setups for common project shapes
evolving-structure.md	when and how to restructure safely
writing/	HOW TO WRITE IT
README.md	
writing-process.md	plan → outline → draft → review → publish
style-guide.md	voice, headings, lists, code, terminology
page-patterns.md	page anatomy for each kind of page
diagrams-and-visuals.md	Mermaid, C4, screenshots, tables
writing-for-ai-agents.md	making docs easy for LLMs and agents to use
maintenance.md	ownership, reviews, linting, CI, deprecation
templates/	COPY-PASTE STARTERS
README.md	template index
*-template.md	README, CONTRIBUTING, CHANGELOG, docs index, module README, ADR, RFC, BRD, PRD, user story, use case, business rules, API endpoint, runbook, postmortem, tutorial, how-to, AGENTS.md

Why topic-named folders and not docs/ ? This repository is a guide, so every top-level folder is named after its topic. You can open `writing/` or `templates/` straight from the file tree without following links. The order in the tree above is the suggested reading order. In your own projects `docs/` is still a fine default. See [naming the docs root \(p. 110\)](#) for alternatives.

Folder → question it answers:

Folder	Answers
Foundations overview (p. 12)	Why document, who reads it, how Markdown works

Folder	Answers
Document Types overview (p. 26)	<i>What</i> kinds of documents exist and when to write each
Organizing Docs overview (p. 60)	<i>Where</i> to store docs and <i>how</i> to structure folders
Choosing a Structure overview (p. 120)	<i>Which</i> structure fits <i>my</i> project
Writing overview (p. 133)	<i>How</i> to write, illustrate and maintain docs
Templates overview (p. 156)	<i>Give me</i> a starting file to copy

■ Documentation types at a glance

Every project needs some of these. The full catalog, with when-to-use rules, is in [Document Types overview](#) (p. 26).

Family	Typical documents	Main audience	Details
Developer	README, CONTRIBUTING, local setup, coding conventions, CHANGELOG	Engineers working <i>on</i> the code	Developer Docs (p. 29)
API	API reference (OpenAPI/GraphQL schema), auth guide, error catalog, SDK guides	Engineers <i>consuming</i> the code	API Docs (p. 34)
Business / requirements	BRD, PRD, SRS, user stories, use cases, business rules, glossary	Product, business, QA, engineers	Business & Requirements Docs (p. 38)
Architecture	System overview, C4 diagrams, ADRs, RFCs / design docs	Engineers, architects, tech leads	Architecture Docs (p. 44)
Operations	Runbooks, deployment guides, incident postmortems, SLOs	On-call, DevOps/SRE	Operations Docs (p. 48)
User	Tutorials, how-to guides, FAQ, release notes	End users, customers, support	User Docs (p. 51)
Process / team	Onboarding, roadmap, meeting notes, policies, SECURITY.md	Team members, stakeholders	Process & Team Docs (p. 54)
AI agent	AGENTS.md, CLAUDE.md, rules files, llms.txt	AI coding assistants	AI Agent Docs (p. 56)

■ Ways of organizing docs at a glance

There is no single correct structure. Pick the one that matches how people *look for* information in your project. Full comparison matrix: [Organizing Docs overview](#) (p. 60). The theory behind it: [The Six Dimensions of Doc Organization](#) (p. 63).

Model	Top-level folders look like	Best when
By doc type (p. 68)	<code>architecture/</code> , <code>api/</code> , <code>requirements/</code> , <code>runbooks/</code>	Small/medium single product, one team
By audience (p. 70)	<code>developers/</code> , <code>users/</code> , <code>operators/</code> , <code>business/</code>	Distinct reader groups with little overlap
By domain / feature (p. 73)	<code>billing/</code> , <code>auth/</code> , <code>catalog/</code>	Large product, teams own domains
Diátaxis (p. 76)	<code>tutorials/</code> , <code>how-to/</code> , <code>reference/</code> , <code>explanation/</code>	Product docs, libraries, public docs sites
By lifecycle (p. 79)	<code>proposals/</code> , <code>active/</code> , <code>archive/</code>	Lots of specs/RFCs that change status

Model	Top-level folders look like	Best when
Model: Co-located with Code (p. 82)	<code>services/payments/README.md</code> next to the code	Monorepos, microservices, component libraries
Per-service repos + portal (p. 85)	docs in each repo, aggregated in a handbook or portal	Many repos, microservice organizations
Sequential & dated (p. 88)	<code>0012-use-stripe.md</code> , <code>2026-09-18-outage.md</code>	ADRs, RFCs, postmortems, meeting notes (inside a folder)
By version (p. 91)	<code>v1/</code> , <code>v2/</code> or a versioned docs site	Several live major versions (public APIs, SDKs)
Flat + metadata (p. 93)	one folder, tags in front matter	Fewer than ~20 docs, wikis
Model: Hybrid (Combining Models) (p. 96) (most common)	type at the top, domain inside, co-located READMEs	Most real projects past the early stage

Quick picker: go to [Decision Guide](#) (p. 121). It asks about 8 questions and gives you a recommended tree.

■ The 60-second recommended default

If you don't want to decide yet, start here. It scales well and is easy to restructure later.

```

your-project/
├── README.md           what it is, quick start, links to everything below
├── CONTRIBUTING.md     how to work on it
├── CHangelog.md        what changed per version
├── AGENTS.md           instructions for AI coding agents (optional)
├── docs/              (or handbook/, wiki/, knowledge/, ...)
│   ├── README.md      index of everything in this folder
│   ├── getting-started/ setup, first run, onboarding
│   ├── architecture/  overview + decisions/ (ADRs)
│   ├── api/           reference + guides
│   ├── requirements/  PRDs, user stories, business rules
│   ├── operations/    deployment, runbooks, incidents
│   └── glossary.md     shared vocabulary

```

Why this default works and when to leave it: see [Scenarios: Recommended Setups](#) (p. 126).

■ Suggested reading paths

You are...	Read in this order
Starting docs for a new project	Principles of Good Documentation (p. 13) → Decision Guide (p. 121) → Folder Structures: Ready-Made Trees (p. 104) → Templates overview (p. 156)
Fixing messy existing docs	Organizing Docs overview (p. 60) → Evolving the Structure (p. 129) → Maintenance (p. 152)
Writing one specific document	doc-types catalog (p. 26) → matching type page → Page Patterns (p. 141) → template (p. 156)
Product / business analyst	Business & Requirements Docs (p. 38) → Style Guide (p. 137) → PRD template (p. 171)
Setting up docs for AI agents	AI Agent Docs (p. 56) → Writing for AI Agents (p. 149) → AGENTS template (p. 186)
New to Markdown	Markdown Essentials (p. 19)
Want to go deeper	Further Reading (p. 188): the best external articles, specs and books on each topic

■ Conventions used in this guide

Every page in this guide follows the same shape, so humans can scan it and AI agents can parse it reliably:

- **Header block:** a `> **What:** / **Read when:**` quote at the top.
- **TL;DR:** 3–7 bullets with the key points.
- **Body:** H2 sections in a predictable order, tables for comparisons, fenced code for anything you copy.
- **Related:** links to the parent index and related pages at the bottom.
- **Labels:**  *Do*,  *Don't*,  *Watch out*.
- **Links:** always relative (`../doc-types/api-docs.md`), so they work on GitHub, GitLab, in IDEs and on doc sites.

■ For AI agents reading this repository

- The guide is plain Markdown. Start from this file and follow links. Each folder's `README.md` lists its pages.
- When asked to **create documentation** for a project, apply [Decision Guide \(p. 121\)](#) first, then use the files in [Templates overview \(p. 156\)](#).
- When asked **where a doc belongs**, use the master table in [Document Types overview \(p. 26\)](#) and the trees in [Folder Structures: Ready-Made Trees \(p. 104\)](#).
- When **writing**, follow [Style Guide \(p. 137\)](#) and [Writing for AI Agents \(p. 149\)](#).

PART I

Foundations

Why document, who reads it, how Markdown works

[foundations/](#)

Overview	12
Principles of Good Documentation	13
Documentation Audiences	16
Markdown Essentials	19
Glossary	23

PART I

Overview

What: The ideas and tools every other section builds on. **Read when:** you're starting out, or you want to know *why* the rest of the guide recommends what it does.

■ Pages

Page	What you'll learn	Time
Principles of Good Documentation (p. 13)	Ten principles of good documentation, and the anti-patterns they prevent	8 min
Documentation Audiences (p. 16)	The reader groups a project has, what each needs and how they search	6 min
Markdown Essentials (p. 19)	Markdown syntax, GitHub-Flavored Markdown, front matter, Mermaid, callouts	12 min
Glossary (p. 23)	Definitions of every term this guide uses (ADR, BRD, Diátaxis, docs-as-code, ...)	reference

■ Key idea of this section

Documentation is a **product** with **users**. Like any product, it needs a clear purpose, a known audience, a structure people can find their way around, and maintenance.

CHAPTER 1.1

Principles of Good Documentation

What: Ten principles that decide whether docs get used or ignored. **Read when:** before you create a doc structure, or when you have to argue for a documentation decision.

TL;DR

- One document has **one audience and one job**.
- Keep a **single source of truth**: link to things, don't copy them.
- Docs live **close to the code** and change **in the same PR** (docs-as-code).
- Write for **scanning first**, reading second.
- Every doc has an **owner, a status and a review date**.
- **Wrong docs are worse than missing docs**. Delete or archive anything stale.

■ 1. One audience, one job

A document that tries to serve a new developer, an auditor and a customer at the same time serves none of them well.

Before writing, fill in this sentence:

"This document helps [audience] to [do / decide / understand X]."

If you can't finish it in one sentence, split the doc.

✗ **Mixed**

payments.md covering business rules, API endpoints and the on-call procedure

✓ **Split**

requirements/payments-rules.md, api/payments.md, runbooks/payments-failures.md

■ 2. Single source of truth (SSOT)

Every fact lives in **one** place. Everywhere else **links** to it.

- ✓ The API reference is generated from the OpenAPI spec, and guides link to it.
- ✓ Environment variables are listed in `.env.example` (or one config doc), and other pages link there.
- ✗ The same setup steps copied into README, CONTRIBUTING and the wiki, which then drift apart within weeks.

When duplication is unavoidable (for example a short summary in the README), mark which copy is canonical:

> Summary only. Canonical version: architecture/overview.md

MARKDOWN

■ 3. Docs-as-code

Treat docs like source code:

Practice	Why
Store in the same Git repo (Markdown)	Versioned with the code, so history and blame work
Change in the same pull request as the code	Docs can't fall behind silently
Review in PRs	Catches mistakes and spreads knowledge
Lint and link-check in CI	Prevents broken links and formatting rot
Publish automatically (optional)	A docs site is always current

Exceptions: content written mainly by non-engineers (marketing, some business docs) may fit a wiki or CMS better. See [Storage Locations](#) (p. 99).

■ 4. Proximity: keep docs near what they describe

The closer a doc is to its subject, the more likely it is to be updated.

Distance from code → Likelihood of staying correct

code comment / docstring	highest
README.md in the module folder	
/docs folder in the same repo	
separate docs repository	
external wiki (Confluence, Notion)	lowest

Use the closest location that the audience can still find and read.

■ 5. Write for scanning

Most readers skim first, so design for that:

- Put the **answer first** (TL;DR, summary, "use X when Y").
- Use **descriptive headings**: "Rotate the API key", not "Procedure".
- Keep paragraphs to **3–4 sentences**.
- Use **tables** for comparisons and **numbered lists** for sequences.
- Make code **copy-pasteable** and correct.

■ 6. Show, then tell

A working example beats three paragraphs of explanation. Pair every concept with a real example: a command, a request/response, a sample user story, or a diagram.

■ 7. Progressive disclosure

Layer information from simple to deep:

```

README (what + quick start)
├─ Getting started guide (first real task)
├─ How-to guides (specific tasks)

```

- └ Reference (every option)
 - └ Explanation (why it works this way)

A reader should be able to stop at any layer with something useful.

■ 8. Explicit ownership and status

Docs without owners rot. Each important doc should state:

```
---
owner: team-payments      # who keeps it correct
status: active            # draft | active | deprecated | archived
last_reviewed: 2026-10-01
---
```

YAML

See [Navigation & Metadata](#) (p. 113).

■ 9. Delete or archive stale docs

A doc that is 30% wrong costs readers time and trust in *every* doc. Prefer:

1. **Fix** it, if it's still needed.
2. **Archive** it (move to `archive/`, add a banner) if it has historical value, such as old specs and superseded ADRs.
3. **Delete** it otherwise. Git keeps the history.

■ 10. Consistency beats perfection

A consistent, average structure is easier to use than a brilliant structure followed half the time. Write your conventions down (a short `docs/README.md` or `CONTRIBUTING.md` section) and use templates ([Templates overview](#) (p. 156)).

■ Common anti-patterns

Anti-pattern	Symptom	Fix
Wall of text	No headings, 20-line paragraphs	Add structure; see Style Guide (p. 137)
Junk drawer	<code>docs/misc/</code> , <code>notes.md</code> , <code>stuff-v2-final.md</code>	Pick an organizing model (p. 60)
Copy-paste drift	Same instructions in 3 places, all different	Single source of truth + links
Wiki graveyard	Hundreds of pages, nobody knows which are current	Status metadata, owners, archiving
Docs-only-in-heads	"Ask Alex, they know how deploy works"	Write a runbook, starting with a bad one
Over-documentation	Docs that restate what the code says, line by line	Document <i>why</i> and <i>how to use</i> , not <i>what each line does</i>
Orphan pages	Pages no index or page links to	Every folder has a README index

RELATED: [Documentation Audiences](#) (p. 16) · [Organizing Docs overview](#) (p. 60) · [Maintenance](#) (p. 152)

CHAPTER 1.2

Documentation Audiences

What: The reader groups a project has, what each one needs, and how each one looks for information.

Read when: deciding what to write, or when choosing to organize docs [by audience](#) (p. 70).

TL;DR

- Most projects have 4–7 distinct audiences. List yours **before** designing a structure.
- Audiences differ in **goal, vocabulary, depth** and **how they search**.
- An audience with very different needs (for example external API consumers or end users) usually deserves its **own section, or even its own site**.
- AI agents are now a real audience. They need explicit, structured and unambiguous docs.

■ Audience map

Audience	Main goal	Wants	Doesn't want	Typical docs
New developer (onboarding)	Get productive fast	Setup steps, architecture overview, "where is X"	Deep history, every edge case	README, getting-started, architecture overview, glossary
Maintainer / core developer	Change code safely	Conventions, decisions and their reasons, internals	Marketing, basics	ADRs, design docs, coding conventions, CONTRIBUTING
API consumer (internal or external)	Integrate quickly	Auth, endpoints, examples, errors, limits	Internal implementation	API reference, quick start, SDK guides, changelog
Product / business	Define and verify behavior	Requirements, business rules, scope, status	Code-level detail	BRD, PRD, user stories, business rules, roadmap
QA / testers	Verify behavior	Acceptance criteria, rules, edge cases, test data	Architecture internals	User stories + acceptance criteria, test plans
Operations / SRE / on-call	Keep it running, fix it fast at 3 a.m.	Runbooks, dashboards, alerts, rollback steps	Long explanations	Runbooks, deployment guides, postmortems, SLOs
End user / customer	Get their task done	Task-focused guides, screenshots, FAQ	Technical jargon	User guides, tutorials, FAQ, release notes
Security / compliance / auditor	Check controls and evidence	Data flows, policies, access, decisions	Opinions without evidence	SECURITY.md, data-flow diagrams, policies, ADRs
AI coding agent	Act correctly in the repo	Commands, conventions, boundaries, file map	Ambiguity, implied knowledge	AGENTS.md / CLAUDE.md, rules files, structured docs



Figure 1.1 Which documentation families each audience relies on, derived from the “Typical docs” column of the audience map above. Bars count how many audiences each family serves.

■ How each audience searches

This decides how you organize and name things.

Audience	Typical search behavior	Implication
New developer	Linear: reads from the top	Give a clear <i>reading path</i> in README
Maintainer	Targeted: "why did we choose Kafka?"	Searchable decision log (ADRs), good file names
API consumer	By endpoint or task: "create invoice"	Reference organized by resource, plus task guides
Business	By feature or domain: "refund rules"	Requirements organized by domain/feature
On-call	Under stress, by symptom: "500s on checkout"	Runbooks named after symptoms/alerts , not systems
End user	Search box and FAQ	Task-named pages, synonyms in text
AI agent	Reads entry files, follows links, uses grep/search	Predictable paths, explicit index files, consistent headings

■ Write an audience list for your project

Add this short table to your docs index (for example `docs/README.md`). It forces clarity and helps contributors put new docs in the right place:

Who these docs are for

MARKDOWN

```
| Audience | Start here |
|---|---|
| New developers | [Getting started](getting-started/README.md) |
| API consumers | [API docs](api/README.md) |
```

```
| Product & business | [Requirements](requirements/README.md) |  
| On-call | [Runbooks](operations/runbooks/README.md) |
```

■ Internal vs external audiences

	Internal	External
Tone	Direct, can use team jargon (defined in glossary)	Polished, no internal jargon
Security	Can mention internal hosts and tools	Never expose internal URLs, secrets or infrastructure
Location	Repo, internal wiki	Public docs site, developer portal
Review	Peer review	Peer + editorial (+ legal/security if needed)

⚠ **Never mix internal and external docs in the same published output.** Keep them in separate folders (`internal/` vs `public/`) or separate repos, and publish only the public one.

RELATED: [Principles of Good Documentation](#) (p. 13) · [doc types catalog](#) (p. 26) · [organizing by audience](#) (p. 70)

CHAPTER 1.3

Markdown Essentials

What: Everything you need to write project docs in Markdown: core syntax, GitHub-Flavored Markdown (GFM), front matter, diagrams, callouts and linking rules. **Read when:** you're new to Markdown, or you want your docs to render correctly on GitHub/GitLab, in IDEs and on doc sites.

TL;DR

- Use **CommonMark** + **GFM**. It renders the same almost everywhere.
- Use **one H1 per file** and don't skip heading levels.
- Use **relative links** between docs and **always** give fenced code blocks a language tag.
- Use **YAML front matter** for metadata (owner, status, dates).
- Use **Mermaid** for diagrams kept as text, so they can be diffed and reviewed.

■ Why Markdown

Strength	Why it matters
Plain text	Works with Git: diffs, blame, PR reviews
Renders everywhere	GitHub, GitLab, Bitbucket, IDEs, static site generators
Low barrier	Engineers, PMs and analysts can all edit it
Tool ecosystem	Linters, link checkers, doc site generators
AI-friendly	LLMs read and write Markdown natively, with little token overhead

When Markdown is not enough: heavy formatting for print or legal documents (use a DOCX/PDF pipeline), large structured data (YAML/JSON/CSV next to the doc), and machine-readable API contracts (OpenAPI/GraphQL SDL, with Markdown as the explanation).

Alternatives you may meet: **AsciiDoc** (richer, for books and complex manuals), **reStructuredText** (Python/Sphinx ecosystem), **MDX** (Markdown + React components, for Docusaurus and similar).

■ Core syntax cheat sheet

```
# H1: document title (exactly one per file)
## H2: main sections
### H3: subsections (rarely go deeper than H4)
```

Paragraph text. ****Bold**** for key terms, **italic** for emphasis, ``inline code`` for identifiers, file names, commands and values.

- Unordered list item
- Another item
 - Nested item (indent 2 spaces)

```
1. Ordered step
```

MARKDOWN

2. Next step

```
[Link text](relative/path.md)
[Link to a section](relative/path.md#section-heading)
![Alt text that describes the image](images/diagram.png)

> Blockquote: use for notes, summaries, quotes.

--- (horizontal rule)

```bash
npm install # fenced code block with a language tag
```
```

■ GitHub-Flavored Markdown (GFM) extras

Tables

| Option | Default | Description |
|----------------------|-----------------|---|
| <code>timeout</code> | <code>30</code> | Seconds before the request is cancelled |

MARKDOWN

Alignment: `:---` left, `:---:` center, `---` right. Keep table cells short; if a cell needs a paragraph, use a list or subsection instead.

Task lists

- [x] Write the draft
- [] Get review from the API owner

MARKDOWN

Callouts (alerts)

Supported by GitHub, and by many doc sites with a plugin:

```
> [!NOTE]
> Useful information the reader should know.

> [!TIP]
> Optional advice that helps.

> [!IMPORTANT]
> Key information needed to succeed.

> [!WARNING]
> Urgent info that needs immediate attention to avoid problems.

> [!CAUTION]
> Risks or negative outcomes of an action.
```

MARKDOWN

Portable fallback that works everywhere: `> ⚠️ **Warning:** text`.

Collapsible sections

```
<details>
<summary>Full error log</summary>

Long content here. Leave a blank line after the summary tag.
```

MARKDOWN

```
</details>
```

Use these for long logs or optional detail. Don't hide essential steps in them.

Footnotes, strikethrough, autolinks

```
A claim that needs a source.[^1]
~~deprecated text~~
https://example.com becomes a link automatically.

[^1]: Source or extra detail.
```

MARKDOWN

■ Front matter (metadata)

A YAML block at the very top of the file. GitHub shows it as a table, doc site generators use it, and scripts or AI agents can parse it.

```
---
title: Payment Refund Rules
owner: team-payments
status: active           # draft | review | active | deprecated | archived
audience: [product, qa, developers]
last_reviewed: 2026-10-01
tags: [payments, refunds, business-rules]
---

# Payment Refund Rules
```

MARKDOWN

Keep the field set **small and consistent across the project**. The recommended fields are in [Navigation & Metadata: Front matter schema](#) (p. 114).

■ Diagrams as code: Mermaid

GitHub, GitLab and most doc sites render Mermaid inside fenced blocks:

```
```mermaid
flowchart LR
 User -->|HTTPS| API[API Gateway]
 API --> Orders[Order Service]
 Orders --> DB[(PostgreSQL)]
```
```

MARKDOWN

Common diagram types: `flowchart`, `sequenceDiagram`, `erDiagram`, `stateDiagram-v2`, `classDiagram`, `gantt`, `C4Context`. More in [Diagrams & Visuals](#) (p. 146).

■ Linking rules

| Rule | Example | Why |
|---|--|---|
| Use relative paths between docs | <code>../api/README.md</code> | Works on every host, in forks and offline |
| Link to a file , not to a folder | <code>api/README.md</code> , not <code>api/</code> | Some renderers don't resolve folder links |

| Rule | Example | Why |
|--|---|--|
| Use descriptive link text | see [refund rules](...) | "click here" means nothing to screen readers or AI |
| Section anchors are lowercase, hyphenated | #rotate-the-api-key | GitHub's auto-generated anchor format |
| Use absolute URLs only for external resources | https://spec.openapis.org/... | |
| Don't link to a branch-specific blob URL for internal docs |  github.com/org/repo/blob/main/docs/x.md | Breaks in forks and old versions |

Anchor generation (GitHub): lowercase, spaces become -, most punctuation is removed. ## 2. By Audience → #2-by-audience.

■ Formatting conventions (recommended)

| Topic | Convention |
|-------------|---|
| Line length | No hard wrap, or one sentence per line (gives cleaner diffs). Pick one per repo. |
| Lists | - for bullets, 1. for every ordered item (rendered numbers them) or real numbers. Be consistent. |
| Emphasis | bold and <i>italic</i> . Avoid __ and <i>_</i> variants. |
| Code blocks | Always fenced with a language: <code>bash</code> , <code>json</code> , <code>yaml</code> , <code>http</code> , <code>sql</code> , <code>text</code> |
| Commands | Show the command alone, without the \$ prompt, so it copy-pastes cleanly. Show output in a separate <code>text</code> block. |
| Headings | Sentence case: "Configure the database", not "Configure The Database" |
| Images | Store next to the doc in <code>images/</code> or <code>assets/</code> , with meaningful names and alt text |
| File end | Single trailing newline |

Enforce these with **markdownlint**; see [Maintenance: Automation](#) (p. 153).

RELATED: [Style Guide](#) (p. 137) · [Diagrams & Visuals](#) (p. 146) · [Glossary](#) (p. 23)

CHAPTER 1.4

Glossary

What: Definitions of the terms used throughout this guide, in alphabetical order. **Read when:** you meet an unfamiliar acronym or term.

| Term | Definition | More |
|---|---|--|
| Acceptance criteria (AC) | Testable conditions a feature must meet to be accepted. Often written as Given/When/Then. | Business & Requirements Docs: Acceptance criteria (p. 42) |
| ADR (Architecture Decision Record) | A short, numbered, immutable record of one significant technical decision: its context, the decision and its consequences. | Architecture Docs: Architecture Decision Records (ADRs) (p. 45) |
| AGENTS.md | An open-convention Markdown file at the repo root that gives AI coding agents project instructions. <code>CLAUDE.md</code> is the Claude Code equivalent. | AI Agent Docs (p. 56) |
| API reference | Exhaustive, structured description of every endpoint, field, type and error. Ideally generated from a spec. | API Docs (p. 34) |
| AsyncAPI | Specification format for event-driven/message APIs (Kafka, AMQP, WebSockets). | API Docs: Event-driven APIs (p. 36) |
| BRD (Business Requirements Document) | Describes the business need, goals, stakeholders, scope and high-level requirements. Answers <i>why</i> and <i>what for the business</i> . | Business & Requirements Docs: BRD (Business Requirements Document) (p. 40) |
| Business rule | A statement that defines or constrains some aspect of the business, such as "Refunds over €500 need manager approval". | Business & Requirements Docs: Business rules catalog (p. 42) |
| C4 model | A way to diagram software architecture at four zoom levels: Context, Containers, Components, Code. | Architecture Docs: C4 model (p. 45) |
| CHANGELOG | A human-readable, version-by-version list of notable changes. | Developer Docs: CHANGELOG.md (p. 32) |
| Co-located docs | Docs stored next to the code they describe, such as <code>src/billing/README.md</code> . | Model: Co-located with Code (p. 82) |
| Design doc / RFC | A proposal describing how to build something before building it, circulated for feedback. | Architecture Docs: RFCs and design docs (p. 46) |
| Diátaxis | A documentation framework that splits content into four types: tutorials, how-to guides, reference and explanation. | Model: Diátaxis (by User Need) (p. 76) |
| Docs-as-code | Treating docs like code: plain text, in Git, reviewed in PRs, tested in CI, published automatically. | Principles of Good Documentation: 3. Docs-as-code (p. 14) |
| Docs root | The top-level folder holding project docs (<code>docs/</code> , <code>handbook/</code> , etc.). | Naming Conventions: Naming the documentation root (p. 110) |
| Front matter | A YAML metadata block at the top of a Markdown file. | Markdown Essentials: Front matter (metadata) (p. 21) |
| FRS / SRS | Functional / Software Requirements Specification: detailed, often formal, list of what the system must do. | Business & Requirements Docs: SRS / FRS (Software / Functional Requirements Specification) (p. 41) |

| Term | Definition | More |
|--|---|---|
| GFM | GitHub-Flavored Markdown: CommonMark plus tables, task lists, strikethrough, autolinks, alerts. | Markdown Essentials: GitHub-Flavored Markdown (GFM) extras (p. 20) |
| How-to guide | Goal-oriented steps to solve one specific problem, for a reader who already knows the basics. | Page Patterns: How-to guide (p. 142) |
| llms.txt | A proposed convention: a Markdown file at a site root that lists and summarizes key docs for LLMs. | AI Agent Docs: llms.txt (p. 57) |
| Mermaid | A text-based diagram syntax rendered by GitHub, GitLab and most doc tools. | Diagrams & Visuals (p. 146) |
| NFR (Non-functional requirement) | A quality attribute: performance, security, availability, accessibility, etc. | Business & Requirements Docs: Non-functional requirements (NFRs) (p. 43) |
| OpenAPI | Machine-readable specification format for HTTP/REST APIs (formerly Swagger). | API Docs: Spec-first: OpenAPI (p. 35) |
| Postmortem | A blameless write-up of an incident: timeline, impact, root cause, action items. | Operations Docs: Incident postmortems (p. 49) |
| PRD (Product Requirements Document) | Describes a product or feature: problem, users, scope, requirements, success metrics. | Business & Requirements Docs: PRD (Product Requirements Document) (p. 40) |
| Reference doc | Accurate, complete, structured facts (APIs, config options, CLI flags), meant to be looked up rather than read through. | Page Patterns: Reference page (p. 142) |
| Runbook | Step-by-step operational procedure for a specific situation or alert. | Operations Docs: Runbooks (p. 48) |
| SSOT (Single source of truth) | Each fact is maintained in exactly one place, and everything else links to it. | Principles of Good Documentation: 2. Single source of truth (SSOT) (p. 13) |
| Tutorial | Learning-oriented lesson that takes a beginner through a complete, guaranteed-to-work exercise. | Page Patterns: Tutorial (p. 141) |
| Use case | A structured description of how an actor interacts with the system to reach a goal, including alternative flows. | Business & Requirements Docs: Use cases (p. 42) |
| User story | A short requirement in the format "As a [role], I want [capability], so that [benefit]". | Business & Requirements Docs: User stories (p. 41) |

PART II

Document Types

What kinds of documents exist and when to write each

[doc-types/](#)

| | |
|------------------------------|----|
| Overview | 26 |
| Developer Docs | 29 |
| API Docs | 34 |
| Business & Requirements Docs | 38 |
| Architecture Docs | 44 |
| Operations Docs | 48 |
| User Docs | 51 |
| Process & Team Docs | 54 |
| AI Agent Docs | 56 |

PART II

Overview

What: Every common type of project document: what it's for, who reads it, when it's needed and where it usually lives. **Read when:** you need to decide *what* to write, or *where* a document belongs.

TL;DR

- Pick the **type** before you write. The type decides the structure, tone and location.
- Not every project needs every type. Use the "Needed when" column.
- If a doc doesn't fit any type, it's probably **two docs**, or it isn't needed.

■ Pages in this section

| Page | Covers |
|--|---|
| Developer Docs (p. 29) | README, CONTRIBUTING, getting started, coding conventions, CHANGELOG, module READMEs, code comments vs docs |
| API Docs (p. 34) | REST/OpenAPI, GraphQL, gRPC, events/AsyncAPI, reference vs guides, auth, errors, versioning |
| Business & Requirements Docs (p. 38) | BRD, PRD, SRS/FRS, user stories, use cases, acceptance criteria, business rules, NFRs, domain glossary |
| Architecture Docs (p. 44) | Architecture overview, C4 diagrams, ADRs, RFCs/design docs, data models |
| Operations Docs (p. 48) | Deployment, environments, runbooks, monitoring/alerts, incident postmortems, SLOs, disaster recovery |
| User Docs (p. 51) | Tutorials, how-to guides, FAQ, troubleshooting, release notes, admin guides |
| Process & Team Docs (p. 54) | Onboarding, team handbook, roadmap, meeting notes, decision log, SECURITY.md, CODE_OF_CONDUCT, LICENSE |
| AI Agent Docs (p. 56) | AGENTS.md, CLAUDE.md, rules files, llms.txt, agent skills, agent-generated docs |

■ Master table

| Doc | Family | Audience | Purpose | Needed when | Usual location | Lifecycle |
|-------------------------------|------------------|--------------|---------------------------------------|--------------------|---|-------------|
| <code>README.md</code> (root) | Developer | Everyone | What it is, quick start, map | Always | repo root | Living |
| <code>CONTRIBUTING.md</code> | Developer | Contributors | How to set up, branch, test, open PRs | >1 contributor | repo root | Living |
| <code>CHANGELOG.md</code> | Developer / user | Users, devs | What changed per release | Versioned releases | repo root | Append-only |
| Getting started / setup | Developer | New devs | First run in <30 min | Non-trivial setup | <code>docs/getting-started/</code> | Living |
| Coding conventions | Developer | Devs | Style, patterns, do/don't | >2 devs | <code>docs/development/</code> or <code>CONTRIBUTING</code> | Living |

| Doc | Family | Audience | Purpose | Needed when | Usual location | Lifecycle |
|---|--------------|---------------------|---|------------------------------------|---|--------------------------------------|
| Module README | Developer | Devs | Purpose and usage of a package or service | Monorepo, libraries | next to code | Living |
| API reference | API | API consumers | Every endpoint/field/error | Any API | generated from spec, <code>docs/api/</code> | Living, versioned |
| API guides | API | API consumers | Auth, quick start, common tasks | Any external/shared API | <code>docs/api/guides/</code> | Living |
| BRD | Business | Stakeholders, PM | Business need, goals, scope | Large initiatives, client projects | <code>docs/requirements/</code> or wiki | Approved → frozen |
| PRD | Business | PM, design, eng, QA | What to build and why | Each feature/epic | <code>docs/requirements/</code> or <code>docs/product/</code> | Draft → approved → shipped |
| SRS / FRS | Business | Eng, QA, client | Detailed formal requirements | Contracts, regulated domains | <code>docs/requirements/</code> | Versioned, signed off |
| User stories + AC | Business | Eng, QA | Small testable requirements | Agile delivery | issue tracker (mirrored in repo if needed) | Per sprint |
| Business rules | Business | Product, QA, eng | Canonical rules of the domain | Complex domain logic | <code>docs/requirements/rules/</code> | Living |
| Domain glossary | Business | Everyone | Shared vocabulary | Always worthwhile | <code>docs/glossary.md</code> | Living |
| Architecture overview | Architecture | Devs, architects | Big picture of the system | >1 component | <code>docs/architecture/</code> | Living |
| ADR | Architecture | Devs, future devs | Why a decision was made | Any significant decision | <code>docs/architecture/decisions/</code> | Immutable (superseded, not edited) |
| RFC / design doc | Architecture | Devs, reviewers | Propose before building | Large or risky changes | <code>docs/rfcs/</code> or <code>docs/design/</code> | Draft → accepted/rejected → archived |
| Deployment guide | Operations | DevOps, devs | How to release | Any deployed system | <code>docs/operations/</code> | Living |
| Runbook | Operations | On-call | Fix a specific problem fast | Production systems | <code>docs/operations/runbooks/</code> | Living |
| Postmortem | Operations | Eng, management | Learn from incidents | After incidents | <code>docs/operations/postmortems/</code> | Immutable |
| Tutorial / how-to | User | End users, devs | Learn / do a task | Users exist | <code>docs/guides/</code> or docs site | Living |
| FAQ / troubleshooting | User | Users, support | Answer common problems | Repeated questions | docs site | Living |
| Release notes | User | Users, customers | What's new, in user terms | Customer-facing releases | docs site / <code>docs/releases/</code> | Append-only |
| Onboarding | Process | New team members | First week plan | Growing team | <code>docs/onboarding/</code> or handbook | Living |
| Roadmap | Process | Stakeholders | Planned direction | Product teams | wiki / <code>docs/product/</code> | Living |
| Meeting notes / decision log | Process | Team | Record decisions | Team ceremonies | wiki / <code>docs/meetings/</code> | Append-only |
| <code>SECURITY.md</code> | Process | Security reporters | How to report vulnerabilities | Public repos | repo root or <code>.github/</code> | Living |
| <code>AGENTS.md</code> / <code>CLAUDE.md</code> | AI agent | AI agents | How to work in this repo | Using AI coding agents | repo root (+ subfolders) | Living |

■ How doc families relate

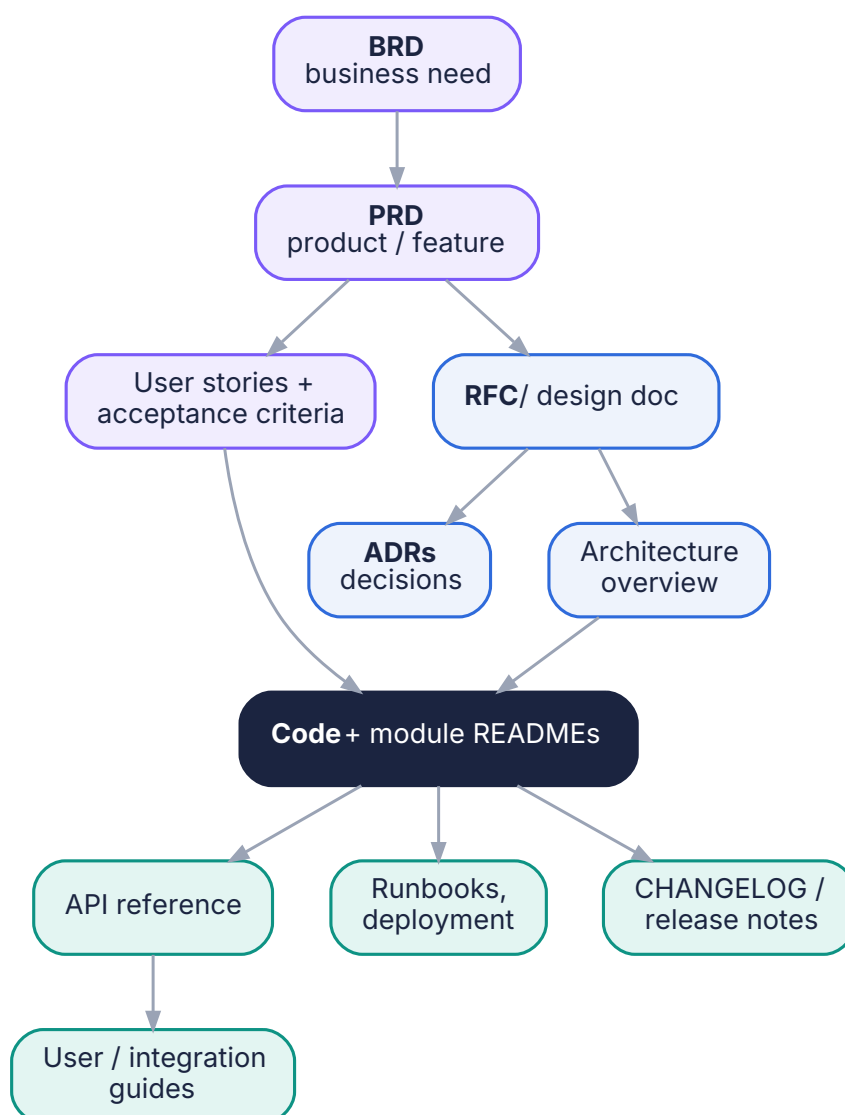


Figure 2.1 How documentation families feed into one another, from business need to release notes.

The flow is: **why** (business) → **what** (product) → **how** (architecture/design) → **built** (code + reference) → **run** (operations) → **use** (user docs).

■ Is this one doc or two? Quick test

Split a document when any of these is true:

- It has **two different audiences** (for example business rules and SQL schema).
- It has **two different lifecycles** (an immutable decision and a living guide).
- It mixes **learning** and **looking up** (tutorial and reference).
- Its table of contents has **more than about 12 H2 sections**.
- People link to **one part** of it far more than the rest.

CHAPTER 2.1

Developer Docs

What: Docs for people who work *on* the codebase: README, CONTRIBUTING, getting started, conventions, CHANGELOG, module READMEs. **Read when:** setting up a repo's docs, onboarding developers, or deciding what goes in the README versus elsewhere.

TL;DR

- The **root README** is the front door. It covers what the project is, how to run it and where everything else is. Keep it short and link out.
- **CONTRIBUTING** explains *how to work here*: setup, branching, tests, PR rules.
- **Getting started** should get a new developer from clone to running in **under 30 minutes**.
- Document **why** and **how to use** in docs, and **what this line does** in code comments, and only when that isn't obvious.
- Put a **module README** in every significant package or service in a monorepo.

■ The developer doc set

| Doc | Required? | Location | Length |
|-------------------------------|--------------------------|--|-----------------------------|
| <code>README.md</code> | Always | repo root | 1–3 screens |
| <code>CONTRIBUTING.md</code> | >1 contributor | repo root or <code>.github/</code> | 1–4 screens |
| <code>CHANGELOG.md</code> | Versioned software | repo root | grows |
| Getting started / local setup | Setup has >3 steps | <code><docs-root>/getting-started/</code> | as needed |
| Coding conventions | >2 developers | <code><docs-root>/development/conventions.md</code> or <code>CONTRIBUTING</code> | 1–5 screens |
| Testing guide | Non-trivial test setup | <code><docs-root>/development/testing.md</code> | |
| Module / package README | Monorepo, libraries | <code>packages/<name>/README.md</code> | 1–2 screens |
| Troubleshooting (dev) | Recurring setup problems | <code><docs-root>/getting-started/troubleshooting.md</code> | |
| <code>.env.example</code> | Uses env vars | repo root | one line per var, commented |

`<docs-root>` is your docs folder (`docs/` , `handbook/` , ...). See [naming the docs root](#) (p. 110).

■ Root README.md

The most-read file in the project. It answers in this order: **What is this?** **Can I use it?** **How do I start?** **Where do I go next?**

Recommended sections

| # | Section | Contents | Required |
|----|------------------------------|---|----------------|
| 1 | Title + one-line description | What it is in one sentence | ✓ |
| 2 | Badges (optional) | Build, coverage, version, license | |
| 3 | Overview | 2–5 sentences or bullets: problem, solution, key features | ✓ |
| 4 | Quick start | Minimum commands to run it | ✓ |
| 5 | Requirements / prerequisites | Runtime versions, tools, accounts | ✓ |
| 6 | Usage | 1–3 core examples | Libraries/CLIs |
| 7 | Project structure | Short tree of top-level folders | Apps |
| 8 | Documentation | Links to the docs index and main sections | ✓ |
| 9 | Contributing | Link to CONTRIBUTING | |
| 10 | License | Name + link | Public repos |

README rules

- ✓ Keep it under about 3 screens, and **link out** for details.
- ✓ The quick start must actually work on a clean machine. Test it.
- ✓ Include a "Documentation" section that links to the docs index.
- ✗ Don't paste the whole API reference, architecture or changelog into it.
- ✗ Don't let it become a FAQ dump. Move that to troubleshooting.

Template: [Root README template](#) (p. 159)

■ CONTRIBUTING.md

Answers "how do I make a change here correctly?"

| Section | Contents |
|---------------------|---|
| Development setup | Link to getting-started, or inline steps if short |
| Branching & commits | Branch naming, commit format (such as Conventional Commits) |
| Code style | Formatter/linter commands; link to conventions doc |
| Testing | How to run tests, coverage expectations |
| Pull requests | Size, description template, review rules, CI checks |
| Documentation | Which docs to update with which changes (see below) |
| Release process | Or link to operations docs |

Include a "docs with code" rule:

Documentation changes

MARKDOWN

Update docs in the same PR when you:

- add or change a public API endpoint → ``docs/api/``
- add an env variable → ``.env.example`` + ``docs/getting-started/configuration.md``

- make an architectural decision → new ADR in `docs/architecture/decisions/`
- change user-visible behavior → `CHANGELOG.md` under "Unreleased"

Template: [CONTRIBUTING template](#) (p. 162)

■ Getting started / local setup

Goal: a new developer runs the project locally in under 30 minutes without asking anyone.

Structure:

1. **Prerequisites**, with exact versions (Node 22.x, PHP 8.3, Docker 27+).
2. **Clone & install**, as copy-pasteable commands.
3. **Configure**: `.env` setup, secrets (where to get them, never the values).
4. **Run**: start command and expected output.
5. **Verify**: "Open `http://localhost:3000`. You should see..."
6. **Next steps**: links to architecture overview, conventions, first good issues.
7. **Troubleshooting**: the 3–5 most common errors and their fixes.

🔔 Re-test it every time onboarding fails, and ask every new hire to fix what was wrong. That keeps it current.

■ Coding conventions

Write down only what tools **can't** enforce. Let formatters and linters handle the rest and just link to their configs.

| Document | Don't document |
|--|---|
| Architecture patterns ("services never call repositories of another module") | Indentation, quotes, semicolons (formatter) |
| Naming of domain concepts | Rules a linter already fails on |
| Error-handling approach | Generic language best practices |
| Where new code of each kind goes | |
| Do/don't examples from <i>this</i> codebase | |

Format each rule as **Rule** → **Why** → **Example** (✅/❌):

```
### Money is always stored in minor units

**Why:** floating-point rounding errors in totals.

```php
// ✓
$order->totalCents = 1999;
// ✗
$order->total = 19.99;
```
```

MARKDOWN

■ Module / package README

In monorepos and multi-package libraries, each significant unit gets its own README:

payments-service

MARKDOWN

Handles card payments and refunds via Stripe.

- **Owner:** team-payments
- **Depends on:** orders-service, PostgreSQL, Stripe API
- **Used by:** checkout-web, admin-panel

Run locally

Configuration

Key concepts

Related docs

- [Refund business rules](../../docs/requirements/payments/refund-rules.md)
- [ADR-0012: Use Stripe](../../docs/architecture/decisions/0012-use-stripe.md)

See [co-located model](#) (p. 82).

■ CHANGELOG.md

A curated, human-readable list of notable changes per version. It is **not** a git log dump.

The **Keep a Changelog** format is widely used:

Changelog

MARKDOWN

All notable changes to this project are documented here.

Format: [Keep a Changelog](https://keepachangelog.com/en/1.1.0/),
versioning: [SemVer](https://semver.org/).

[Unreleased]

Added

- Refund endpoint `POST /v2/refunds`.

[2.3.0] - 2026-09-30

Changed

- Invoices now include tax breakdown.

Fixed

- Duplicate emails on password reset (#482).

Deprecated

- `GET /v1/invoices` (removal planned for 3.0.0).

Categories: Added, Changed, Deprecated, Removed, Fixed, Security.

| CHANGELOG | Release notes |
|-------------------------|--------------------------------------|
| Developers, integrators | End users, customers |
| Complete, technical | Selected highlights, benefit-focused |
| In repo | Docs site, email, in-app |

Can be generated from Conventional Commits (tools: `release-please`, `semantic-release`, `changesets`), but **always review it** before release.

Template: [CHANGELOG template](#) (p. 164)

■ Code comments vs docs

| Put it in... | When it explains... |
|------------------------------------|---|
| Code comment | Why this non-obvious line exists, workarounds, gotchas |
| Docstring / PHPDoc / JSDoc / TSDoc | Public function/class contract: params, return, throws, example |
| Module README | What this module is for, how to use it, how it fits in |
| Docs folder | Cross-cutting topics: architecture, setup, processes, rules |
| ADR | Why a significant decision was made |

Rule of thumb: if the explanation becomes wrong when **this file** changes, put it in a comment. If it becomes wrong when **the design** changes, put it in docs.

RELATED: [API Docs](#) (p. 34) · [Architecture Docs](#) (p. 44) · [AI Agent Docs](#) (p. 56)

CHAPTER 2.2

API Docs

What: How to document APIs (REST, GraphQL, gRPC, events/webhooks) for internal and external consumers. **Read when:** you expose an API to anyone other than the code next to it, including other teams and your own frontend.

TL;DR

- API docs have **two parts: reference** (every endpoint, field and error, ideally *generated* from a spec) and **guides** (quick start, auth, common tasks, written by hand).
- Use a **machine-readable spec** as the single source of truth: OpenAPI (REST), SDL (GraphQL), `.proto` (gRPC), AsyncAPI (events).
- Every endpoint needs a **real request and response example**.
- Document **errors, auth, rate limits, pagination, versioning and deprecation** once, centrally.
- **External** API docs usually deserve their own site or portal, separate from internal docs.

■ The two halves of API docs

| | Reference | Guides |
|-------------------|--|---------------------------------|
| Question answered | "What exactly does <code>POST /refunds</code> accept?" | "How do I issue a refund?" |
| Source | Generated from spec (OpenAPI, SDL, proto) | Handwritten Markdown |
| Organized by | Resource / endpoint / type | Task / use case |
| Completeness | 100%, every field | Common paths only |
| Diátaxis type | Reference | Tutorial + how-to + explanation |

You need both. Reference alone is a dictionary with no grammar. Guides alone leave edge cases undocumented.

■ Recommended API doc set

```

api/
├── README.md           overview: what the API does, base URLs, versions, links
├── getting-started.md  first successful call in 5 minutes
├── authentication.md   how to get and use credentials, scopes
├── concepts/           explanation pages
│   ├── pagination.md
│   ├── idempotency.md
│   ├── errors.md       error format + full error code catalog
│   ├── rate-limits.md
│   └── versioning.md   version policy, deprecation, sunset
├── guides/            task-oriented how-tos
│   ├── create-an-order.md
│   └── handle-webhooks.md
├── reference/         generated (or a link to the rendered spec)
│   └── openapi.yaml    the source of truth
└── webhooks.md        events, payloads, retries, signature verification

```

└─ sdks.md
└─ changelog.md

client libraries + examples
API-specific changes, breaking changes highlighted

■ Spec-first: OpenAPI

Keep the spec in the repo, review it in PRs and generate the reference from it.

```

openapi: 3.1.0
info:
  title: Payments API
  version: 2.3.0
paths:
  /v2/refunds:
    post:
      summary: Create a refund
      description: Refunds all or part of a captured payment. Refunds over €500 require approval.
      operationId: createRefund
      tags: [Refunds]
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/RefundRequest' }
            example: { paymentId: "pay_123", amountCents: 1500, reason: "damaged_item" }
      responses:
        '201': { description: Refund created }
        '409': { description: Payment already fully refunded }

```

YAML

| Approach | How | Best when |
|---------------------------|--|---|
| Spec-first (design-first) | Write the spec, review it, then implement and validate code against the spec | Public APIs, multiple consumers, contract testing |
| Code-first | Annotations/attributes in code generate the spec | Internal APIs, fast iteration, one consumer |

Either way, **commit the resulting spec** and render it with a tool such as Redoc, Swagger UI, Scalar or Stoplight Elements.

Spec formats by API style

| API style | Spec format | Rendered with (examples) |
|--------------------|----------------------------------|-------------------------------------|
| REST / HTTP | OpenAPI 3.x | Redoc, Swagger UI, Scalar |
| GraphQL | Schema SDL + descriptions | GraphiQL, SpectaQL, GraphQL Voyager |
| gRPC | .proto with comments | protoc-gen-doc, Buf Schema Registry |
| Events / messaging | AsyncAPI | AsyncAPI Studio / generator |
| Webhooks | OpenAPI 3.1 webhooks or AsyncAPI | as above |

■ What every endpoint page must include

| Element | Example |
|----------------------|--|
| Method + path | POST /v2/refunds |
| One-sentence purpose | "Refunds all or part of a captured payment." |

| Element | Example |
|------------------------------|--|
| Auth & required scopes | Bearer token, scope refunds:write |
| Path / query / header params | Table: name, type, required, description, constraints |
| Request body schema | Fields, types, required, validation rules |
| Request example | Full, copy-pasteable curl (+ SDK snippets) |
| Response example(s) | Success + the most common errors |
| Status / error codes | Table of codes and what to do about each |
| Side effects | "Sends refund.created webhook", "Emails the customer" |
| Idempotency | Header support, retry safety |
| Rate limits | If different from default |
| Business rules | Link to the canonical rule (Business & Requirements Docs: Business rules catalog (p. 42)) |

Template: [Hand-written API endpoint reference template](#) (p. 177)

Errors

Document the error **format** once and list **every code**:

```
{
  "error": {
    "code": "refund_exceeds_payment",
    "message": "Refund amount 2000 exceeds remaining refundable amount 1500.",
    "requestId": "req_8f3a..."
  }
}
```

JSON

| Code | HTTP | Meaning | What the client should do |
|------------------------|------|------------------|--|
| refund_exceeds_payment | 409 | Amount too large | Lower the amount; fetch payment to see the remaining balance |
| approval_required | 202 | Over €500 | Poll refund status or wait for refund.approved webhook |

The last column is the one most often missing, and the one consumers need most.

Versioning and deprecation

Document your policy explicitly in `concepts/versioning.md`:

- **How versions are expressed:** URL (`/v2/`), header, or date-based (`2026-10-01`).
- **What counts as breaking:** removed fields, new required params, changed types, changed semantics.
- **Deprecation process:** announce → mark deprecated in spec (`deprecated: true`) → Deprecation / Sunset headers → remove after N months.
- **Where changes are announced:** API changelog, email, status page.

For multiple live major versions, see [by-version model](#) (p. 91).

Event-driven APIs

For Kafka/RabbitMQ topics, webhooks and domain events, document:

| Item | Detail |
|----------------------|---------------------------------------|
| Event name & version | <code>order.paid.v1</code> |
| Producer / consumers | Who publishes, who subscribes |
| Channel / topic | <code>orders.events</code> |
| Payload schema | AsyncAPI / JSON Schema / Avro |
| Example payload | Full JSON |
| Delivery guarantees | at-least-once, ordering, retries, DLQ |
| When emitted | The exact business trigger |

An **event catalog** (one page or generated site listing every event) is the API reference of event-driven systems.

■ Internal vs external API docs

| Internal API | External / public API |
|--|---|
| Reference + short README is often enough | Full portal: quick start, guides, SDKs, changelog, status |
| Can live co-located with the service | Separate site or developer portal |
| Can mention internal services | No internal hostnames, architecture or secrets |
| Engineers review | Engineers, technical writer and product review |

See [Internal vs Public Docs](#) (p. 102).

■ API doc quality checklist

- ☐ Spec is in the repo and validated in CI (such as `spectral lint`)
- ☐ Every operation has summary, description, and request + response examples
- ☐ Every error code is listed with a "what to do" column
- ☐ Auth guide gets a newcomer to a first successful call in ≤5 minutes
- ☐ Pagination, rate limits, idempotency documented once and linked
- ☐ Breaking changes are flagged in the API changelog
- ☐ Examples are tested (generated from tests, or run in CI)

RELATED: [Developer Docs](#) (p. 29) · [Business & Requirements Docs](#) (p. 38) · [page-patterns: reference](#) (p. 142)

CHAPTER 2.3

Business & Requirements Docs

What: Documents that define *why* something is built and *what* it must do: BRD, PRD, SRS/FRS, user stories, use cases, acceptance criteria, business rules, NFRs and the domain glossary. **Read when:** starting a project or feature, writing specs with stakeholders, or deciding how formal requirements need to be.

TL;DR

- Requirements go from **broad to specific**: BRD (business why) → PRD (product what) → user stories / use cases (detailed behavior) → acceptance criteria (testable).
- **Choose formality by risk**. Startups need a lean PRD and stories. Contracts and regulated domains need an SRS with IDs and traceability.
- Keep **business rules in one catalog**, give each one an ID, and link to them from stories, code and tests.
- Every requirement must be **testable**. If QA can't verify it, rewrite it.
- Maintain a **domain glossary**. Most requirement bugs are vocabulary bugs.

■ The requirements ladder

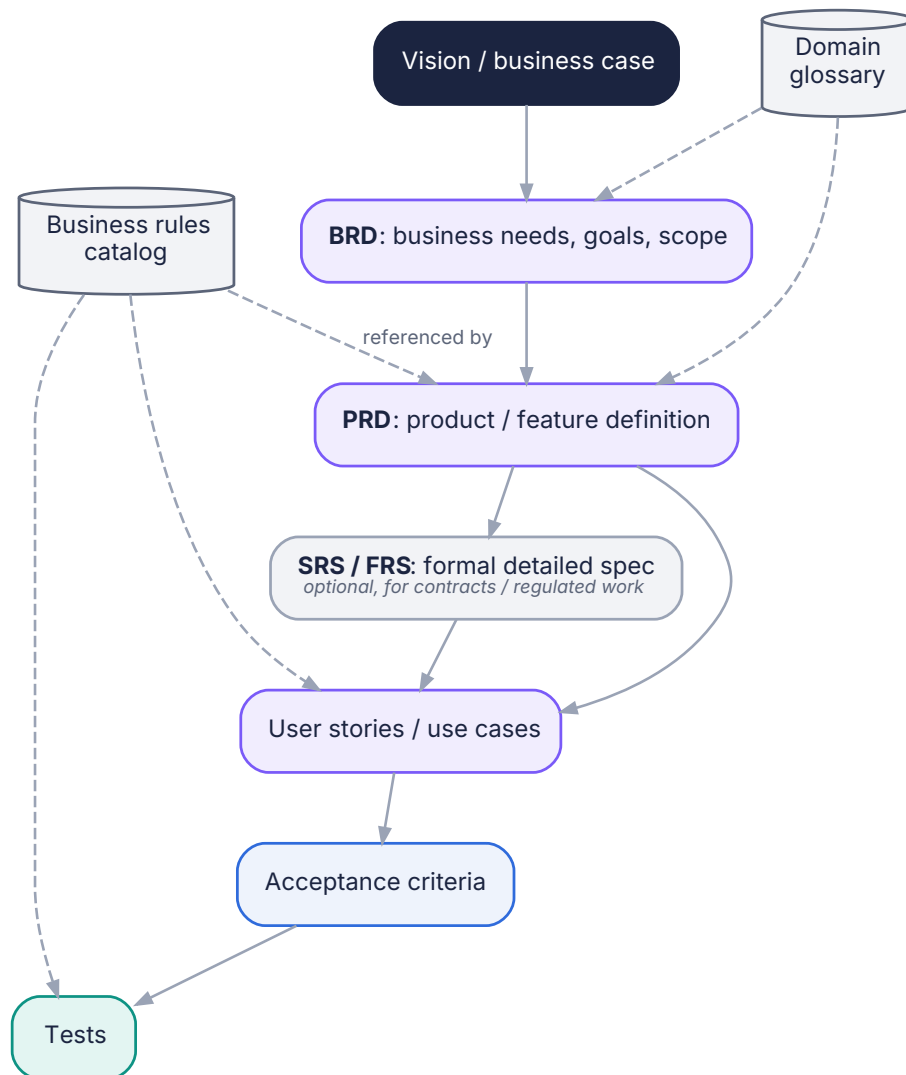


Figure 2.2 The requirements chain. Dashed lines show shared reference material (rules catalog, glossary).

| Level | Document | Answers | Owner | Lifespan |
|-------|--------------------------|--|-----------------------|------------------------------------|
| 1 | BRD | Why do it? What business outcome? | Business sponsor / BA | Frozen after approval |
| 2 | PRD | What are we building, for whom, and how will we know it works? | Product manager | Until feature ships, then archived |
| 3 | SRS / FRS | Exactly what must the system do? (formal) | BA / systems analyst | Versioned, signed off |
| 4 | User stories / use cases | How does each user interaction behave? | PM / team | Per iteration |
| 5 | Acceptance criteria | How do we verify it? | PM + QA | With the story |
| — | Business rules | What are the domain's rules? | Domain owner | Living |
| — | NFRs | How well must it work? | Architect / PM | Living |
| — | Glossary | What do our words mean? | Everyone | Living |

■ Choosing the right level of formality

| Context | Recommended set |
|---|--|
| Solo / early startup | One-page PRD per feature + stories in tracker + glossary |
| Product company, agile teams | PRD per epic + stories with AC + business rules catalog + NFRs |
| Agency / client project | BRD (signed) + PRD/SRS + stories + change log of requirements |
| Enterprise / regulated (finance, health, gov) | BRD + SRS with requirement IDs + traceability matrix + formal sign-off |
| Internal tool | Short PRD or a design doc that includes requirements |

Rule: the more expensive a misunderstanding is, the more formal the document.

■ BRD (Business Requirements Document)

Purpose: justify and frame an initiative in business terms. No solutions or technology.

| Section | Contents |
|---------------------------|--|
| Executive summary | Problem and expected outcome in 3–5 sentences |
| Background / problem | Current situation, pain, evidence (data) |
| Business objectives | Measurable goals (SMART): "Reduce refund handling time from 3 days to 4 hours" |
| Stakeholders | Who is affected, who decides (RACI optional) |
| Scope | In scope / out of scope (explicitly) |
| Business requirements | High-level needs, numbered BR-01 ... |
| Constraints & assumptions | Budget, deadlines, regulation, dependencies |
| Risks | With impact and mitigation |
| Success metrics | How the outcome will be measured |
| Approval | Names, dates |

Template: [Business Requirements Document template](#) (p. 169)

■ PRD (Product Requirements Document)

Purpose: define a product or feature so design, engineering and QA build the same thing.

| Section | Contents |
|---------------------------|---|
| Header | Status, owner, reviewers, target release, links (BRD, designs, epics) |
| Problem statement | User problem, evidence, why now |
| Goals & non-goals | What success is; what this is explicitly not |
| Users / personas | Who it's for |
| User journeys / scenarios | Narrative of key flows |
| Requirements | Functional requirements, prioritized (MoSCoW or P0/P1/P2), with IDs |
| Business rules | Links to the rules catalog (don't redefine rules here) |
| NFRs | Performance, security, accessibility specifics |

| Section | Contents |
|-----------------|--------------------------------|
| Designs | Links to Figma/wireframes |
| Success metrics | KPIs + how they're measured |
| Open questions | With owner and due date |
| Release plan | Phases, feature flags, rollout |

- ✓ A good PRD is 2–6 pages. If it's longer, split it into sub-features.

Template: [Product Requirements Document template](#) (p. 171)

■ SRS / FRS (Software / Functional Requirements Specification)

Purpose: formal, complete, verifiable specification. Typical for contracts, tenders, safety-critical or regulated systems (structure often based on ISO/IEC/IEEE 29148).

Key properties of each requirement: **unique ID, single statement, testable, prioritized, traceable.**

```
### FR-PAY-012: Refund approval threshold
```

MARKDOWN

- ****Statement:**** The system shall require manager approval for refunds above €500.00.
- ****Priority:**** Must
- ****Source:**** BR-04, Finance policy FP-2025-7
- ****Rationale:**** Fraud prevention.
- ****Verification:**** Test, see TC-PAY-044, TC-PAY-045

Use "shall" for mandatory, "should" for desired, and "may" for optional, and use them consistently.

Traceability matrix

| Business req | Functional req | Story | Test |
|--------------|----------------|---------|------------|
| BR-04 | FR-PAY-012 | PAY-231 | TC-PAY-044 |

In Markdown projects this can be a table in `requirements/traceability.md` or be generated from front matter IDs.

■ User stories

Format:

```
**As a** support agent
```

MARKDOWN

```
**I want** to issue a partial refund from the order page
```

```
**so that** I can resolve damaged-item complaints without involving finance.
```

Quality check, INVEST: Independent, Negotiable, Valuable, Estimable, Small, Testable.

Where they live: usually the **issue tracker** (Jira, Linear, GitHub Issues). Mirror them into Markdown only if the repo is the source of truth (for example a client deliverable, or AI-agent-driven development). In that case, store them by feature: `requirements/<feature>/stories/`.

Template: [User story with acceptance criteria template](#) (p. 173)

■ Acceptance criteria

Use **Given / When / Then** (Gherkin style) for behavior, or a checklist for simple cases:

```
Scenario: Refund above approval threshold
  Given a captured payment of €800
  And I am a support agent
  When I request a refund of €600
  Then the refund status is "pending_approval"
  And a finance manager receives an approval request
```

GHERKIN

✓ Cover the happy path, the main alternatives, the edge cases (zero, max, duplicates) and the error cases. ✗
Don't write "works correctly", "is fast" or "user-friendly". None of those can be tested.

■ Use cases

More detailed than stories. Use them when flows have many branches or several actors.

| Field | Example |
|-----------------------|------------------------------------|
| ID / name | UC-07 Issue refund |
| Primary actor | Support agent |
| Preconditions | Payment is captured |
| Main success scenario | 1. Agent opens order... 2. ... |
| Alternative flows | 3a. Amount > €500 → approval flow |
| Exceptions | Payment provider unavailable |
| Postconditions | Refund recorded, customer notified |
| Business rules | BR-PAY-04 |

Template: [Use case template](#) (p. 175)

■ Business rules catalog

Business rules are the **most reused and most duplicated** knowledge in a project. Give them **one home**, give each an **ID**, and **link** to them from PRDs, stories, code and tests.

```
ID	Rule	Source	Owner	Since
BR-PAY-04	Refunds over €500 require finance manager approval.	Finance policy FP-2025-7	Finance	2025-03
BR-PAY-05	Refunds are allowed up to 90 days after capture.	Terms of service §8	Legal	2024-01
```

MARKDOWN

Organize by domain: `requirements/rules/payments.md`, `requirements/rules/shipping.md`. In code and tests, reference the ID:

```
// BR-PAY-04: refunds over €500 require approval
```

PHP

That makes "where is this rule implemented?" answerable with a single grep.

■ Non-functional requirements (NFRs)

Quality attributes, always **measurable**:

| Category | ❌ Vague | ✅ Measurable |
|---------------|--------------------|--|
| Performance | "Fast checkout" | "p95 checkout API latency < 300 ms at 200 req/s" |
| Availability | "Highly available" | "99.9% monthly uptime for checkout" |
| Security | "Secure" | "PII encrypted at rest (AES-256); OWASP ASVS L2" |
| Accessibility | "Accessible" | "WCAG 2.2 AA for all customer screens" |
| Scalability | "Scales" | "Handles 10× Black Friday peak (≈5k orders/min)" |

Store system-wide NFRs in one doc (`requirements/non-functional.md`). Put feature-specific ones in the PRD.

■ Domain glossary

A table of terms with precise definitions and the code names they map to:

| Term | Definition | In code | Not to be confused with |
|-----------------------|---|-----------------------------------|-------------------------|
| Capture | Moving authorized funds from the customer | <code>Payment::capture()</code> | Authorization |
| Partial refund | Refund of less than the captured amount | <code>Refund(type=partial)</code> | Chargeback |

Put it at `<docs-root>/glossary.md` and link it from README. It's one of the most valuable docs for new team members **and** for AI agents.

■ Where business docs should live

| Situation | Recommendation |
|--------------------------------------|--|
| Engineers and product both use Git | Markdown in repo: <code>requirements/</code> organized by domain (p. 73) |
| Business authors don't use Git | Wiki/Notion as source of truth; link from the repo, don't copy |
| Client deliverable / sign-off needed | Markdown → PDF export with version and approval table, or a formal document system |
| Specs drive AI-assisted development | Markdown in repo (agents can read it) with stable requirement IDs |

Lifecycle tip: PRDs and BRDs go **stale after shipping**. Move them to `archive/` or mark them `status: shipped`, and keep the **rules catalog** and **glossary** as the living truth. See [by-lifecycle model](#) (p. 79).

RELATED: [Architecture Docs](#) (p. 44) · [organizing by domain](#) (p. 73) · [PRD template](#) (p. 171)

CHAPTER 2.4

Architecture Docs

What: Docs that explain how the system is structured and why: architecture overview, C4 diagrams, ADRs, RFCs/design docs and data models. **Read when:** the system has more than one moving part, or you're making a decision someone will later ask "why?" about.

TL;DR

- Keep **one living architecture overview** that covers the big picture in 1–3 pages with diagrams.
- Use **C4 levels** to draw diagrams at the right zoom: Context → Containers → Components.
- Record each significant decision as an **ADR**: short, numbered and immutable. Supersede it rather than editing it.
- Propose big changes **before** building them in an **RFC / design doc**.
- Prefer **diagrams as code** (Mermaid, Structurizr, PlantUML) so they're reviewable and diffable.

■ The architecture doc set

| Doc | Living or immutable | Location |
|---------------------------|--|--|
| Architecture overview | Living | <code>architecture/README.md</code> or <code>architecture/overview.md</code> |
| C4 diagrams | Living | <code>architecture/diagrams/</code> or embedded in overview |
| ADRs | Immutable (status changes only) | <code>architecture/decisions/NNNN-title.md</code> |
| RFCs / design docs | Immutable after decision | <code>rfcs/</code> or <code>architecture/design/</code> |
| Data model / ERD | Living | <code>architecture/data-model.md</code> |
| Integration map | Living | <code>architecture/integrations.md</code> |
| Quality attributes / NFRs | Living | link to requirements NFRs (p. 43) |

■ Architecture overview

One page that a new senior engineer reads on day one.

| Section | Contents |
|------------------------|---|
| Purpose & context | What the system does and who/what it talks to (C4 Context diagram) |
| Building blocks | Main apps/services/databases (C4 Container diagram) + one line each |
| Key flows | 2–4 sequence diagrams for the most important journeys |
| Technology stack | Table: layer → tech → why (link ADRs) |
| Cross-cutting concerns | Auth, logging, error handling, config, i18n |
| Deployment view | Where it runs (or link to operations docs) |
| Key decisions | List of the most important ADRs |

| Section | Contents |
|-------------------------------|-------------|
| Known limitations / tech debt | Honest list |

A popular formal alternative is **arc42**, a 12-section template. It's useful for large or regulated systems, but too much for most small projects.

■ C4 model

Diagram at four zoom levels. Most projects only need levels 1 and 2, plus level 3 for complex parts.

| Level | Shows | Audience | Needed |
|--------------------------|--|-----------------------------------|------------------------|
| 1. System Context | Your system as a box, users and external systems around it | Everyone, including non-technical | Always |
| 2. Container | Deployable/runnable units: web app, API, DB, queue | Engineers, ops | Almost always |
| 3. Component | Major components inside one container | Engineers of that container | For complex containers |
| 4. Code | Classes/tables | Rarely worth maintaining by hand | Generate if needed |

```

flowchart LR
    customer([Customer]) -->|browses, pays| web[Web Shop<br/>Next.js]
    web -->|JSON/HTTPS| api[Shop API<br/>Laravel]
    api --> db[(PostgreSQL)]
    api -->|charges| stripe[Stripe]
    api -->|events| queue[(RabbitMQ)]
    queue --> mailer[Mailer worker]
  
```

MERMAID

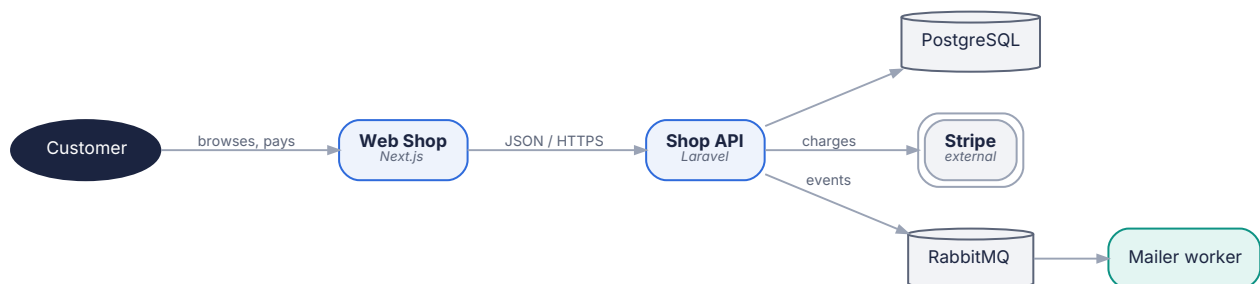


Figure 2.3 The Mermaid C4 context example above, rendered.

More diagram guidance: [Diagrams & Visuals](#) (p. 146).

■ Architecture Decision Records (ADRs)

An ADR captures **one** decision: the context, what was decided and what follows from it.

Rules:

- Number them sequentially: `0001-record-architecture-decisions.md`, `0002-use-postgresql.md`.
- Keep them **short**, 1–2 pages.

- Make them **immutable** once accepted. To change a decision, write a new ADR that **supersedes** the old one, and update the old one's status only.
- Statuses: `proposed` → `accepted` | `rejected` → `later deprecated` | `superseded` by `ADR-00NN`.
- Keep an index (`decisions/README.md`) listing number, title, status and date.

Write an ADR when the decision:

- is hard or expensive to reverse (database, framework, cloud, sync vs async),
- affects several teams or modules,
- was controversial or had serious alternatives,
- will make a future developer ask "why on earth did they...?"

```
# ADR-0012: Use Stripe for card payments

- **Status:** accepted
- **Date:** 2026-03-14
- **Deciders:** @anna, @li, team-payments

## Context
We need card payments in 6 EU countries by Q3. Team has no PCI expertise...

## Decision
We will use Stripe Payments with Stripe Elements on the frontend.

## Alternatives considered
- Adyen: better rates at scale, longer onboarding (8+ weeks).
- In-house via acquirer: requires PCI DSS Level 1, out of reach.

## Consequences
- ✓ PCI scope reduced to SAQ A.
- ✗ Vendor lock-in on payment method storage.
- Follow-up: abstract provider behind `PaymentGateway` interface.
```

MARKDOWN

Formats: **Nygaard** (original: Context/Decision/Consequences), **MADR** (adds options and pros/cons), **Y-statements** (one-sentence format). Pick one and template it.

Template: [Architecture Decision Record \(Nygaard style + alternatives\) template](#) (p. 166) · Organization pattern: [Model: Sequential & Dated Records](#) (p. 88)

■ RFCs and design docs

An RFC or **design doc** proposes a solution *before* building it, so others can find problems while they're still cheap to fix.

| | ADR | RFC / design doc |
|---------|---------------------------------|---|
| When | At or after the decision | Before the decision |
| Size | 1–2 pages | 3–15 pages |
| Scope | One decision | A whole feature/system change, may produce several ADRs |
| Content | Context, decision, consequences | Problem, goals, proposed design, alternatives, rollout, risks |

Typical design doc sections

1. Summary (one paragraph)
2. Context and problem

3. Goals / non-goals
4. Proposed design: diagrams, APIs, data model changes
5. Alternatives considered
6. Risks, security and privacy, performance
7. Migration / rollout / rollback plan
8. Testing and observability
9. Open questions
10. Decision record (outcome, date, links to ADRs)

RFC lifecycle

draft → in-review → accepted | rejected | withdrawn → implemented → archived

Store by number (`rfcs/0042-multi-currency.md`) with status in front matter, or by status folder. See [Model: Organize by Lifecycle / Status](#) (p. 79).

Template: [RFC / design doc template](#) (p. 167)

■ Data model docs

- An ERD (Mermaid `erDiagram`) for core entities. Don't include every table.
- A **table of entities** with meaning, owner module and key invariants.
- **Generate** a full schema doc from the database or migrations if needed (for example `tbls`, `SchemaSpy`). Don't hand-maintain it.

```
erDiagram
    ORDER ||--o{ ORDER_LINE : contains
    ORDER ||--o{ PAYMENT : "paid by"
    PAYMENT ||--o{ REFUND : "refunded by"
```

MERMAID

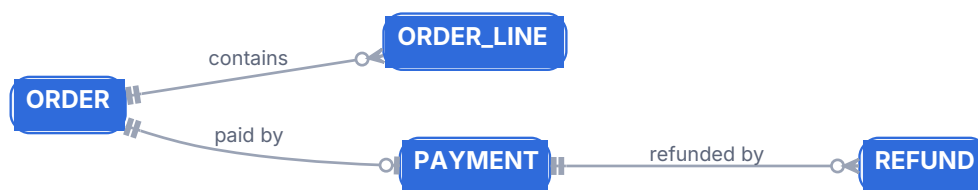


Figure 2.4 The ERD example above, rendered (crow's-foot notation).

RELATED: [Business & Requirements Docs](#) (p. 38) · [Operations Docs](#) (p. 48) · [Diagrams & Visuals](#) (p. 146)

CHAPTER 2.5

Operations Docs

What: Docs for running the system in production: environments, deployment, configuration, monitoring, runbooks, incident postmortems, SLOs and disaster recovery. **Read when:** the project is deployed anywhere other than a laptop, and certainly before the first on-call shift.

TL;DR

- Ops docs are read **under stress**. Make them short, numbered and copy-pasteable, with the critical step first.
- Name runbooks after the **alert or symptom** ("High 5xx on checkout"), not the system.
- Every alert should link to **its runbook**.
- Write **blameless postmortems** after incidents, with owned action items.
- **Never** put secrets in docs. Document *where* secrets live and *who* can access them.

■ The operations doc set

| Doc | Purpose | Location |
|----------------------------|--|---|
| Environments | What environments exist, URLs, differences, access | <code>operations/environments.md</code> |
| Deployment / release | How code gets to production, how to roll back | <code>operations/deployment.md</code> |
| Configuration | All config/env vars, defaults, where set | <code>operations/configuration.md</code> (or generated from <code>.env.example</code>) |
| Monitoring & alerts | Dashboards, alerts, what each alert means | <code>operations/monitoring.md</code> |
| Runbooks | Step-by-step response per alert/situation | <code>operations/runbooks/</code> |
| Postmortems | Learn from incidents | <code>operations/postmortems/YYYY-MM-DD-title.md</code> |
| SLOs / SLAs | Reliability targets and error budgets | <code>operations/slos.md</code> |
| Backup & disaster recovery | Backup schedule, restore procedure, RTO/RPO | <code>operations/disaster-recovery.md</code> |
| Access & on-call | Who is on call, escalation path, access requests | <code>operations/on-call.md</code> |
| Infrastructure | IaC layout, cloud accounts, network overview | <code>operations/infrastructure.md</code> or co-located with IaC code |

■ Runbooks

A runbook turns tribal knowledge into a procedure anyone on-call can follow.

Structure

| Section | Contents |
|-------------------|--|
| Title | The alert name or symptom, exactly as shown in the alerting tool |
| Severity & impact | What users experience |
| Quick mitigation | The 1–3 actions that stop the bleeding, first |
| Diagnosis | Dashboards, queries, log searches, with links and exact commands |
| Resolution steps | Numbered, copy-pasteable, with expected output |
| Verification | How to confirm it's fixed |
| Escalation | Who to page and when |
| Background | Optional: why this happens, link to architecture |

```
# Alert: CheckoutHigh5xxRate
```

MARKDOWN

```
**Impact:** Customers cannot pay. **Severity:** SEV-1.
```

```
## Quick mitigation
```

1. Check the status page of Stripe: <https://status.stripe.com>
2. If a deploy happened in the last 30 min, roll back:

```
```bash
./scripts/rollback.sh checkout-api
```
```

```
## Diagnosis
```

- Dashboard: Grafana → "Checkout / Errors"
- Logs: ``service:checkout-api status:5xx`` in the last 15 min
- ...

Runbook rules

-  **Mitigation before diagnosis.** Restore service first, investigate later.
-  Commands are exact and safe to copy. Mark destructive ones with .
-  Test runbooks in game days or drills.
-  Link every alert to its runbook (most alerting tools support a `runbook_url` field).
-  "Investigate the logs" isn't a step. Say *which* logs and *what* to look for.

Template: [Runbook template](#) (p. 179)

Deployment docs

Cover: the pipeline overview (diagram), how to release, feature flags, database migrations, **rollback procedure**, release checklist and freeze periods. If the pipeline is fully automated, document what to do **when it fails**.

Incident postmortems

Blameless: focus on systems and processes, not on who made a mistake.

| Section | Contents |
|---------|--|
| Summary | What happened, in 3 sentences |
| Impact | Duration, users affected, revenue/SLO impact |

| Section | Contents |
|------------------------|--|
| Timeline | Timestamped events (detection → mitigation → resolution) |
| Root cause(s) | Technical + contributing factors (5 Whys or similar) |
| What went well / badly | Detection, response, tooling |
| Action items | Table: action, owner, ticket, due date |

Naming: `postmortems/2026-09-18-checkout-outage.md`. Postmortems are **immutable records**. Track action items in the issue tracker. See [Model: Sequential & Dated Records](#) (p. 88).

Template: [Blameless incident postmortem template](#) (p. 181)

■ Configuration docs

Keep one canonical list, ideally generated from or checked against `.env.example` / config schema:

| Variable | Required | Default | Description | Secret? |
|----------------------------|----------|---------|------------------------------|---------------------------------|
| <code>DATABASE_URL</code> | ✓ | — | PostgreSQL connection string | ✓ (Vault: <code>app/db</code>) |
| <code>QUEUE_WORKERS</code> | | 4 | Number of worker processes | |

■ Where ops docs live

| Option | Good for |
|---|--|
| In the app repo (<code>operations/</code>) | Single service, team owns code and ops |
| Co-located with IaC (<code>infra/README.md</code> , <code>infra/runbooks/</code>) | Platform/infra teams |
| Central ops/SRE handbook repo | Many services, shared on-call, cross-service runbooks |
| Incident tool / wiki | Postmortems, if the org standardizes there; link from repo |

⚠ Make sure runbooks are reachable **when production is down**. If your docs site is hosted on the same infrastructure, keep a copy in Git, which can still be read from GitHub/GitLab.

RELATED: [Architecture Docs](#) (p. 44) · [Developer Docs](#) (p. 29) · [Storage Locations](#) (p. 99)

CHAPTER 2.6

User Docs

What: Docs for people who *use* the product: tutorials, how-to guides, feature reference, FAQ, troubleshooting, release notes and admin guides. **Read when:** your product has users outside the dev team, whether customers, internal staff or developers using your library.

TL;DR

- Organize user docs by **user task**, not by your internal module structure.
- **Diátaxis** works especially well here: tutorials, how-to guides, reference, explanation.
- Use the **user's vocabulary**, with screenshots or examples for every non-trivial step.
- **Release notes** tell users what changed *for them*. That makes them different from the CHANGELOG.
- User docs usually belong on a **docs site** or help center, built from Markdown if possible.

■ User doc types

| Type | Purpose | Example title |
|------------------------------|--|--|
| Tutorial | Teach a beginner through a complete first experience | "Build your first invoice in 10 minutes" |
| How-to guide | Solve one specific task | "Issue a partial refund" |
| Feature reference | Describe every setting/field/option | "Refund settings" |
| Concept / explanation | Explain how something works and why | "How refund approvals work" |
| FAQ | Quick answers to repeated questions | "Can I refund after 90 days?" |
| Troubleshooting | Symptom → cause → fix | "Refund button is greyed out" |
| Release notes | What's new and what changed, in user terms | "October 2026 release" |
| Admin guide | Setup and management for admins | "Configure approval thresholds" |
| Installation guide | For self-hosted/downloadable products | "Install on Ubuntu 24.04" |

How each is shaped: [Page Patterns](#) (p. 141).

■ Organizing user docs

Recommended: Diátaxis at the top, product areas inside.

```

user-guide/
├── README.md           landing: who it's for, popular tasks, search tips
├── tutorials/
│   └── first-invoice.md
├── how-to/
│   ├── billing/
│   │   ├── issue-refund.md
│   │   └── change-plan.md
│   └── users/

```

```

└─ invite-teammate.md
└─ reference/
    └─ settings.md
    └─ keyboard-shortcuts.md
└─ concepts/
    └─ refund-approvals.md
└─ troubleshooting.md
└─ faq.md
└─ release-notes/
    └─ 2026-10.md
    └─ 2026-09.md

```

Alternative for large products: **product area at the top**, Diátaxis inside each area. See [diátaxis model](#) (p. 76) and [Model: Hybrid \(Combining Models\)](#) (p. 96).

■ Writing for end users

✓ Do

Use names from the UI exactly: **Settings** → **Billing**

Start with the goal: "To refund an order..."

One action per numbered step

Show the expected result after key steps

Screenshots for complex UI, cropped and annotated

Mention required permissions/plan upfront

✗ Don't

Use internal names: "the BillingConfigService"

Start with background

Combine three clicks in one sentence

Leave the user guessing whether it worked

Full-screen screenshots of everything

Let users discover at step 7 they're not allowed

■ Release notes vs CHANGELOG

| | Release notes | CHANGELOG |
|---------|---|-----------------------------|
| Reader | Users, customers, support, sales | Developers, integrators |
| Content | Highlights, benefits, how to use new features | Every notable change |
| Tone | Friendly, benefit-first | Concise, technical |
| Format | Grouped by theme: New, Improved, Fixed | Keep a Changelog categories |

October 2026

MARKDOWN

✨ New: Partial refunds

You can now refund part of an order directly from the order page.

[Learn how →](../how-to/billing/issue-refund.md)

✨ Improved

- Invoice PDFs now show the tax breakdown.

🐛 Fixed

- Password reset emails were sometimes sent twice.

■ Tooling

| Need | Options (examples) |
|-------------------------|---|
| Docs site from Markdown | MkDocs (Material), Docusaurus, VitePress, Starlight (Astro), mdBook |

| Need | Options (examples) |
|--------------------|---|
| Hosted help center | Intercom, Zendesk Guide, GitBook, Mintlify, ReadMe |
| Versioned docs | Docusaurus versioning, <code>mike</code> for MkDocs |
| Search | Built-in search, Algolia DocSearch |

Prefer tools that read **plain Markdown from Git**, so user docs stay docs-as-code.

RELATED: [Page Patterns](#) (p. 141) · [diataxis model](#) (p. 76) · [Internal vs Public Docs](#) (p. 102)

CHAPTER 2.7

Process & Team Docs

What: Docs about how the team works: onboarding, team handbook, roadmap, meeting notes, decision logs, and community/governance files (SECURITY, CODE_OF_CONDUCT, LICENSE, CODEOWNERS). **Read when:** the team grows past a few people, the project goes open source, or decisions keep getting lost.

TL;DR

- **Onboarding** is a checklist plus links. It doesn't duplicate other docs.
- Keep **decisions** in a searchable log. Meeting notes alone aren't enough.
- Standard **community files** have fixed names and locations, and GitHub/GitLab recognize them.
- Team/process docs often fit a **wiki or handbook repo** better than the product repo.

■ Process doc set

| Doc | Purpose | Usual location |
|----------------------------|---|---|
| Onboarding guide | New member's first days/weeks | <code>onboarding/</code> or team handbook |
| Team handbook | Working agreements, rituals, roles, tools | separate <code>handbook</code> repo or wiki |
| Roadmap | Planned direction and priorities | wiki / product tool / <code>product/roadmap.md</code> |
| Meeting notes | Record of discussions | wiki or <code>meetings/YYYY-MM-DD-topic.md</code> |
| Decision log | Non-architectural decisions (process, product, org) | <code>decisions.md</code> or <code>decisions/</code> |
| Definition of Done / Ready | Shared quality bar | CONTRIBUTING or handbook |
| Release process | Who releases, how, when | operations docs |
| Glossary | Shared vocabulary | <code><docs-root>/glossary.md</code> |

■ Standard repository files

These have **conventional names and locations**. Platforms detect them and link them in the UI.

| File | Purpose | Location |
|---------------------------------|---|---|
| <code>README.md</code> | Front door | root |
| <code>LICENSE</code> | Legal terms | root |
| <code>CONTRIBUTING.md</code> | Contribution guide | root, <code>.github/</code> or <code>docs/</code> |
| <code>CODE_OF_CONDUCT.md</code> | Community behavior rules | root or <code>.github/</code> |
| <code>SECURITY.md</code> | How to report vulnerabilities, supported versions | root or <code>.github/</code> |
| <code>SUPPORT.md</code> | Where to get help | root or <code>.github/</code> |
| <code>CHANGELOG.md</code> | Version history | root |

| File | Purpose | Location |
|--|--|-------------------------------|
| <code>CODEOWNERS</code> | Who reviews which paths (works for docs ownership too) | <code>.github/</code> or root |
| <code>GOVERNANCE.md</code> / <code>MAINTAINERS.md</code> | Decision-making, maintainers (OSS) | root |
| <code>.github/PULL_REQUEST_TEMPLATE.md</code> | PR description template | <code>.github/</code> |
| <code>.github/ISSUE_TEMPLATE/</code> | Issue forms | <code>.github/</code> |

- ✓ Keep **UPPERCASE** names for these. That convention makes them stand out and lets tools find them.

■ Onboarding guide

Structure as a **time-boxed checklist**:

```
# Onboarding

## Day 1
- [ ] Get access: GitHub org, Slack #team-payments, Vault (ask @lead)
- [ ] Run the project locally: [Getting started](../getting-started/README.md)
- [ ] Read: [Architecture overview](../architecture/README.md) (30 min)

## Week 1
- [ ] Read the [glossary](../glossary.md) and [top 10 ADRs](../architecture/decisions/README.md)
- [ ] Ship a small fix (label `good-first-issue`)
- [ ] Shadow an on-call shift

## Who to ask
Topic	Person/channel
Payments domain	@anna, #payments
```

MARKDOWN

■ Decision log (non-architectural)

ADRs cover technical decisions. Other decisions (process, product scope, tooling) also get lost. A lightweight log works:

```
Date	Decision	Context / link	Decided by
2026-09-02	Move standup to async in Slack	[notes](meetings/2026-09-02-retro.md)	team
2026-08-20	Drop IE11 support	[PRD-14](../product/prd-14.md)	product + eng
```

MARKDOWN

■ Meeting notes

- Name them `YYYY-MM-DD-topic.md` so they sort chronologically.
- Always include a **Decisions** section and an **Action items** section (owner + due date).
- Copy decisions into the decision log, because nobody re-reads old meeting notes.

RELATED: [Developer Docs](#) (p. 29) · [sequential-and-dated model](#) (p. 88) · [Storage Locations](#) (p. 99)

CHAPTER 2.8

AI Agent Docs

What: Docs written specifically for AI coding agents and LLM tools: `AGENTS.md`, `CLAUDE.md`, tool-specific rules files, `llms.txt`, agent skills, and where to keep agent-generated documents. **Read when:** your team uses AI coding assistants, or you want LLMs to understand your project or product docs correctly.

TL;DR

- Put an `AGENTS.md` (or the tool's file, such as `CLAUDE.md`) at the repo root. It covers commands, conventions, boundaries and a map of the docs.
- Keep it **short and router-like**. Link to existing docs instead of duplicating them.
- Add **nested agent files** in subfolders for area-specific rules (monorepos).
- Make **human docs agent-friendly** too: explicit, structured, consistent paths. See [Writing for AI Agents](#) (p. 149).
- Give **agent-generated docs** (plans, reports, scratch notes) their **own folder**, separate from human-curated docs.

Agent instruction files

| File | Read by | Notes |
|--|--|---|
| <code>AGENTS.md</code> | 20+ agents: Codex, Amp, Jules, Cursor, Factory, GitHub Copilot, Zed, Warp, opencode, others (some, like Aider and Gemini CLI, need config) | Tool-neutral open format, stewarded under the Linux Foundation. Current list: agents.md |
| <code>CLAUDE.md</code> | Claude Code | Supports nested files and <code>@path</code> imports. Put <code>@AGENTS.md</code> in it to reuse the shared file. |
| <code>.cursor/rules/*.mdc</code> | Cursor | Rules with glob scoping |
| <code>.github/copilot-instructions.md</code> | GitHub Copilot | Repo-wide. Path-specific rules go in <code>.github/instructions/**/*.instructions.md</code> with an <code>applyTo:</code> glob. |
| <code>GEMINI.md</code> | Gemini CLI | Context file name is configurable (can point at <code>AGENTS.md</code>) |
| <code>.windsurf/rules/</code> | Windsurf | |

Avoiding duplication across tools: keep the content in `AGENTS.md`. Make the tool-specific files a thin pointer (for example `CLAUDE.md` containing `@AGENTS.md`) or a symlink, or generate them from one source.

⚠ Tool support changes quickly. Check each tool's docs before relying on a file name.

What to put in AGENTS.md

| Section | Example contents |
|-----------------|--|
| Project summary | 2–3 sentences: what, stack, main folders |

| Section | Example contents |
|------------------|--|
| Commands | Install, run, test, lint, build, exact commands |
| Code conventions | Only the non-obvious ones, or a link to the conventions doc |
| Architecture map | Where things live: "Domain logic in <code>src/Domain/</code> , never in controllers" |
| Docs map | "Business rules: <code>docs/requirements/rules/</code> . ADRs: <code>docs/architecture/decisions/</code> " |
| Boundaries | What not to touch: generated files, migrations already applied, secrets, vendor dirs |
| Workflow rules | "Run tests before claiming done", "Update CHANGELOG under Unreleased" |
| Safety | Destructive operations need confirmation, never commit <code>.env</code> |

Rules for agent files

- ✔ **Short.** Agents load it on every task, so long files waste context and dilute attention. Aim for under 200 lines and link out.
- ✔ **Imperative and specific:** "Use `pnpm`, not `npm`" beats "We prefer modern tooling".
- ✔ **Commands in code blocks**, exactly as they should be run.
- ✔ **Nested files** for subprojects: `apps/web/AGENTS.md` adds rules for that area only.
- ✗ Don't paste the architecture doc into it. **Link** to it.
- ✗ Don't include secrets, tokens or internal URLs you wouldn't commit anyway.

Template: [AGENTS template](#) (p. 186)

■ llms.txt

A proposed convention (llmstxt.org) for **published docs sites**: a Markdown file at `/llms.txt` that gives LLMs a curated index of the most important pages. Format: an H1 name, a blockquote summary, then H2 sections that list links with one-line descriptions. An `## Optional` section marks content that can be skipped.

Many sites also publish `/llms-full.txt` with the full text of all pages inline. That's a widely used de facto convention, not part of the original proposal. Adoption by the major AI providers isn't confirmed, so treat both files as a cheap extra, not a guarantee.

```
# Payments API
```

MARKDOWN

```
> Payments API for EU merchants: card payments, refunds, payouts.
```

```
## Docs
```

- [Quick start](https://docs.example.com/quick-start.md): first payment in 5 minutes
- [Authentication](https://docs.example.com/auth.md): API keys and scopes
- [Refunds](https://docs.example.com/refunds.md): full and partial refunds, approval rules

```
## Optional
```

- [ChangeLog](https://docs.example.com/changelog.md)

Relevant for **public product or API docs**. For in-repo docs, `AGENTS.md` plus a good docs index does the same job.

■ Agent skills, commands and rules folders

Many agent tools support reusable instruction packs, such as Claude Code skills in `.claude/skills/<name>/SKILL.md` and slash commands in `.claude/commands/`. Treat them like code:

- One folder per skill/command, with a clear name and description.

- Version them in Git and review them in PRs.
- Keep **project knowledge** (business rules, architecture) in the normal docs and have skills **link** to it. Skills hold **procedures**, not facts.

■ Where to put agent-generated documents

Agents produce plans, specs, research notes, progress reports and session logs. Mixing these into the human docs folder causes **noise and staleness**.

| Kind | Recommended location | Commit? |
|--|--|----------------------------------|
| Agent instructions (curated) | AGENTS.md ,
CLAUDE.md , .claude/ , .cursor/ | ✓ |
| Implementation plans / specs being worked on | .agents/plans/ , planning/ ,
work/specs/ | ✓ (team-visible) or ✗ (personal) |
| Phase reports / decision logs from agent workflows | .agent-toolkit/reports/ , .agents/
reports/ | ✓ if they record decisions |
| Scratch notes, research dumps, session logs | .agents/scratch/ (git-ignored) or system
temp | ✗ |
| Final, human-reviewed docs | normal docs root (docs/ , handbook/ , ...) | ✓ |

Principles:

- ✓ Use a **dedicated, clearly named folder** for agent working documents (.agents/ , planning/ , .ai/), **not** the human docs root.
- ✓ **Promote** agent output into the docs root only after human review, for example a plan becoming an ADR or a design doc.
- ✓ Add a header to agent-written files: > Generated by an AI agent on 2026-10-09. Status: draft, not reviewed.
- ✓ Git-ignore scratch output so it doesn't pollute history.
- ✗ Don't let agents write into docs/ freely without review. It quickly becomes a junk drawer.

Example layout:

```

repo/
├── AGENTS.md           curated instructions (committed)
├── CLAUDE.md          "@AGENTS.md" import + Claude-specific bits
├── .claude/
│   ├── skills/
│   └── commands/
├── .agents/
│   ├── plans/         active implementation plans (committed)
│   ├── reports/       workflow decision logs (committed)
│   └── scratch/       ignored by .gitignore
└── docs/              human-curated documentation only

```

RELATED: [Writing for AI Agents](#) (p. 149) · [Developer Docs](#) (p. 29) · [Navigation & Metadata](#) (p. 113)

PART III

Organizing Docs

Where to store docs and how to structure folders

organizing/

| | |
|---|-----|
| Overview | 60 |
| The Six Dimensions of Doc Organization | 63 |
| Organizing Models | 67 |
| Model: Organize by Doc Type | 68 |
| Model: Organize by Audience | 70 |
| Model: Organize by Domain / Feature | 73 |
| Model: Diátaxis (by User Need) | 76 |
| Model: Organize by Lifecycle / Status | 79 |
| Model: Co-located with Code | 82 |
| Model: Per-Service Repos + Central Portal | 85 |
| Model: Sequential & Dated Records | 88 |
| Model: Organize by Version | 91 |
| Model: Flat Structure + Metadata (Wiki-style) | 93 |
| Model: Hybrid (Combining Models) | 96 |
| Storage Locations | 99 |
| Internal vs Public Docs | 102 |
| Folder Structures: Ready-Made Trees | 104 |
| Naming Conventions | 110 |

| | |
|---|-----|
| Navigation & Metadata | 113 |
| PART III
Organization Smells & Fixes | 117 |
| Overview | |

What: All the main ways to divide, group and store project documentation, compared side by side, with one detailed page per model. **Read when:** you're creating a doc structure, your current docs are hard to navigate, or you're unsure where a new doc belongs.

TL;DR

- Docs can be split along **six dimensions: type, audience, domain, lifecycle, version, visibility**. Every organizing model is a choice of which dimension goes where. See [The Six Dimensions of Doc Organization](#) (p. 63).
- **Rule of three levels:** one dimension decides the **top-level folders**, a second the **subfolders**, and the rest go into **file names or front-matter metadata**. Never nest more than 3 levels without a strong reason.
- Choose the top-level dimension by asking: **"What do readers know when they start looking?"** They might know the kind of doc, their role, the feature, or the version.
- Most mature projects end up **hybrid**: type or audience at the top, domain inside, module READMEs co-located with the code.
- Every folder gets a `README.md` index. Structure without navigation is still a maze.

Pages in this section

Concepts

| Page | What it covers |
|--|--|
| The Six Dimensions of Doc Organization (p. 63) | The 6 axes of organization and the "primary / secondary / metadata" rule |
| Storage Locations (p. 99) | Where docs physically live: repo root, docs folder, next to code, separate repo, wiki, docs site |
| Internal vs Public Docs (p. 102) | Separating internal and published docs safely |

Organizing models (one page each)

Index: [Organizing Models](#) (p. 67)

| Model | Top-level folders | Page |
|-------------------------|--|---|
| By doc type | <code>architecture/</code> , <code>api/</code> , <code>requirements/</code> , <code>operations/</code> | Model: Organize by Doc Type (p. 68) |
| By audience | <code>developers/</code> , <code>users/</code> , <code>operators/</code> , <code>business/</code> | Model: Organize by Audience (p. 70) |
| By domain / feature | <code>billing/</code> , <code>auth/</code> , <code>catalog/</code> | Model: Organize by Domain / Feature (p. 73) |
| Diátaxis (by user need) | <code>tutorials/</code> , <code>how-to/</code> , <code>reference/</code> , <code>explanation/</code> | Model: Diátaxis (by User Need) (p. 76) |
| By lifecycle / status | <code>proposed/</code> , <code>active/</code> , <code>archive/</code> | Model: Organize by Lifecycle / Status (p. 79) |

| Model | Top-level folders | Page |
|----------------------------|--|---|
| Co-located with code | <code>src/billing/README.md</code> , <code>packages/*/docs/</code> | Model: Co-located with Code (p. 82) |
| Per-service repos + portal | docs in each repo, aggregated centrally | Model: Per-Service Repos + Central Portal (p. 85) |
| Sequential & dated | <code>0001-...md</code> , <code>2026-10-09-...md</code> | Model: Sequential & Dated Records (p. 88) |
| By version | <code>v1/</code> , <code>v2/</code> , versioned site | Model: Organize by Version (p. 91) |
| Flat + metadata | one folder, tags in front matter, wiki-style, PARA | Model: Flat Structure + Metadata (Wiki-style) (p. 93) |
| Hybrid | combinations of the above | Model: Hybrid (Combining Models) (p. 96) |

Practical structure

| Page | What it covers |
|--|--|
| Folder Structures: Ready-Made Trees (p. 104) | Complete, copyable trees for 8 common project shapes |
| Naming Conventions (p. 110) | File and folder names, numbering, dates, IDs, naming the docs root |
| Navigation & Metadata (p. 113) | Index READMEs, cross-links, front matter schema, status, tags |
| Organization Smells & Fixes (p. 117) | Signs your organization is failing and how to fix each |

■ Comparison matrix

Ratings: ●●● strong · ●●○ okay · ●○○ weak.

| Model | Findability for newcomers | Scales to large docs | Ownership clarity | Stays in sync with code | Easy to start | Typical failure |
|----------------------|---------------------------|----------------------|-------------------|-------------------------|---------------|--|
| By doc type | ●●● | ●●○ | ●○○ | ●●○ | ●●● | Giant folders mixing all domains |
| By audience | ●●● | ●●○ | ●●○ | ●●○ | ●●○ | Duplication across audiences |
| By domain | ●●○ | ●●● | ●●● | ●●○ | ●●○ | Cross-cutting docs have no home |
| Diátaxis | ●●● | ●●● | ●○○ | ●●○ | ●○○ | Misclassified pages, forced fits |
| By lifecycle | ●○○ | ●●○ | ●●○ | ●○○ | ●●● | Broken links when files move |
| Co-located | ●○○ | ●●● | ●●● | ●●● | ●●● | No big picture, hard to browse |
| Per-service + portal | ●●○ | ●●● | ●●● | ●●● | ●○○ | Portal tooling overhead |
| Sequential & dated | ●○○ | ●●● | ●●○ | n/a | ●●● | Unreadable without an index |
| By version | ●●○ | ●●○ | ●●○ | ●●○ | ●○○ | Fixes not backported |
| Flat + metadata | ●○○ | ●○○ | ●○○ | ●○○ | ●●● | Turns into a junk drawer |
| Hybrid | ●●● | ●●● | ●●● | ●●● | ●○○ | Inconsistent rules if not written down |

Note: sequential/dated and lifecycle are usually **sub-patterns** used *inside* a folder (`decisions/`, `rfcs/`, `postmortems/`) rather than a whole-project model.

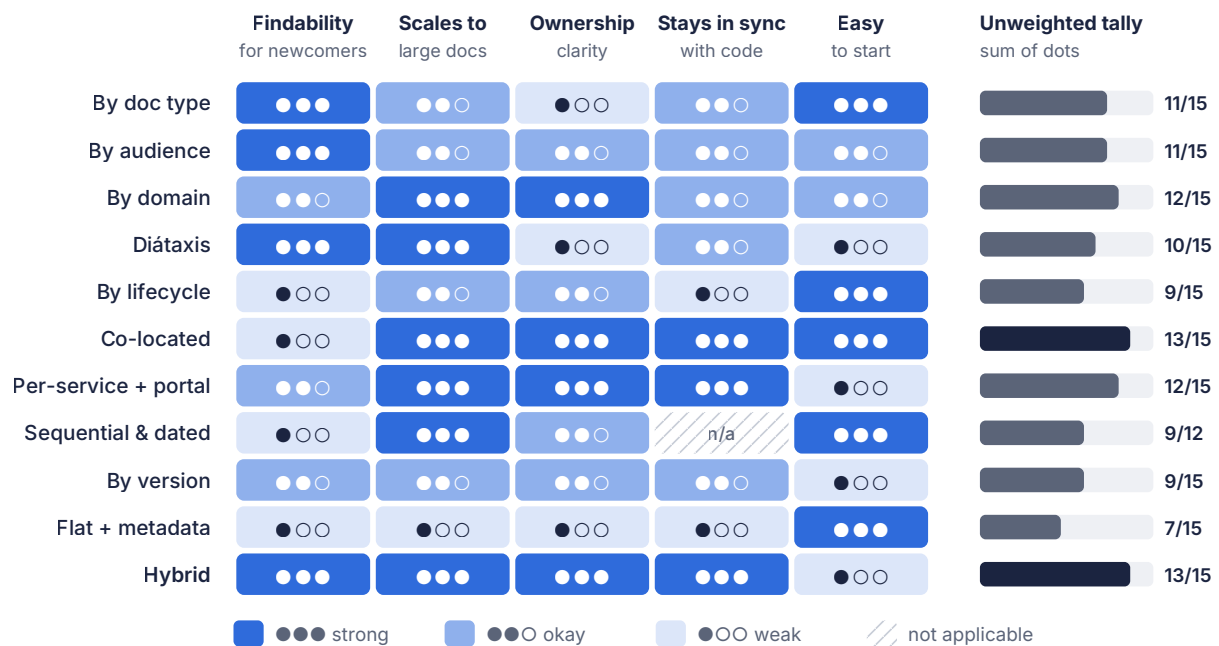


Figure 3.1 The comparison matrix as a heat map. The tally on the right is a plain sum of dots, not a recommendation; weigh the criteria that matter for your project.

■ Fast mapping: what readers know → which model

| When readers start looking, they know... | Use as top level |
|---|--|
| "I need the architecture / the API / a runbook" (the kind of doc) | By doc type (p. 68) |
| "I'm a user / developer / operator" (their role) | By audience (p. 70) |
| "This is about billing" (the feature or area) | By domain (p. 73) |
| "I want to learn / do / look up / understand" (their need) | Diátaxis (p. 76) |
| "This is about the <code>payments</code> service" (the component) | Model: Co-located with Code (p. 82) or Model: Per-Service Repos + Central Portal (p. 85) |
| "I'm on version 2" (the version) | By version (p. 91) |

Still unsure? Use the [decision guide](#) (p. 121).

CHAPTER 3.1

The Six Dimensions of Doc Organization

What: The axes along which any document can be classified, and how to decide which axis becomes folders and which becomes metadata. **Read when:** before picking an organizing model. This page explains the reasoning behind all the models.

TL;DR

- Every document has a position on **six dimensions**: type, audience, domain, lifecycle, version, visibility.
- Folders can express only **one dimension per level**. Choose wisely.
- **Primary dimension** → top-level folders. **Secondary** → subfolders. Everything else → file names, front matter, or tags.
- Choose the primary dimension by **how readers search**. Choose the secondary by **how writers own** the content.
- Visibility (internal/public) is often the one dimension that **must** be a hard split (separate folder or repo), for safety.

■ The six dimensions

| # | Dimension | Question it answers | Example values |
|---|------------|--|---|
| 1 | Type | What kind of document is it? | architecture, API, requirements, runbook, ADR, guide |
| 2 | Audience | Who is it for? | developers, users, operators, business, partners, AI agents |
| 3 | Domain | What part of the product/business is it about? | billing, auth, catalog, shipping |
| 4 | Lifecycle | What state is it in? | draft, proposed, active, deprecated, archived |
| 5 | Version | Which release does it describe? | v1, v2, 2026-10 |
| 6 | Visibility | Who may see it? | public, internal, confidential |

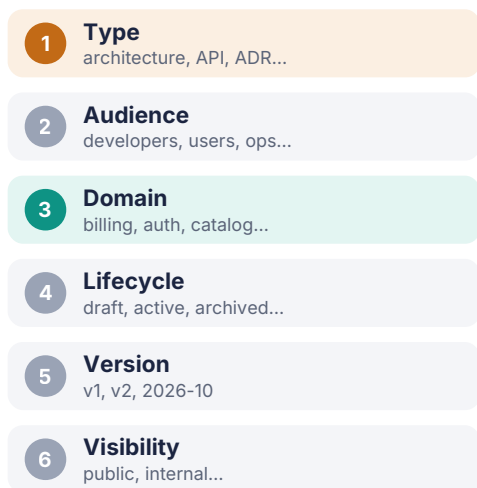
One document, all six dimensions:

```
# requirements/billing/refund-rules.md
type: business-rules      # 1
audience: [product, qa, dev] # 2
domain: billing           # 3
status: active            # 4
version: current          # 5
visibility: internal       # 6
```

YAML

Here **type** and **domain** became folders, and the other four live in front matter.

SIX DIMENSIONS



WHERE EACH ONE GOES — WORKED EXAMPLE

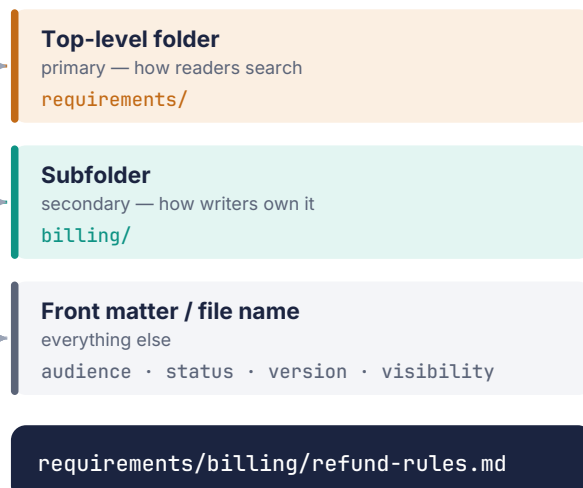


Figure 3.2 The worked example: type becomes the top-level folder, domain the subfolder, and the other four dimensions live in front matter.

Why you can't have it all in folders

Folders form a **tree**, and a tree has exactly one parent per item. If you nest all six dimensions, you get paths like this:

```
× internal/developers/v2/active/billing/architecture/refund-flow.md
```

That's impossible to navigate, and moving a document from `active` to `archive` changes its URL and breaks links.

The solution is the **three-level rule**:

```
<primary dimension>/<secondary dimension>/<file named with tertiary info>.md  
+ remaining dimensions in front matter or tags
```

```
✓ requirements/billing/refund-rules.md (status, audience, visibility in front matter)
```

Choosing the primary dimension (top-level folders)

The primary dimension is the one readers use to **start** searching.

| If most readers start with... | Primary dimension | Model |
|--|-------------------------|---|
| "I need <i>the architecture</i> ", "where are <i>the runbooks</i> ?" | Type | Model: Organize by Doc Type (p. 68) |
| "I'm a <i>customer / developer / operator</i> " | Audience | Model: Organize by Audience (p. 70) |
| "Something about <i>billing</i> " | Domain | Model: Organize by Domain / Feature (p. 73) |
| "I want to <i>learn / do / look up / understand</i> " | Type (Diátaxis variant) | Model: Diátaxis (by User Need) (p. 76) |
| "I'm on <i>v2</i> " | Version | Model: Organize by Version (p. 91) |

Test: take your 10 most common reader questions (ask the team, or check search logs and Slack) and ask, for each one, which dimension the reader knows first. The dimension that wins most often is your primary.

■ Choosing the secondary dimension (subfolders)

The secondary dimension is usually chosen by **ownership** and **growth**. Ask which split keeps each folder owned by one team and under about 15–20 files.

| Primary | Common secondary | Example |
|----------|--------------------|---|
| Type | Domain | <code>requirements/billing/</code> , <code>requirements/auth/</code> |
| Audience | Type (or Diátaxis) | <code>developers/architecture/</code> , <code>users/how-to/</code> |
| Domain | Type | <code>billing/architecture.md</code> , <code>billing/api.md</code> , <code>billing/runbooks/</code> |
| Diátaxis | Domain | <code>how-to/billing/</code> , <code>reference/billing/</code> |
| Version | Diátaxis or type | <code>v2/guides/</code> , <code>v2/reference/</code> |

■ Tertiary and beyond: metadata

Put the rest into front matter, tags or naming:

| Dimension | Best expressed as |
|-----------------------------|---|
| Lifecycle | <code>status:</code> in front matter + a status banner; physical <code>archive/</code> folder only for long-dead docs |
| Version | Git tags/branches, or a versioned docs site, rather than folders (unless several versions must be browsed side by side) |
| Audience (when not primary) | <code>audience:</code> in front matter, and "Who is this for" in the header |
| Sequence/time | File name prefix: <code>0012-...</code> , <code>2026-10-09-...</code> |
| Visibility | Hard split (folder or repo). Don't rely on metadata alone. See below. |

■ Visibility: the exception

Visibility is about **safety**, not navigation. Metadata won't stop an internal doc from being published by mistake.

- ✓ Separate by **folder** (`public/` vs `internal/`) with a publish pipeline that only builds `public/`.
- ✓ Or separate by **repository** for strong isolation (public docs repo, private code repo).
- ✗ Don't rely on `visibility: internal` front matter alone unless the build tool enforces it and you test that.

Details: [Internal vs Public Docs](#) (p. 102).

■ Worked example

Project: a SaaS with 3 teams (billing, catalog, platform), an internal API used by a frontend, and on-call duty.

1. Top reader questions: "how do I set up locally?", "what are the refund rules?", "how does the billing service talk to Stripe?", "what to do on alert X?", "why did we pick Kafka?"
2. What readers know first: mostly the **kind** of doc (setup, rules, runbook, decision), sometimes the domain.
3. Primary = **type**. Secondary = **domain** where folders grow (requirements, runbooks).
4. Lifecycle → front matter. ADRs are sequential. Visibility is all internal, so no split is needed.

```
docs/
├── README.md
├── getting-started/
├── architecture/
│   ├── overview.md
│   └── decisions/
├── requirements/
│   ├── billing/
│   ├── catalog/
│   └── rules/
├── api/
├── operations/
│   └── runbooks/
│       ├── billing/
│       └── platform/
```

sequential ADRs

That's a [Model: Hybrid \(Combining Models\)](#) (p. 96): type at the top, domain inside, sequential ADRs.

RELATED: [organizing overview](#) (p. 60) · [decision guide](#) (p. 121) · [Navigation & Metadata](#) (p. 113)

CHAPTER 3.2

Organizing Models

What: One page per way of organizing docs. Every page has the same sections: what it looks like → pros/cons → best for → avoid when → how it fails → migrating. **Read when:** comparing options. For the side-by-side matrix, see [the organizing overview](#) (p. 61).

| Model | One-line summary |
|---|---|
| Model: Organize by Doc Type (p. 68) | Folders per kind of doc. The intuitive default for small and medium projects. |
| Model: Organize by Audience (p. 70) | Folders per reader group. Best when audiences barely overlap. |
| Model: Organize by Domain / Feature (p. 73) | Folders per business domain or feature. Best when teams own domains. |
| Model: Diátaxis (by User Need) (p. 76) | Tutorials / how-to / reference / explanation. Best for product and user docs. |
| Model: Organize by Lifecycle / Status (p. 79) | Draft → active → archived. A sub-pattern for specs and RFCs. |
| Model: Co-located with Code (p. 82) | Docs next to the code. Best for module and service docs. |
| Model: Per-Service Repos + Central Portal (p. 85) | Docs in each repo plus a central portal. For polyrepo organizations. |
| Model: Sequential & Dated Records (p. 88) | Numbered and dated records. For ADRs, postmortems and meetings. |
| Model: Organize by Version (p. 91) | Docs per version. Only when several versions are live. |
| Model: Flat Structure + Metadata (Wiki-style) (p. 93) | Flat folder + tags. For small sets and wikis. |
| Model: Hybrid (Combining Models) (p. 96) | Combining models, one dimension per level. What most mature projects use. |

CHAPTER 3.3

Model: Organize by Doc Type

What: Top-level folders are named after the *kind* of document: architecture, API, requirements, operations, guides. **Read when:** you have one product and one or a few teams, and readers usually know what *kind* of doc they want.

TL;DR

- The **most common and intuitive** model, and the default recommendation for small and medium projects.
- Works because readers usually think "I need *the runbook*" or "*the API docs*".
- Weak spot: as the product grows, each type folder mixes all domains. Fix this by adding **domain subfolders** (→ [Model: Hybrid \(Combining Models\)](#) (p. 96)).

What it looks like

```
docs/
├── README.md           index + "who is this for"
├── getting-started/    setup, first run, troubleshooting
├── architecture/
│   ├── README.md       overview + diagrams
│   └── decisions/       ADRs (sequential)
├── api/                reference + guides
├── requirements/       PRDs, stories, business rules
├── development/        conventions, testing, release process
├── operations/         deployment, runbooks, postmortems
└── glossary.md
```

Typical top-level folders

| Folder | Holds | See |
|---------------------------|---------------------------------------|---|
| getting-started/ | Setup, onboarding, first steps | Developer Docs (p. 29) |
| architecture/ | Overview, diagrams, ADRs, data model | Architecture Docs (p. 44) |
| api/ | API reference, guides, webhooks | API Docs (p. 34) |
| requirements/ or product/ | BRD, PRDs, stories, rules, NFRs | Business & Requirements Docs (p. 38) |
| development/ | Conventions, testing, tooling | Developer Docs (p. 29) |
| operations/ | Deploy, config, runbooks, postmortems | Operations Docs (p. 48) |
| guides/ or user-guide/ | End-user docs | User Docs (p. 51) |
| rfcs/ or design/ | Proposals / design docs | Architecture Docs: RFCs and design docs (p. 46) |

Use **only the folders you need**. Empty `requirements/` folders "for later" just add noise.

■ Pros and cons

| ✓ Pros | ✗ Cons |
|---|--|
| Immediately understandable to everyone | Type folders grow large and mix many domains |
| Matches how most people search | Domain ownership is unclear: who owns <code>requirements/</code> ? |
| Maps cleanly to doc templates (one template per folder) | Info about one feature is spread over 4 folders |
| Easy to start, easy to explain | Some docs fit two types (a design doc with requirements) |

■ Best for

- Single product or monolith, 1–3 teams.
- Internal docs where most readers are developers.
- Projects starting their docs from scratch.

■ Avoid when

- Multiple teams own clearly separate domains (→ [Model: Organize by Domain / Feature](#) (p. 73) or [Model: Hybrid \(Combining Models\)](#) (p. 96)).
- Audiences are very different, for example public users and internal engineers (→ [Model: Organize by Audience](#) (p. 70)).
- You have dozens of services (→ [Model: Co-located with Code](#) (p. 82), [Model: Per-Service Repos + Central Portal](#) (p. 85)).

■ How it fails (and the fix)

| Symptom | Fix |
|--|--|
| <code>requirements/</code> has 60 files with no grouping | Add domain subfolders: <code>requirements/billing/</code> |
| A <code>misc/</code> or <code>other/</code> folder appears | A doc type is missing or the doc isn't needed. Decide which. |
| Feature info is hard to piece together | Add a per-domain index page that links across type folders |
| "Is this architecture or development?" debates | Write a one-line rule for each folder in <code>docs/README.md</code> |

■ Migrating

- **To this model** (from a flat folder): create type folders, move files, add a README index per folder, and fix links (see [Evolving the Structure](#) (p. 129)).
- **From this model to hybrid**: keep type at the top and add domain subfolders only in folders with more than ~15 files.

RELATED: [Model: Organize by Domain / Feature](#) (p. 73) · [Model: Hybrid \(Combining Models\)](#) (p. 96) · [Folder Structures: Ready-Made Trees](#) (p. 104)

CHAPTER 3.4

Model: Organize by Audience

What: Top-level folders are named after *who reads them*: developers, users, operators, business, partners.

Read when: your readers are distinct groups with different goals and vocabulary, who rarely need each other's docs.

TL;DR

- Readers pick their door by **role**, and everything behind that door is relevant to them.
- Excellent for products with **external users and internal engineers**, or for platforms with separate **operator** and **developer** readers.
- Weak spot: **duplication**. Some facts matter to several audiences. Keep each fact in one place and link to it.

What it looks like

```
docs/
├── README.md           "Choose your path" landing page
├── users/              end users / customers
│   ├── getting-started.md
│   ├── how-to/
│   └── faq.md
├── developers/         people building on or working in the code
│   ├── setup.md
│   ├── architecture/
│   └── api/
├── operators/          people running/hosting it
│   ├── installation.md
│   ├── configuration.md
│   └── runbooks/
├── business/           product, sales, compliance
│   ├── requirements/
│   └── policies/
├── shared/             facts used by several audiences
└── glossary.md
```

Landing page pattern:

Documentation

MARKDOWN

```
I am a...	Start here
👤 User of the app	[User guide](users/getting-started.md)
🛠️ Developer	[Developer setup](developers/setup.md)
⚙️ Operator / admin	[Installation](operators/installation.md)
📄 Product / business	[Requirements](business/requirements/README.md)
```

■ Pros and cons

| ✓ Pros | ✗ Cons |
|--|--|
| Readers find their section instantly | Same fact needed by several audiences → duplication risk |
| Each section can use the right tone and depth | People with several roles have to look in several places |
| Easy to publish one audience separately (public user docs) | Domain ownership spread across audience folders |
| Clear for reviewers (the users section is reviewed by support) | Tricky when audiences overlap a lot (devs = operators in DevOps teams) |

■ Best for

- Products with **public users + internal team** (SaaS, apps).
- Self-hosted / open-source software (users, operators, contributors).
- Platforms and internal developer platforms (consumers vs maintainers).
- Organizations where business and engineering docs are read by different people.

■ Avoid when

- The team is small and everyone reads everything (→ [Model: Organize by Doc Type](#) (p. 68)).
- Audiences overlap heavily, for example a DevOps team that both builds and runs.
- You can't define audiences crisply. If people argue "is this for devs or ops?", the split will drift.

■ How it fails (and the fix)

| Symptom | Fix |
|--|--|
| Setup steps copied into <code>developers/</code> and <code>operators/</code> | Put them in <code>shared/</code> (or the more specific audience) and link from the other |
| A <code>shared/</code> folder grows huge | You probably need Model: Organize by Doc Type (p. 68) or Model: Hybrid (Combining Models) (p. 96) as the primary instead |
| Internal info leaks into <code>users/</code> | Use a hard split, separate repo or publish pipeline: Internal vs Public Docs (p. 102) |

■ Common variants

- **Two-audience split:** `public/` and `internal/`, or `user-guide/` and `dev-guide/`. This is the simplest and the most common.
- **Audience → Diátaxis:** `users/tutorials/`, `users/how-to/`, `users/reference/`. This suits large product docs.
- **Audience as separate sites:** a user help center, a developer portal and an internal handbook. Each can use its own model inside.

■ Migrating

- **To this model:** list audiences ([Documentation Audiences](#) (p. 16)), tag every existing doc with its main audience, then move docs and create `shared/` for the rest.

- **Away from it:** usually toward [Model: Hybrid \(Combining Models\)](#) (p. 96): keep an audience split only at the **public vs internal** level and use type or domain inside.

RELATED: [Documentation Audiences](#) (p. 16) · [Internal vs Public Docs](#) (p. 102) · [Model: Diátaxis \(by User Need\)](#) (p. 76)

CHAPTER 3.5

Model: Organize by Domain / Feature

What: Top-level folders are business domains, bounded contexts or features (billing, auth, catalog). Each domain folder holds all doc types about that domain. **Read when:** several teams each own a part of the product, or readers mostly think in terms of features.

TL;DR

- Everything about **billing** is in `billing/`: rules, API, architecture, runbooks.
- Mirrors **team ownership** (Conway's law). Each team owns its folder, which you can enforce with `CODEOWNERS`.
- Weak spot: **cross-cutting docs** (setup, overall architecture, conventions) have no natural home. Add a `platform/` or `_shared/` area for them.
- Pairs well with domain-driven design and modular monoliths.

What it looks like

```
docs/
├── README.md           index of domains + cross-cutting docs
├── _shared/            cross-cutting (underscore sorts it first)
│   ├── getting-started.md
│   ├── architecture-overview.md
│   ├── conventions.md
│   └── glossary.md
├── billing/
│   ├── README.md       what this domain does, owner, key links
│   ├── requirements.md  PRD / stories / scope
│   ├── rules.md         business rules (BR-BIL-xx)
│   ├── architecture.md  internals, diagrams
│   ├── api.md           or link to generated reference
│   ├── decisions/       domain-scoped ADRs
│   └── runbooks/
├── catalog/
│   └── ...
└── identity/
    └── ...
```

Each domain folder uses the **same internal layout**. Write that down as a template so that each domain doesn't invent its own.

Domain README pattern

Billing

MARKDOWN

Handles subscriptions, invoices, payments and refunds.

- ****Owner:**** team-billing (#billing on Slack)
- ****Code:**** ``src/Billing/``, ``services/billing-api/``

```
- **Depends on:** identity, catalog
- **Key docs:** [Rules](rules.md) · [API](api.md) · [Runbooks](runbooks/README.md)
```

Pros and cons

| ✓ Pros | ✗ Cons |
|---|---|
| Clear ownership: one team ↔ one folder | Cross-cutting docs need a separate home |
| All knowledge about a feature in one place | Readers looking for "all runbooks" must visit every domain |
| Scales with the number of teams/domains | Domain boundaries change, and moving folders breaks links |
| Matches code structure in modular/DDD codebases | New readers may not know domain names yet (needs a good index + glossary) |

Best for

- Products with 3+ **teams** owning distinct areas.
- Domain-driven design, modular monoliths.
- **Business/requirements docs**, which are naturally discussed per domain.
- Large requirement sets (as the secondary dimension under `requirements/`).

Avoid when

- Small team, small product: domains would be 1–2 files each.
- Domains aren't stable yet (early startup pivoting).
- Most readers are on-call engineers who need **all runbooks together**. Add a cross-domain runbook index, or keep runbooks type-based.

How it fails (and the fix)

| Symptom | Fix |
|---|---|
| Each domain folder has a different layout | Add a domain-folder template; check in review |
| Shared docs duplicated in every domain | Move to <code>_shared/</code> and link |
| "Where are all the ADRs?" | Generate or maintain a cross-domain index (<code>decisions-index.md</code>) |
| Domain renamed in code but not in docs | Keep domain names in the glossary; rename via a migration PR |

Variants

- **Domain at the second level** (most common): `requirements/billing/`, `operations/runbooks/billing/`. See [Model: Hybrid \(Combining Models\)](#) (p. 96).
- **Feature folders for product specs**: `product/features/partial-refunds/{prd.md, stories.md, designs.md}`, archived when shipped.
- **Domain docs co-located** in the code module: `src/Billing/docs/`. See [Model: Co-located with Code](#) (p. 82).

■ Migrating

- **To this model:** define the domain list (match code modules and teams), create `_shared/`, move docs, add a `README.md` per domain and set up `CODEOWNERS`.
- **Away from it:** if domains are tiny, collapse them into [Model: Organize by Doc Type](#) (p. 68) with domain-named files (`requirements/billing.md`).

RELATED: [Model: Hybrid \(Combining Models\)](#) (p. 96) · [Model: Co-located with Code](#) (p. 82) · [Business & Requirements Docs](#) (p. 38)

CHAPTER 3.6

Model: Diátaxis (by User Need)

What: A documentation framework that splits content into four kinds by what the reader needs: **tutorials** (learning), **how-to guides** (doing), **reference** (looking up) and **explanation** (understanding). **Read when:** you build product, library, API or platform docs for users, especially a public docs site.

TL;DR

- Four kinds, four folders: `tutorials/`, `how-to/`, `reference/`, `explanation/`.
- Each kind has **one job** and a **distinct writing style**. Mixing them is the most common cause of confusing docs.
- Widely used for user-facing and developer-facing product docs.
- Less suited to **internal project docs** like ADRs, PRDs or runbooks. Use it for the *user/consumer* part only, or treat those as special types.

■ The four quadrants

| | Practical (doing) | Theoretical (knowing) |
|-------------------|--|--|
| Learning (study) | Tutorials: "Build your first bot" | Explanation: "How the event loop works" |
| Working (at work) | How-to guides: "Deploy to AWS" | Reference: "CLI flags" |

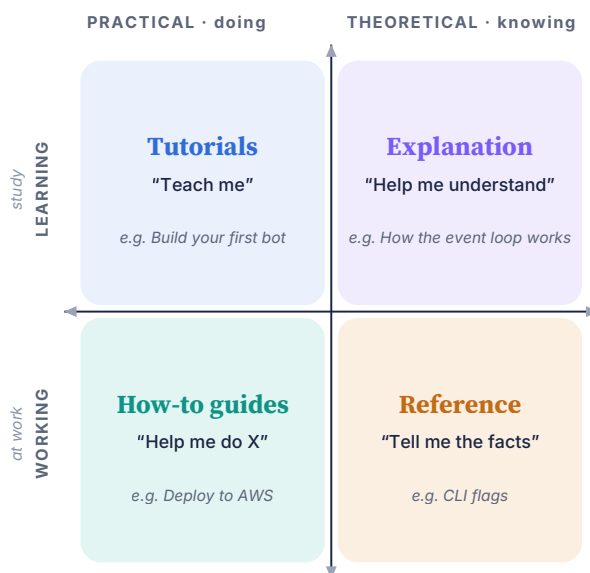


Figure 3.3 The Diátaxis compass: two axes, four kinds of documentation, one job each.

| Kind | Reader says | Shape | Rules |
|---------------------|----------------------|--|---|
| Tutorial | "Teach me" | A guided lesson with a guaranteed result | One path, no choices, every step works, minimal explanation |
| How-to guide | "Help me do X" | Steps toward a specific goal | Assumes competence; title starts "How to..." or a verb; no teaching |
| Reference | "Tell me the facts" | Structured, complete, consistent | Mirrors the product's structure; no instructions or opinions; often generated |
| Explanation | "Help me understand" | Discursive discussion | Context, background, alternatives, <i>why</i> ; no steps |

Page layouts for each: [Page Patterns](#) (p. 141).

■ What it looks like



Popular naming variants: `guides/` for how-to, `concepts/` or `topics/` for explanation, `learn/` for tutorials.

■ Pros and cons

| ✓ Pros | ✗ Cons |
|--|--|
| Based on how people actually use docs | Requires discipline to classify pages correctly |
| Improves writing: each page has one mode | Doesn't cover internal docs well (ADRs, PRDs, runbooks, meeting notes) |
| Scales well to large docs sites | Information about one feature spreads across 4 folders |
| Well-known, with good public guidance | Writers new to it often produce "how-to-tutorial-explanation" hybrids |

■ Best for

- **Public product docs**, libraries, SDKs, CLIs, developer platforms.
- User docs in [Model: Organize by Audience](#) (p. 70) setups (`users/tutorials/`, `users/how-to/`, ...).
- API docs: tutorial = quick start, how-to = guides, reference = spec, explanation = concepts.

■ Avoid when

- It's mainly **internal engineering knowledge** (decisions, requirements, ops). Use [Model: Organize by Doc Type](#) (p. 68).
- Docs are tiny (fewer than ~10 pages). Four folders of 2 pages each is overkill. Use Diátaxis as a writing discipline, not a folder structure.

■ Classification cheat sheet

Ask two questions about a page:

1. Is it about **action** (doing) or **cognition** (knowing)?
2. Is the reader **learning** or **working**?

| Action + learning | → Tutorial |
|----------------------|---------------|
| Action + working | → How-to |
| Cognition + working | → Reference |
| Cognition + learning | → Explanation |

If a page fits two, **split it**. For example, a how-to with a long "why" section becomes a how-to that links to an explanation page.

■ How it fails (and the fix)

| Symptom | Fix |
|---|--|
| Tutorials full of options and caveats | Move options to how-to/reference, keep one happy path |
| Reference pages with opinions and steps | Move steps to how-to, opinions to explanation |
| Huge <code>how-to/</code> folder | Add domain subfolders: <code>how-to/billing/</code> |
| Readers can't find feature X | Add a feature index or good search; cross-link the 4 kinds per feature |

■ Migrating

- **To Diátaxis:** classify each existing page (spreadsheet: page → kind → action: move/split), split mixed pages first, then move. Do one section at a time.
- **Hybrid:** keep Diátaxis for the user/consumer section only and use type-based folders for internal docs. See [Model: Hybrid \(Combining Models\)](#) (p. 96).

RELATED: [Page Patterns](#) (p. 141) · [User Docs](#) (p. 51) · [Model: Organize by Audience](#) (p. 70)

CHAPTER 3.7

Model: Organize by Lifecycle / Status

What: Folders (or metadata) reflect where a document is in its life: draft, proposed, active, shipped, deprecated, archived. **Read when:** you have many documents that change status, such as RFCs, specs, PRDs or policies, or old docs are cluttering current ones.

TL;DR

- Lifecycle is almost always a **secondary pattern**, used *inside* a folder like `rfcs/` or `product/`, rather than a project-wide top level.
- Prefer **status in front matter** over status folders. Moving files breaks links.
- **One exception:** a physical `archive/` folder for long-dead docs is fine, and it keeps current folders clean.
- Always show status visibly at the top of the doc, not only in metadata.

■ Two ways to express lifecycle

A. Status folders

```
rfcs/
├── README.md
├── drafts/
│   └── multi-currency.md
├── accepted/
│   └── 0041-event-sourcing-orders.md
├── rejected/
│   └── 0039-graphql-gateway.md
└── archive/
    └── 0012-legacy-sync.md
```

B. Status in metadata (recommended)

```
rfcs/
├── README.md           index table with status column
├── 0039-graphql-gateway.md  status: rejected
├── 0041-event-sourcing-orders.md status: accepted
├── 0042-multi-currency.md  status: draft
└── archive/             only for things nobody should read anymore
```

```
---
id: RFC-0042
title: Multi-currency support
status: draft      # draft | review | accepted | rejected | withdrawn | implemented
owner: "@anna"
created: 2026-09-20
updated: 2026-10-05
---
```

MARKDOWN

```
# RFC-0042: Multi-currency support
```

```
> **Status:** ● Draft. Open for comments until 2026-10-20.
```

| | Status folders | Status metadata |
|-----------------------------|-----------------------------|--|
| Browsing by status | ✓ Easy | Needs index table or tooling |
| Stable links | ✗ Links break on every move | ✓ Path never changes |
| Git history per file | ⚠ Rename noise | ✓ Clean |
| Works with AI agents/search | Both fine | ✓ Status grep-able: <code>grep "status: accepted"</code> |

Common lifecycles

| Doc type | Lifecycle |
|--------------------|--|
| RFC / design doc | draft → review → accepted / rejected / withdrawn → implemented |
| ADR | proposed → accepted / rejected → deprecated / superseded |
| PRD / feature spec | draft → approved → in development → shipped → archived |
| Policy | draft → approved → active → under review → retired |
| Guide / reference | draft → active → deprecated → removed |
| Postmortem | draft → reviewed → published (immutable) |

Status banners

Make status visible to readers who never look at metadata:

```
> [!WARNING]
> **Deprecated** since 2026-06. Replaced by [Payments v2 guide](../v2/payments.md).
> This page will be removed after 2027-01.
```

MARKDOWN

```
> ▪ **Archived.** Kept for history. Describes the system as of 2024; do not use for current work.
```

MARKDOWN

Pros and cons

| ✓ Pros | ✗ Cons |
|--|---|
| Current docs aren't buried under dead ones | Status folders break links |
| Clear where each proposal stands | Status gets stale if nobody updates it |
| Makes review workflows explicit | Adds overhead for docs that are simply "living" |

Best for

- RFCs, design docs, PRDs, feature specs, policies.
- Large business-requirements sets where shipped specs pile up.
- Cleaning up legacy docs (an `archive/` folder).

■ Avoid when

- Docs are "living" (README, guides, reference). They're simply current, or they get deleted.
- As a project-wide top level (`active/` , `archive/`), because it hides every other dimension.

■ How it fails (and the fix)

| Symptom | Fix |
|---|--|
| Everything is "draft" forever | Add status to review checklist; Maintenance (p. 152) |
| Archived docs still rank first in search | Exclude <code>archive/</code> from site search; add banners |
| Broken links after moving to <code>accepted/</code> | Switch to metadata status, or add redirects |

RELATED: [Model: Sequential & Dated Records](#) (p. 88) · [Navigation & Metadata](#) (p. 113) · [Architecture Docs](#) (p. 44)

CHAPTER 3.8

Model: Co-located with Code

What: Docs live right next to the code they describe: a `README.md` in each module, package or service folder, sometimes with a local `docs/` subfolder. **Read when:** you have a monorepo, multiple packages, microservices or a component library, or docs keep drifting out of date.

TL;DR

- **Proximity = freshness.** Developers see the doc when they change the code, and reviewers see both in one PR.
- Every significant module gets a **README**: purpose, owner, how to use and run it, links.
- Weak spot: **no big picture.** Always combine it with a central index or overview (→ [Model: Hybrid \(Combining Models\)](#) (p. 96)).
- Ideal for **component and module-level** docs. Not suited to business requirements or cross-cutting docs.

What it looks like

Monorepo

```

repo/
├── README.md           big picture + map of packages
├── docs/               cross-cutting only: architecture, conventions, ADRs
├── apps/
│   ├── web/
│   │   ├── README.md
│   │   └── docs/       app-specific how-tos (optional)
│   └── admin/
│       └── README.md
├── services/
│   ├── billing-api/
│   │   ├── README.md
│   │   ├── openapi.yaml  API reference source
│   │   └── docs/
│   │       ├── runbooks/
│   │       └── decisions/  service-level ADRs
│   └── catalog-api/
│       └── README.md
└── packages/
    ├── ui-kit/
    │   ├── README.md
    │   └── src/Button/Button.mdx  component docs next to component
    └── money/
        └── README.md
  
```

Modular monolith

```

src/
├── Billing/
│   ├── README.md      domain overview, rules links, public interfaces
│   └── Domain/
  
```

```

└─ Http/
└─ Catalog/
└─ README.md

```

■ What belongs co-located vs central

| Co-located (next to code) | Central (docs root) |
|---|--|
| Module purpose, public interface, usage | System architecture overview |
| How to run/test this service | Getting started for the whole repo |
| Service config, service runbooks | Coding conventions, contribution process |
| Service-specific ADRs | Cross-cutting ADRs |
| API spec of this service | Business requirements, glossary |
| Component examples (Storybook/MDX) | Onboarding, process docs |

■ Pros and cons

| ✓ Pros | ✗ Cons |
|--|--|
| Docs updated in the same PR as code | Hard to get an overview or browse "all docs" |
| Ownership automatic (code owners = doc owners) | Business people won't find docs inside <code>src/</code> |
| Moves/deletes with the code | Inconsistent formats across modules without a template |
| Great for AI agents working on one module | Cross-cutting knowledge has no home (needs central docs) |

■ Best for

- Monorepos, microservices in one repo, multi-package libraries.
- Component libraries and design systems (docs next to each component).
- Teams that struggle with stale central docs.

■ Avoid when

- It would be the only model in a project with business stakeholders. They need a central, browsable place.
- Modules are tiny or unstable, so READMEs would churn constantly.

■ Make it navigable

1. Root README with a package map:

```

## Packages
Package	Purpose	Owner
[billing-api](services/billing-api/README.md)	Payments, invoices, refunds	team-billing
[ui-kit](packages/ui-kit/README.md)	Shared React components	team-web

```

MARKDOWN

1. Standard module README template. See [Developer Docs: Module / package README](#) (p. 31).

2. **Optional aggregation:** a docs site that collects all `**/README.md` and `**/docs/**`, using Backstage TechDocs, the MkDocs monorepo plugin, Docusaurus multi-instance or similar.
3. **Nested agent files:** `services/billing-api/AGENTS.md` for module-specific AI rules.

■ How it fails (and the fix)

| Symptom | Fix |
|-------------------------------------|---|
| Modules with no README | CI check: every folder in <code>services/*</code> must have README.md |
| Every README has different sections | Template + review |
| "Where's the overall architecture?" | Central <code>docs/architecture/</code> + root map |

RELATED: [Model: Per-Service Repos + Central Portal](#) (p. 85) · [Model: Hybrid \(Combining Models\)](#) (p. 96) · [Storage Locations](#) (p. 99)

CHAPTER 3.9

Model: Per-Service Repos + Central Portal

What: In a multi-repo (polyrepo) architecture, each service or repo keeps its own docs, and a central portal, catalog or index repo ties them together. **Read when:** your organization has many repositories (microservices, many teams), and nobody can find anything across them.

TL;DR

- Each repo uses the **same standard layout** for its own docs (README, docs/, API spec, runbooks).
- A **central place** aggregates or indexes them: a developer portal (such as Backstage), a docs site that pulls repos in, or a simple index repo.
- **Organization-wide** docs (architecture landscape, standards, onboarding, glossary) live in a dedicated **handbook / engineering docs repo**.
- The hard part is **consistency**. Enforce it with a repo template and CI checks.

What it looks like



Standard per-repo layout (make it a repo template)

| Path | Required | Purpose |
|----------------------|----------|------------------------------------|
| README.md | ✓ | Purpose, owner, quick start, links |
| docs/index.md | ✓ | Docs landing for the portal |
| docs/architecture.md | ✓ | Service internals, dependencies |

| Path | Required | Purpose |
|---|-------------------------|---------------------------------------|
| <code>docs/runbooks/</code> | Production services | On-call procedures |
| <code>docs/decisions/</code> | When decisions are made | Service-level ADRs |
| API spec (<code>openapi.yaml</code> , <code>.proto</code> , <code>asyncapi.yaml</code>) | If it has an API | Reference source |
| Portal metadata file | If using a portal | Owner, lifecycle, dependencies, links |

■ Aggregation options

| Option | Effort | Notes |
|--|--------|---|
| Index page in the handbook repo listing all repos + links | Low | Manual, OK for <20 repos |
| Docs site that pulls repos at build time (git submodules, a multirepo plugin, CI sync) | Medium | One search across all docs |
| Developer portal (Backstage TechDocs, Port, Cortex, etc.) | High | Catalog, ownership, docs, API specs in one UI |
| Generated catalog from repo metadata | Medium | Always current service list |

■ Pros and cons

| ✓ Pros | ✗ Cons |
|-------------------------------------|---|
| Docs stay with code and team | Cross-service knowledge needs a separate home |
| Strong ownership | Inconsistent layouts without enforcement |
| Scales to hundreds of services | Portal setup and maintenance cost |
| Service docs versioned with service | Search across repos needs tooling |

■ Best for

- Polyrepo microservice organizations, roughly 10+ repos.
- Platform engineering teams that build an internal developer portal.

■ Avoid when

- Few repos (<5). A central index page in each README is enough.
- A monorepo. Use [Model: Co-located with Code](#) (p. 82) + central `docs/` instead.

■ How it fails (and the fix)

| Symptom | Fix |
|--|--|
| Each repo's docs look different | Repo template + CI lint ("docs/index.md must exist") |
| Org-level decisions scattered across service repos | Org ADRs in handbook repo, service ADRs in service repos |
| Portal shows stale/abandoned services | Lifecycle field in metadata; periodic ownership review |

RELATED: [Model: Co-located with Code](#) (p. 82) · [Storage Locations](#) (p. 99) · [Folder Structures: Ready-Made Trees](#) (p. 104)

CHAPTER 3.10

Model: Sequential & Dated Records

What: Documents identified by a sequence number (`0012-use-stripe.md`) or a date (`2026-10-09-checkout-outage.md`), stored flat in one folder and listed by an index. **Read when:** you keep a series of records that are created over time and rarely changed: ADRs, RFCs, postmortems, meeting notes, release notes, requirement IDs.

TL;DR

- Use **numbers** when order and stable IDs matter (ADRs, RFCs). Use **dates** when time matters (postmortems, meetings, release notes).
- Records are **append-only**: new files are added, and old ones are superseded or archived, never rewritten.
- Always keep an **index** (`README.md`) with ID, title, status and date. File listings alone aren't enough.
- This is a **folder-level pattern**, used inside any project model.

Numbered records

```
architecture/decisions/
├── README.md
├── 0001-record-architecture-decisions.md
├── 0002-use-postgresql.md
├── 0003-modular-monolith.md
├── 0004-use-rabbitmq-for-async.md
└── 0005-replace-rabbitmq-with-kafka.md (supersedes 0004)
```

| Rule | Why |
|---|--|
| Zero-pad to 4 digits: <code>0001</code> | Correct sorting up to 9999 |
| Number + kebab-case title | Human-readable and unique |
| Never reuse or renumber | IDs are referenced from code, PRs, other docs |
| Title in file = title in heading | <code># ADR-0005: Replace RabbitMQ with Kafka</code> |
| Superseding = new file + status update on old one | History stays intact |

Avoiding number collisions

Two people creating `0006` in parallel branches is common. Options:

- Reserve numbers in the index via a tiny PR first.
- Use the next number at **merge** time (rename during review).
- Use **date-based IDs** instead: `2026-10-09-use-kafka.md`.
- Use tooling (`adr-tools` , `log4brains`) that allocates numbers.

Dated records

```
operations/postmortems/
├── README.md
├── 2026-08-02-db-failover-delay.md
├── 2026-09-18-checkout-outage.md

meetings/
├── 2026-10-01-sprint-planning.md
├── 2026-10-08-architecture-sync.md

release-notes/
├── 2026-09.md
├── 2026-10.md
```

- Use ISO 8601 (YYYY-MM-DD). It sorts correctly and isn't ambiguous across countries.
- With many files, group by year (2026/). Avoid month folders unless volume is very high.

The index page

Architecture Decision RecordsMARKDOWN

```
ID	Title	Status	Date
[0005](0005-replace-rabbitmq-with-kafka.md)	Replace RabbitMQ with Kafka	✓ accepted	2026-07-11
[0004](0004-use-rabbitmq-for-async.md)	Use RabbitMQ for async	✖ superseded by 0005	2025-02-03
[0003](0003-modular-monolith.md)	Modular monolith	✓ accepted	2024-11-20
```

Newest first is usually more useful for postmortems and meetings. Oldest first suits ADRs, which are read as a story. Pick one order and keep it.

Requirement IDs (sequence without files)

Not every sequence needs one file per item. Business rules and requirements usually live as rows in a table with stable IDs (BR-PAY-04, FR-PAY-012). See Business & Requirements Docs: Business rules catalog (p. 42). The same rules apply: never reuse an ID, and deprecate rather than delete.

Pros and cons

| ✓ Pros | ✗ Cons |
|----------------------------|---|
| Stable, citable IDs | Unreadable without an index |
| Natural history / timeline | Number collisions in parallel work |
| Trivial to add new records | Finding "the current decision on X" needs the index or search |

Best for

ADRs, RFCs, postmortems, meeting notes, release notes, changelogs, requirement and rule IDs.

■ Avoid when

Living docs (guides, reference, overviews). Numbering them suggests order and immutability that doesn't exist.

RELATED: [Model: Organize by Lifecycle / Status \(p. 79\)](#) · [Naming Conventions \(p. 110\)](#) · [Architecture Docs: Architecture Decision Records \(ADRs\) \(p. 45\)](#)

CHAPTER 3.11

Model: Organize by Version

What: Docs are split per product or API version, so readers on v1 and v2 each see accurate docs. **Read when:** several major versions are supported at the same time and they behave differently, for example a public API, SDK, library or self-hosted product.

TL;DR

- **Most projects don't need version folders.** Git already versions docs together with the code. Docs on a release tag describe that release.
- You need *browsable* versions only when **users on old versions still need docs** and versions differ meaningfully.
- Prefer **versioned docs-site builds** (from Git tags/branches) over copying folders by hand.
- If you do copy folders, define a **backport policy**, or old versions rot.

Options, from simplest to heaviest

| Option | How | Use when |
|--------------------------------------|---|--|
| 1. Git only | Docs live with code; readers browse the tag (<code>/tree/v1.4.0/docs</code>) | Internal projects, single supported version |
| 2. "Since / until" notes | Inline markers: "Added in 2.3", "Deprecated in 3.0" | Small differences between versions |
| 3. Versioned site from branches/tags | Docs site builds <code>latest</code> , <code>2.x</code> , <code>1.x</code> from release branches (Docusaurus versioning, <code>mike</code> for MkDocs, Read the Docs) | Libraries, SDKs, self-hosted products |
| 4. Version folders in repo | <code>v1/</code> , <code>v2/</code> side by side | Public APIs with different major versions served at once |

Inline version notes (option 2)

```
### `timeout` option
> Added in **2.3**. Before 2.3, use `requestTimeout` (removed in 3.0).
```

MARKDOWN

Cheap and effective. Readers immediately see whether a feature applies to them.

Version folders (option 4)

```
api/
├── README.md
├── v1/
│   ├── README.md
│   ├── reference/
│   └── guides/
└── v2/
```

which version to use, support timeline, migration links

- deprecated, sunset 2027-03-31 with deprecation banner

✓ current

```

├── README.md
├── reference/
├── guides/
└── migration-v1-to-v2.md

```

| Rule | Why |
|---|---|
| Version-independent docs (concepts, auth) stay outside version folders | Avoid N copies of the same text |
| Old versions get a banner and a sunset date | Readers know they're on old docs |
| Migration guide between each major version | The most-read page during upgrades |
| Freeze old versions, and only fix critical errors | Backporting everything costs too much |
| Published site shows a version switcher and defaults to latest | Search engines and readers land on current docs |

■ Pros and cons

| ✓ Pros | ✗ Cons |
|-------------------------------------|---|
| Accurate docs per version | Duplication, multiplied by each version |
| Users on old versions are supported | Fixes need backporting |
| Clear upgrade path | Search results may surface old versions |

■ Best for

Public APIs with several live major versions, SDKs, frameworks, self-hosted / on-premise software, products with long-term-support (LTS) releases.

■ Avoid when

SaaS with continuous deployment (there's only "now"), internal apps, early-stage projects.

RELATED: [API Docs: Versioning and deprecation](#) (p. 36) · [Model: Organize by Lifecycle / Status](#) (p. 79) · [Model: Hybrid \(Combining Models\)](#) (p. 96)

CHAPTER 3.12

Model: Flat Structure + Metadata (Wiki-style)

What: Few or no folders. Documents sit in one place and are organized by **links, tags and front-matter metadata** instead of hierarchy. This also covers personal and team knowledge systems such as PARA and Zettelkasten-style notes. **Read when:** your docs are small (fewer than ~20 pages), you're using a wiki or note tool, or content cuts across categories so much that no tree fits.

TL;DR

- **Small projects:** a flat folder with good file names is perfectly fine. Don't over-structure 8 files.
- **Larger projects:** a flat structure works **only** with strong metadata, index pages and search. Otherwise it turns into a junk drawer.
- Wikis (Confluence, Notion, GitHub Wiki, Obsidian) often work this way by default.
- Graduate to folders once you pass about 20 files, or once people start asking "where do I put this?"

What it looks like

Small project (good)

```
docs/
├── README.md           index with one line per doc
├── setup.md
├── architecture.md
├── api.md
├── deployment.md
├── decisions.md        ADRs as sections, or a log table
└── glossary.md
```

Larger flat set with metadata (OK with discipline)

```
notes/
├── index.md            generated or curated hub pages
├── billing-refund-rules.md    tags: [billing, rules]
├── billing-stripe-integration.md tags: [billing, architecture]
├── catalog-search-ranking.md  tags: [catalog, explanation]
└── ...
```

```
---
title: Refund rules
type: business-rules
domain: billing
tags: [refunds, finance]
status: active
---
```

YAML

A prefix naming convention (`<domain>-<topic>.md`) gives you **pseudo-folders** that still sort together.

■ PARA (for team or personal knowledge bases)

A popular structure for note systems, organized by **actionability**:

| Folder | Contains | Project-docs example |
|-------------------------|----------------------------------|--|
| <code>projects/</code> | Efforts with a goal and deadline | <code>projects/checkout-redesign/</code> |
| <code>areas/</code> | Ongoing responsibilities | <code>areas/security/</code> , <code>areas/on-call/</code> |
| <code>resources/</code> | Reference material on topics | <code>resources/stripe-api-notes.md</code> |
| <code>archive/</code> | Inactive items from the above | finished projects |

It's good for **team wikis** and **personal notes**, and less suited to product docs that readers browse by topic.

■ Hub pages (Maps of Content)

In flat systems, **index** pages do the work that folders would otherwise do:

```
# Billing: hub

## Rules
- [Refund rules](billing-refund-rules.md)
- [Invoice numbering](billing-invoice-numbering.md)

## Architecture
- [Stripe integration](billing-stripe-integration.md)
```

MARKDOWN

■ Pros and cons

| ✓ Pros | ✗ Cons |
|--|---|
| Zero setup, no "which folder?" debates | Hard to browse at scale |
| Links and tags allow many-to-many classification | Depends on search and discipline |
| Easy to restructure later (no paths to move) | Metadata drifts if not validated |
| Natural for wikis and note tools | Poor for ownership (no folder = no owner) |

■ Best for

- Small projects (<20 docs).
- Wikis, team knowledge bases, research notes.
- Early-stage projects before the structure is clear.

■ Avoid when

- Many contributors and many docs without a curator.
- Product or API docs for external readers.

■ Graduation path

When the flat folder passes about 20 files:

1. Look at the prefixes and tags. The most common ones show your natural primary dimension.
2. Create folders for that dimension ([The Six Dimensions of Doc Organization](#) (p. 63)).
3. Move files and update links. See [Evolving the Structure](#) (p. 129).

RELATED: [Navigation & Metadata](#) (p. 113) · [Organization Smells & Fixes](#) (p. 117) · [Storage Locations](#) (p. 99)

CHAPTER 3.13

Model: Hybrid (Combining Models)

What: Using different organizing models at different levels or in different parts of the docs. This is what most mature projects actually do. **Read when:** no single model fits, or your project has grown past a simple structure.

TL;DR

- A hybrid is fine, and usually best, as long as each level follows one rule and the rules are written down.
- Proven combinations: Type → Domain, Audience → Diátaxis, Central + co-located, Type with sequential sub-folders.
- Write the rules in the docs-root `README.md` under a "How these docs are organized" heading.
- Don't mix dimensions within the same level: `architecture/`, `billing/` and `users/` as siblings is a red flag.

■ The golden rule: one dimension per level

✓ docs/<TYPE>/<DOMAIN>/file.md
 docs/requirements/billing/refund-rules.md
 docs/operations/runbooks/billing/stripe-down.md

✗ docs/
 ├── architecture/ ← type
 ├── billing/ ← domain
 ├── users/ ← audience
 └── old/ ← lifecycle

The ✗ version forces every writer to choose between equally valid folders, and every reader to check all of them.

Allowed exception: cross-cutting folders that are clearly special, like `_shared/`, `archive/` and `glossary.md`, can sit alongside the others.

■ Proven hybrid patterns

1. Type → Domain (most common for product teams)

```
docs/
├── getting-started/
├── architecture/
│   ├── overview.md
│   └── decisions/
├── requirements/
│   ├── billing/
│   ├── catalog/
│   └── rules/
└── api/
```

sequential

domain

```

└─ operations/
   └─ runbooks/
      └─ billing/           domain
      └─ platform/
      └─ postmortems/      dated

```

Use this when one product has several domains and readers think in doc types first.

2. Audience → Diátaxis (product with public docs)

```

docs/
├─ user-guide/           public, end users
│  └─ tutorials/
│  └─ how-to/
│  └─ reference/
│  └─ concepts/
├─ developer-guide/     public, integrators
│  └─ quick-start.md
│  └─ guides/
│  └─ api-reference/
└─ internal/            not published
   └─ architecture/
   └─ decisions/
   └─ runbooks/

```

Use this for SaaS products and platforms with external users and integrators.

3. Central + co-located (monorepo)

```

repo/
├─ README.md             map of everything
├─ docs/                 cross-cutting, type-based
│  └─ architecture/
│  └─ conventions/
│  └─ decisions/
├─ services/
│  └─ billing/
│     └─ README.md       co-located
│     └─ docs/
│        └─ runbooks/
│        └─ decisions/

```

Use this for monorepos and modular monoliths. Rule: "If it's about one module, put it in the module. If it's about two or more, put it in `docs/`."

4. Domain → Type (large, team-aligned product)

```

docs/
├─ _shared/              getting started, conventions, glossary
├─ billing/
│  └─ README.md
│  └─ requirements/
│  └─ architecture.md
│  └─ decisions/
│  └─ runbooks/
├─ catalog/
└─ ...

```

Use this when each team owns a domain end to end. Add cross-domain index pages (all runbooks, all ADRs).

5. Lifecycle inside type (spec-heavy teams)

```
product/
├── README.md           index with status column
├── features/
│   ├── partial-refunds/  status: in-development
│   └── multi-currency/   status: draft
└── archive/             shipped & superseded specs
```

Use this for product teams with many specs. Living rules move to `requirements/rules/` once a feature ships.

■ Writing down the rules

Put this in the docs root `README.md`:

```
## How these docs are organized
```

- **Top level = doc type.** One folder per kind of document.
- **Second level = domain** (billing, catalog, identity) when a folder has >15 files.
- **ADRs** are numbered `NNNN-title.md` in `architecture/decisions/`.
- **Postmortems** are dated `YYYY-MM-DD-title.md`.
- **Module docs** live next to the code in `services/<name>/README.md`.
- **Status** is in front matter, not in folder names. Dead docs go to `archive/`.
- Not sure where something goes? Ask in #docs or put it in the closest folder and flag it in the PR.

MARKDOWN

■ Hybrid decision table

| Situation | Recommended hybrid |
|---|--|
| One product, 2–5 teams, internal docs | Type → Domain (#1) |
| Product with public user docs | Audience → Diátaxis (#2) |
| Monorepo | Central + co-located (#3) |
| Large product, teams own domains end to end | Domain → Type (#4) + cross-domain indexes |
| Many feature specs | Lifecycle inside type (#5) |
| Polyrepo | Per-service repos (p. 85) + handbook repo using #1 |

RELATED: [The Six Dimensions of Doc Organization](#) (p. 63) · [Folder Structures: Ready-Made Trees](#) (p. 104) · [Decision Guide](#) (p. 121)

CHAPTER 3.14

Storage Locations

What: *Where* documentation physically lives (repo root, docs folder, next to code, separate repo, wiki, docs site, issue tracker) and how to choose for each doc type. **Read when:** deciding between "in the repo" and "in Confluence/Notion", or when docs are spread over too many tools.

TL;DR

- **Default: Markdown in the code repo.** It's versioned, reviewed and close to the code.
- Use a **wiki/Notion** for content written mainly by non-engineers, or for fast-changing team notes. **Link** to it from the repo.
- Use a **docs site** to *publish* Markdown from the repo, not as a separate place to write.
- Decide the **source of truth per doc type** and write it down. The worst setup is the same doc type in two tools.

The locations

| Location | Example | Strengths | Weaknesses |
|------------------------------|--|--|--|
| Repo root files | <code>README.md</code> , <code>CONTRIBUTING.md</code> , <code>CHANGELOG.md</code> , <code>AGENTS.md</code> | Seen first; recognized by platforms and tools | Only for a handful of standard files |
| Docs folder in repo | <code>docs/</code> , <code>handbook/</code> | Versioned with code, PR-reviewed, docs-as-code | Non-engineers may find Git hard |
| Co-located with code | <code>services/billing/README.md</code> | Freshest; ownership automatic | No overview; hidden from business readers |
| Separate docs repo | <code>org/handbook</code> , <code>org/product-docs</code> | Independent release cycle; different access rights; non-dev contributors | Drifts from code; cross-repo PRs needed |
| Wiki / knowledge tool | Confluence, Notion, GitHub Wiki, Outline | Easy editing for everyone, comments, permissions | Weak versioning/review; stale pages pile up; not near code |
| Docs site (published) | MkDocs, Docusaurus, VitePress, Starlight, GitBook | Search, navigation, versioning, public access | Build/hosting effort; is a <i>view</i> , not a <i>source</i> |
| Issue tracker | Jira, Linear, GitHub Issues | Stories, tasks, discussion, status | Not a knowledge base; hard to read as a whole |
| Design tools | Figma, Miro | Visual specs, workshops | Not searchable as text; link, don't embed |
| Generated artifacts | API reference from OpenAPI, schema docs | Always accurate | Need a pipeline; can't hold "why" |

■ Source of truth per doc type (recommended defaults)

| Doc type | Source of truth | Mirrors / links |
|--|--|--|
| README, CONTRIBUTING, CHANGELOG, AGENTS.md | Repo root | — |
| Setup, conventions, architecture, ADRs | Repo docs folder | Published to internal docs site |
| Module/service docs | Co-located with code | Aggregated into portal/site |
| API reference | Spec file in repo (OpenAPI/SDL/proto) | Rendered on docs site |
| API guides | Repo docs folder | Published to developer portal |
| BRD / PRD | Repo if product + eng both use Git; otherwise wiki | Link from repo <code>requirements/README.md</code> |
| User stories | Issue tracker | Linked from PRD |
| Business rules, glossary | Repo (stable IDs, greppable, readable by AI agents) | Linked from wiki |
| Runbooks | Repo (readable during outages via Git host) | Linked from alerts |
| Postmortems | Repo or incident tool | Linked from runbooks |
| Meeting notes, team rituals | Wiki | — |
| Roadmap | Product tool / wiki | Linked from README |
| User docs | Repo → published docs site / help center | — |
| Designs | Figma | Linked from PRD |

■ Choosing: in the repo or in a wiki?

| Question | Repo | Wiki |
|---|------|----------------------|
| Does it change when code changes? | ✓ | |
| Will engineers be the main editors? | ✓ | |
| Should it be reviewed like code? | ✓ | |
| Must AI coding agents read it? | ✓ | |
| Are the main editors non-technical? | | ✓ |
| Is it discussion-heavy, with lots of comments? | | ✓ |
| Is it short-lived (meeting notes, brainstorming)? | | ✓ |
| Does it need fine-grained permissions? | | ✓ (or separate repo) |

Mostly ✓ in one column → put it there.

Repo layout options

Single repo, one docs folder (most projects)

```
repo/
├── README.md
├── docs/
└── src/
```

Monorepo: central + co-located

See [co-located model](#) (p. 82).

Separate docs repo

Use it when docs have a different audience, release cycle or access rights than the code, for example a public product docs site written by a docs team.

```
org/
├── product-app/      code (private)
└── product-docs/    public docs site source
```

! Add a CI check or a PR-template checkbox to the code repo: "Docs updated in product-docs? Link the PR."

Handbook repo (organization-level)

Org-wide engineering standards, onboarding and the architecture landscape. See [Model: Per-Service Repos + Central Portal](#) (p. 85).

Anti-patterns

| ✗ | Why it hurts | ✓ Instead |
|--|-------------------------------|---|
| Same doc type in Confluence <i>and</i> repo | Nobody knows which is current | Pick one; leave a link stub in the other |
| Copying Confluence pages into the repo "for AI" | Two drifting copies | Move the source of truth, or export automatically |
| Docs only in the docs site's CMS | Not versioned with code | Generate the site from repo Markdown |
| Runbooks hosted on infrastructure that fails with production | Unreachable during incidents | Keep them in Git as well |
| Personal Google Docs as specs | Lost when people leave | Team-owned location |

RELATED: [Internal vs Public Docs](#) (p. 102) · [Model: Co-located with Code](#) (p. 82) · [Model: Per-Service Repos + Central Portal](#) (p. 85)

CHAPTER 3.15

Internal vs Public Docs

What: How to keep internal documentation (architecture, runbooks, decisions) separate from published documentation (user guides, public API docs) so nothing leaks and both stay maintainable. **Read when:** any part of your docs is published outside the team, such as an open-source project, a public API or a customer help center.

TL;DR

- Make visibility a **hard boundary** (folder or repo), not just a metadata flag.
- **Publish only from an allow-listed folder.** The build should never scan everything and filter.
- Internal docs may **link** to public docs, but never the other way round.
- Review public docs for **secrets, internal hostnames, customer data and unannounced features.**

■ Three separation levels

| Level | Structure | Isolation | Use when |
|---------------------|---|------------------------------------|--|
| 1. Folder split | <code>docs/public/</code> + <code>docs/internal/</code> in one repo | Medium: relies on the build config | Private repo, public docs site built from <code>public/</code> only |
| 2. Separate repos | <code>product-docs</code> (public) + <code>product</code> (private) | Strong | Open-source docs with a closed-source product, or separate docs team |
| 3. Separate systems | Public docs site + internal wiki/handbook | Strongest | Large orgs, compliance requirements |

■ Folder split example

```
docs/
├── README.md           explains the split
├── public/             ← ONLY this folder is published
│   ├── user-guide/
│   ├── api/
│   └── changelog.md
└── internal/           never published
    ├── architecture/
    ├── decisions/
    ├── runbooks/
    └── requirements/
```

Docs site config points **only** at `docs/public/`:

```
# mkdocs.yml (example)
docs_dir: docs/public
```

YAML

■ What must never be public

| Category | Examples |
|--------------------------|---|
| Secrets & credentials | API keys, passwords, tokens (even "expired" ones) |
| Internal infrastructure | Hostnames, IPs, internal URLs, cloud account IDs, network diagrams |
| Security details | Unpatched vulnerabilities, security architecture specifics |
| Customer & personal data | Names, emails, IDs in examples or screenshots |
| Business-confidential | Pricing strategy, unannounced features, contracts, incident details with customer names |
| Internal people/process | On-call phone numbers, private Slack channels |

✓ Use **obviously fake data** in public examples: `example.com`, `sk_test_...`, "Jane Doe", and the documentation IP ranges `192.0.2.0/24` and `198.51.100.0/24`.

■ Safeguards

- Run **secret scanning** in CI on docs too (gitleaks, trufflehog, GitHub secret scanning).
- Add a **link check** that fails if public docs link into `internal/`.
- Use **CODEOWNERS**: public docs need a docs/product reviewer.
- Add a **PR template checkbox**: "No internal-only information in public docs."

RELATED: [Storage Locations](#) (p. 99) · [Model: Organize by Audience](#) (p. 70) · [The Six Dimensions of Doc Organization: Visibility: the exception](#) (p. 65)

CHAPTER 3.16

Folder Structures: Ready-Made Trees

What: Complete, copyable documentation trees for eight common project shapes, each annotated with the model it uses and why. **Read when:** you know roughly what kind of project you have and want a starting structure. To pick the shape, use the [decision guide](#) (p. 121).

TL;DR

- Start with the **smallest tree that fits**, and only create a folder when it has content.
- Every tree has the same core: **root README** → **docs index** → **a few topic folders** → **glossary**.
- `docs/` is used below as the docs root. Rename it if you want ([naming the docs root](#) (p. 110)).

| # | Project shape | Model |
|----------------------------|---|---|
| A (p. 104) | Small project / solo / MVP | Flat |
| B (p. 104) | Web application, single team | By doc type |
| C (p. 105) | SaaS product, several teams | Hybrid: type → domain |
| D (p. 106) | Monorepo | Central + co-located |
| E (p. 106) | Open-source library / SDK | Diátaxis + root community files |
| F (p. 107) | Public API product | Audience → Diátaxis, versioned |
| G (p. 107) | Enterprise / regulated / client project | Type, formal requirements, traceability |
| H (p. 108) | AI-agent-heavy repository | Any + agent layer |

■ A. Small project / solo / MVP

Model: [flat](#) (p. 93). **Why:** few docs, so structure would add friction.

```

project/
├── README.md           what, quick start, links
├── CHANGELOG.md
├── .env.example        documented env vars
└── docs/
    ├── README.md       one-line index
    ├── architecture.md one page + a Mermaid diagram
    ├── decisions.md     decision log table
    ├── deployment.md
    └── requirements.md  scope, user stories, business rules
  
```

Move on when `docs/` has about 15+ files, or you get a second team member → B.

■ B. Web application, single team

Model: [by doc type](#) (p. 68). **Why:** one team, and readers search by kind of doc.

```

project/
├── README.md
├── CONTRIBUTING.md
├── CHANGELOG.md
├── AGENTS.md                                optional, see H
├── docs/
│   ├── README.md                            index + "how these docs are organized"
│   ├── getting-started/
│   │   ├── README.md                        local setup
│   │   ├── configuration.md
│   │   └── troubleshooting.md
│   ├── architecture/
│   │   ├── README.md                        overview, C4 context + container
│   │   ├── data-model.md
│   │   └── decisions/
│   │       ├── README.md                    ADR index
│   │       └── 0001-record-architecture-decisions.md
│   ├── requirements/
│   │   ├── README.md
│   │   ├── prd-<feature>.md
│   │   ├── business-rules.md
│   │   └── non-functional.md
│   ├── api/
│   │   ├── README.md
│   │   └── openapi.yaml
│   ├── development/
│   │   ├── conventions.md
│   │   └── testing.md
│   ├── operations/
│   │   ├── deployment.md
│   │   └── runbooks/
│   └── glossary.md

```

■ C. SaaS product, several teams

Model: **Model: Hybrid (Combining Models)** (p. 96), type → domain. **Why:** type is still how people search, and domains keep folders small and owned.

```

product/
├── README.md
├── CONTRIBUTING.md
├── CHANGELOG.md
├── AGENTS.md
├── .github/CODEOWNERS                       docs/requirements/billing/ @team-billing ...
├── docs/
│   ├── README.md
│   ├── getting-started/
│   ├── architecture/
│   │   ├── README.md
│   │   ├── domains.md                      map: domain → team → services
│   │   └── decisions/
│   ├── requirements/
│   │   ├── README.md
│   │   ├── billing/
│   │   │   ├── README.md
│   │   │   ├── rules.md                    BR-BIL-xx
│   │   │   └── features/
│   │   │       ├── partial-refunds.md       status: shipped
│   │   │       └── multi-currency.md         status: draft
│   ├── catalog/
│   ├── identity/
│   └── non-functional.md
├── api/
│   └── README.md

```

```

├── guides/
├── reference/           generated
├── operations/
│   ├── environments.md
│   ├── deployment.md
│   ├── runbooks/
│   │   ├── README.md       index by alert name
│   │   ├── billing/
│   │   └── platform/
│   └── postmortems/
│       └── 2026-09-18-checkout-outage.md
├── rfcs/
│   └── 0042-multi-currency.md
├── archive/
└── glossary.md

```

■ D. Monorepo

Model: [central + co-located](#) (p. 82). **Why:** module docs stay fresh next to the code, and cross-cutting docs stay central.

```

monorepo/
├── README.md           map of apps/services/packages
├── CONTRIBUTING.md
├── AGENTS.md           global agent rules
├── docs/               cross-cutting only
│   ├── README.md
│   ├── architecture/
│   │   ├── README.md
│   │   └── decisions/   cross-cutting ADRs
│   ├── conventions/
│   ├── requirements/   product-level requirements
│   └── glossary.md
├── apps/
│   └── web/
│       ├── README.md
│       └── AGENTS.md    web-specific agent rules
├── services/
│   ├── billing/
│   │   ├── README.md
│   │   ├── openapi.yaml
│   │   └── docs/
│   │       ├── runbooks/
│   │       └── decisions/ service-scoped ADRs
├── packages/
│   └── ui-kit/
│       ├── README.md
│       └── src/**/*.*mdx component docs

```

Rule: one module → in the module; two or more → in `docs/`.

■ E. Open-source library / SDK

Model: [Diátaxis](#) (p. 76) for user docs + standard community files. **Why:** the main readers are users of the library, and contributors are secondary.

```

library/
├── README.md           pitch, install, 30-second example, links
├── CHANGELOG.md
├── CONTRIBUTING.md
└── CODE_OF_CONDUCT.md

```

```

├── SECURITY.md
├── LICENSE
├── .github/
│   ├── ISSUE_TEMPLATE/
│   └── PULL_REQUEST_TEMPLATE.md
├── docs/                                published to docs site
│   ├── index.md
│   ├── tutorials/
│   │   └── getting-started.md
│   ├── how-to/
│   │   ├── configure-retries.md
│   │   └── migrate-from-v2.md
│   ├── reference/
│   │   ├── api/                        generated from docstrings
│   │   └── configuration.md
│   ├── explanation/
│   │   └── design-principles.md
│   ├── contributing/                  maintainer docs (architecture, release process)
│   │   ├── architecture.md
│   │   └── releasing.md

```

■ F. Public API product

Model: audience → Diátaxis, with [versioned](#) (p. 91) reference. **Why:** external integrators need a polished portal, and internal docs must not leak.

```

api-product/
├── README.md
├── openapi/
│   ├── v1.yaml
│   └── v2.yaml                                source of truth
├── docs/
│   ├── public/                               ← published developer portal
│   │   ├── index.md
│   │   ├── quick-start.md                  tutorial
│   │   ├── authentication.md
│   │   ├── guides/                         how-to
│   │   ├── concepts/                      explanation: pagination, idempotency, errors, versioning
│   │   ├── reference/
│   │   │   ├── v2/                        generated
│   │   │   └── v1/                        deprecated banner
│   │   ├── webhooks.md
│   │   ├── sdk.md
│   │   ├── changelog.md
│   │   └── migration-v1-to-v2.md
│   └── internal/                           not published
│       ├── architecture/
│       ├── decisions/
│       ├── runbooks/
│       └── requirements/
└── llms.txt                                optional, served at site root

```

■ G. Enterprise / regulated / client project

Model: by doc type with formal requirements and traceability. **Why:** auditors, clients and sign-offs need IDs, versions and approvals.

```

project/
├── README.md
├── CHANGELOG.md
├── docs/
│   └── README.md

```

```

├── 00-governance/
│   ├── document-control.md    versions, approvers, review cycle
│   └── raci.md
├── 01-business/
│   ├── brd.md                BR-xx, signed off
│   └── stakeholders.md
├── 02-requirements/
│   ├── srs.md                FR-xx / NFR-xx
│   ├── use-cases/
│   │   └── UC-07-issue-refund.md
│   ├── business-rules.md
│   └── traceability-matrix.md  BR → FR → story → test
├── 03-architecture/
│   ├── architecture.md        (arc42 or similar)
│   └── decisions/
├── 04-security-compliance/
│   ├── data-flows.md
│   ├── threat-model.md
│   └── controls.md
├── 05-testing/
│   ├── test-strategy.md
│   └── test-plans/
├── 06-operations/
│   ├── deployment.md
│   ├── runbooks/
│   └── disaster-recovery.md
├── 07-user/
│   └── user-manual.md
└── glossary.md

```

Numbered top folders are justified [here](#), because these docs follow a formal sequence that contracts and audits refer to. See [Naming Conventions: Numbering folders and files](#) (p. 112).

■ H. AI-agent-heavy repository

Model: any of the above + an **agent layer** kept separate from human docs. **Why:** agents need short, explicit entry points, and their working files must not pollute curated docs.

```

project/
├── README.md
├── AGENTS.md                router: commands, conventions, docs map, boundaries
├── CLAUDE.md                "@AGENTS.md" import + Claude-specific notes
├── .claude/
│   ├── skills/
│   └── commands/
├── .agents/                 agent working area (not human docs)
│   ├── plans/              active implementation plans
│   ├── reports/            workflow decisions
│   └── scratch/             git-ignored
├── docs/                   human-curated (structure B, C or D)
│   ├── README.md
│   ├── architecture/
│   ├── requirements/
│   │   └── rules/
│   └── glossary.md
├── src/
│   └── billing/
│       └── AGENTS.md        area-specific rules

```

Details: [AI Agent Docs](#) (p. 56) • [Writing for AI Agents](#) (p. 149).

RELATED: [organizing overview](#) (p. 60) · [Scenarios: Recommended Setups](#) (p. 126) · [Model: Hybrid \(Combining Models\)](#) (p. 96)

CHAPTER 3.17

Naming Conventions

What: Rules for naming the docs root, folders and files, including numbering, dates and IDs. **Read when:** setting up a docs structure, or when you find `Final_Spec_v2 (copy).md` in your repo.

TL;DR

- Files and folders: **lowercase kebab-case**, `refund-rules.md`.
- UPPERCASE** only for conventional root files: `README.md`, `CONTRIBUTING.md`, `CHANGELOG.md`, `AGENTS.md`.
- Name files by **topic**, not by doc type or date (unless it's a [sequential/dated record](#) (p. 88)).
- Use **ISO dates** (`2026-10-09`) and **zero-padded numbers** (`0012`).
- Never put versions or statuses in file names: no `-v2`, `-final` or `-old`. Git and front matter handle those.

■ Naming the documentation root

`docs/` is the most widely recognized name. GitHub Pages, many doc tools and most developers expect it. Choose a different name when it signals something useful or avoids a clash.

| Name | Signals | Good for |
|--|--|---|
| <code>docs/</code> | Standard project documentation | Default for most projects |
| <code>documentation/</code> | Same, more explicit | Teams that dislike abbreviations |
| <code>handbook/</code> | How we work, team/org knowledge | Team or company handbooks, engineering handbooks |
| <code>guide/</code> , <code>guides/</code> | Instructional content | Single-purpose guide repos |
| <code>knowledge/</code> , <code>kb/</code> | Knowledge base | Support, internal KB |
| <code>wiki/</code> | Loosely structured, collaborative | Wiki-style repos (watch for junk-drawer drift) |
| <code>manual/</code> | User manual | Products with a formal manual |
| <code>website/</code> , <code>site/</code> | Docs site <i>source</i> including config/theme | Docusaurus/VitePress projects |
| <code>spec/</code> , <code>specs/</code> | Specifications | Requirements-driven or protocol projects |
| <code>design/</code> | Design docs / RFCs | Engineering design archives |
| <code>.agents/</code> , <code>planning/</code> | AI-agent working documents | Keep agent output separate from human docs |
| <i>No root: topic folders at repo root</i> | The repo is the docs | Docs-only repos, like this guide |

Rules:

- ✔ One docs root per repo (plus co-located module docs). Two competing roots (`docs/` and `documentation/`) cause confusion.
- ✔ If you rename it, say so in the root README: "Documentation lives in `handbook/`."

-  Check your tooling. Some hosts and generators default to `docs/`, so you may need a config change.

Files

| Rule |  |  |
|--|---|---|
| Lowercase kebab-case | <code>refund-rules.md</code> | <code>Refund_Rules.md</code> , <code>refundRules.md</code> , <code>refund rules.md</code> |
| Topic, not type | <code>architecture/payments.md</code> | <code>architecture/architecture-doc-payments.md</code> |
| No version/status in name | <code>api-guide.md</code> + git tag | <code>api-guide-v2-final.md</code> |
| No dates (for living docs) | <code>deployment.md</code> | <code>deployment-2026.md</code> |
| Short but specific | <code>stripe-webhooks.md</code> | <code>information-about-how-stripe-webhooks-work.md</code> , <code>misc.md</code> |
| <code>.md</code> extension | <code>setup.md</code> | <code>setup.markdown</code> , <code>setup.txt</code> |
| <code>README.md</code> as folder index | <code>runbooks/README.md</code> | <code>runbooks/index-of-runbooks.md</code> |

`README.md` or `index.md`? GitHub/GitLab display `README.md` automatically when browsing a folder. Many doc site generators expect `index.md`, though most can be configured to use `README`. **For repo-browsed docs, use `README.md`.**

Sequential and dated files

| Pattern | Example | Use for |
|--|--|---------------------------|
| <code>NNNN-kebab-title.md</code> | <code>0012-use-stripe.md</code> | ADRs, RFCs |
| <code>YYYY-MM-DD-kebab-title.md</code> | <code>2026-09-18-checkout-outage.md</code> | Postmortems, meetings |
| <code>YYYY-MM.md</code> | <code>2026-10.md</code> | Monthly release notes |
| <code>vX.Y.md</code> or by SemVer | <code>v2.3.md</code> | Per-version release notes |

ID-prefixed files

For formal requirements, put the ID first so files sort and grep by ID:

```
use-cases/UC-07-issue-refund.md
requirements/FR-PAY-012-refund-approval.md    (only if one file per requirement)
```

Common ID schemes: `BR-` business requirement/rule, `FR-` functional, `NFR-` non-functional, `UC-` use case, `US-` user story, `ADR-`, `RFC-`, `TC-` test case. Add a domain code for scale: `BR-PAY-04`.

Folders

| Rule | Example |
|--|--|
| Lowercase kebab-case, plural for collections | <code>runbooks/</code> , <code>decisions/</code> , <code>use-cases/</code> |
| Singular for a single topic | <code>architecture/</code> , <code>billing/</code> |
| One dimension per level | see Model: Hybrid (Combining Models): The golden rule: one dimension per level (p. 96) |
| Max ~3 levels below docs root | <code>docs/requirements/billing/rules.md</code> |

| Rule | Example |
|--|--|
| Underscore prefix for special/shared folders (sorts first) | <code>_shared/</code> , <code>_templates/</code> , <code>_assets/</code> |
| Assets near their docs | <code>architecture/images/c4-context.png</code> |

■ Numbering folders and files

Prefixes like `01-` , `02-` force an order.

| Use numbering when | Avoid numbering when |
|---|---|
| Content is read in sequence (tutorial steps, a course, onboarding days) | Folders are topics without an inherent order |
| A formal document set is referenced by number (enterprise/regulated, see structure G (p. 107)) | You'll insert items later (renumbering breaks links) |
| Records need stable IDs (ADRs) | Readers navigate directly by name. Names are easier to type and remember. |

If you number: zero-pad (`01` , not `1`), leave gaps for inserts (`10-` , `20-` , `30-`), and never renumber published items.

■ Headings and titles

- The H1 **matches the file's topic**: `refund-rules.md` → `# Refund Rules`.
- For records, include the ID: `# ADR-0012: Use Stripe for card payments`.
- Use sentence case and descriptive wording: `## Rotate the API key` , not `## Procedure 2`.

RELATED: [Navigation & Metadata](#) (p. 113) · [Model: Sequential & Dated Records](#) (p. 88) · [Style Guide](#) (p. 137)

CHAPTER 3.18

Navigation & Metadata

What: How to make docs findable: folder index pages, cross-linking, breadcrumbs, front matter schema, status, tags and ownership. **Read when:** your structure exists but people (or AI agents) still can't find things, or docs go stale unnoticed.

TL;DR

- Every folder has a `README.md` that lists its contents with one line each.
- Every page links up (to its folder index) and across (to related pages).
- Use a **small, fixed front matter schema** (owner, status, last reviewed) and validate it.
- Show **status** visibly on the page, not just in metadata.
- Navigation is a **tree of indexes**: root README → docs index → folder indexes → pages.

■ The index tree

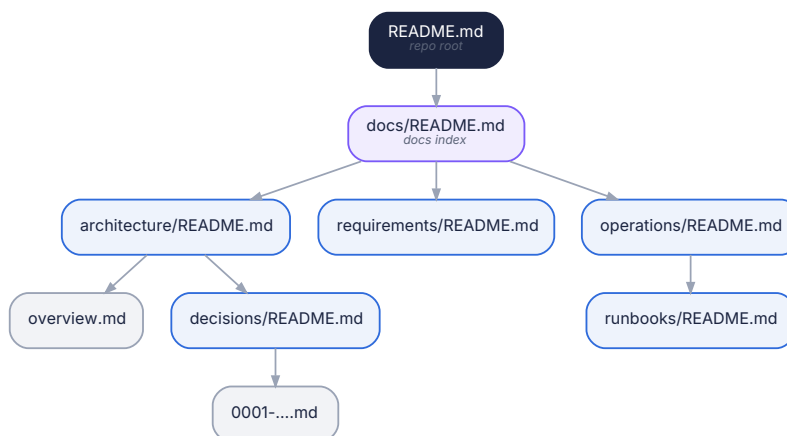


Figure 3.4 Every level links down through a README index.

Every page is reachable from the root in **at most 3–4 clicks**, and no page is an **orphan** (unlinked).

■ Folder index (README.md) pattern

Operations

MARKDOWN

> Deploying, configuring and running the system in production.
 > **Owner:** team-platform

| Page | Description |
|------------------------------------|--------------------------------|
| environments.md | Environments, URLs, access |
| deployment.md | Release and rollback procedure |

```
| [runbooks/](runbooks/README.md) | One runbook per alert |
| [postmortems/](postmortems/README.md) | Incident write-ups, newest first |
```

Keep descriptions to one line. The index is a **menu**, not a summary of the content.

■ The docs root index

docs/README.md (or your docs root's index) should contain:

1. Who these docs are for, with an audience → start-page table.
2. Map: the folders and what each holds.
3. How these docs are organized: the rules ([Model: Hybrid \(Combining Models\): Writing down the rules](#) (p. 98)).
4. How to contribute docs: templates, review, ownership.
5. Most-used pages: the 5–10 pages people open most.

■ Cross-linking rules

| Link type | Where | Example |
|-----------------|-------------------------------|--|
| Up | Bottom (or top) of every page | <code>**Up:** [Operations](README.md)</code> |
| Related | Bottom of page | <code>**Related:** [deployment](deployment.md) · [ADR-0012](...)</code> |
| Inline | Where a concept first appears | <code>...requires [approval](../requirements/rules.md#br-pay-04)...</code> |
| Canonical | Instead of duplicating | <code>> Canonical source: [refund rules](...)</code> |
| Back-references | From code to docs | <code>// See docs/requirements/billing/rules.md#br-bil-04</code> |

✅ Link to **specific sections** with anchors when you can. ❌ Don't write "see the docs". Say *which* doc.

■ Front matter schema

Keep it small and the same everywhere. Recommended fields:

```
---
title: Refund rules                                # optional if H1 is the same
owner: team-billing                               # team or @person responsible for accuracy
status: active                                     # draft | review | active | deprecated | archived
last_reviewed: 2026-10-01                         # ISO date, updated on every real review
audience: [product, qa, developers]              # optional
tags: [refunds, payments]                         # optional, from a controlled list
---
```

YAML

Extra fields for specific types:

| Doc type | Extra fields |
|--------------|--|
| ADR | <code>id</code> , <code>date</code> , <code>deciders</code> , <code>supersedes</code> , <code>superseded_by</code> |
| RFC / PRD | <code>id</code> , <code>reviewers</code> , <code>target_release</code> , <code>decision_date</code> |
| Requirements | <code>id</code> , <code>priority</code> , <code>source</code> , <code>verification</code> |
| Runbook | <code>alert</code> , <code>severity</code> , <code>service</code> |
| Versioned | <code>since</code> , <code>deprecated_in</code> |

Validate it in CI with a small script or a JSON-schema check over front matter, so typos like `staus:` or unknown statuses fail the build.

Status vocabulary

| Status | Meaning | Visible banner |
|-------------------------|---|-----------------------------------|
| <code>draft</code> | Being written; may be wrong | ● Draft |
| <code>review</code> | Ready for feedback | ● In review |
| <code>active</code> | Current and maintained | (none) |
| <code>deprecated</code> | Still valid for old versions / being replaced | ● Deprecated, link to replacement |
| <code>archived</code> | Historical only | ■ Archived |



Figure 3.5 The status vocabulary as a lifecycle. Archived pages are eventually deleted; Git keeps the history.

■ Tags

Tags give you **many-to-many** grouping that folders can't. Use them when:

- Docs sites can build tag pages (Docusaurus, MkDocs Material and others).
- You need cross-cutting views: `security`, `gdpr`, `performance`.

Rules: use a **controlled list** (`tags.md` or the site config), keep them lowercase, and give a page no more than about 5 tags.

■ Search

- Published docs sites: built-in search, or Algolia DocSearch for large sites.
- Repos: GitHub/GitLab code search, and `grep` / `rg` for agents and developers.
- Help search by **using the words readers use**, including synonyms ("refund / chargeback / money back") in the text or in a `keywords:` field.

■ Checklist

- ☐ Root README links to the docs index
- ☐ Every folder has a README index
- ☐ Every page has an "Up" link
- ☐ No orphan pages (check with a script or link checker)
- ☐ Front matter schema documented and validated
- ☐ Status visible on draft/deprecated/archived pages

RELATED: [Naming Conventions](#) (p. 110) · [Maintenance](#) (p. 152) · [Writing for AI Agents](#) (p. 149)

CHAPTER 3.19

Organization Smells & Fixes

What: Symptoms that your documentation structure isn't working, what usually causes them, and how to fix each one. **Read when:** people complain they "can't find anything", or before a docs cleanup.

TL;DR

- Most problems come from **mixed dimensions**, **missing indexes**, **duplication** and **no owner**.
- Fix structure **incrementally**, one folder at a time, with redirects or link updates.
- Don't add folders to solve a findability problem. **Add indexes first**.

■ Smell catalog

| # | Smell | Likely cause | Fix |
|----|--|--|--|
| 1 | "Where do I put this?" asked every week | No written organizing rules; mixed dimensions at one level | Write the rules (Model: Hybrid (Combining Models): Writing down the rules (p. 98)); one dimension per level |
| 2 | <code>misc/</code> , <code>other/</code> , <code>general/</code> , <code>notes/</code> folders | Missing category or unneeded docs | Classify each file: move, merge or delete |
| 3 | <code>-v2</code> , <code>-final</code> , <code>-old</code> , <code>(copy)</code> in names | Versioning by file name | Use Git; status in front matter; delete old copies |
| 4 | Same info in 3 places, all different | No single source of truth | Choose canonical; replace others with links |
| 5 | Folder with 50+ files | Primary dimension too coarse | Add secondary dimension subfolders (The Six Dimensions of Doc Organization (p. 63)) |
| 6 | 5 levels of nesting | Too many dimensions as folders | Flatten; move dimensions to metadata |
| 7 | Folders with 1 file each | Structure created "for later" | Collapse; create folders when content exists |
| 8 | Nobody knows if a page is current | No owner/status/review date | Add front matter; review cycle (Maintenance (p. 152)) |
| 9 | Old specs rank above current docs in search | No lifecycle handling | <code>status</code> , banners, <code>archive/</code> , exclude archive from search |
| 10 | Orphan pages nobody links to | No index pages | README index per folder; orphan check in CI |
| 11 | Business people never read the docs | Docs hidden in code folders or in Git only | Business-facing index; publish to site/wiki; link from tools they use |
| 12 | Runbooks not found during incidents | Organized by system, not by alert | Name by alert; link from alerts |
| 13 | Docs in Confluence <i>and</i> repo disagree | Two sources of truth | Pick one per doc type (Storage Locations (p. 99)) |
| 14 | AI agents give wrong answers about the project | Ambiguous/outdated docs; no agent entry point | AGENTS.md + docs map; fix stale pages (Writing for AI Agents (p. 149)) |

| # | Smell | Likely cause | Fix |
|----|--|--------------------------------|---|
| 15 | Agent-generated plans and reports clutter <code>docs/</code> | No separate agent working area | Move to <code>.agents/</code> or similar; promote reviewed docs only (AI Agent Docs: Where to put agent-generated documents (p. 58)) |
| 16 | Every module README looks different | No template | Template + review checklist |

■ Quick health check (10 minutes)

Score 1 point per "yes":

- ☐ A newcomer can find setup instructions from the root README in one click.
- ☐ The docs root has an index explaining how docs are organized.
- ☐ No folder has more than ~20 files without subfolders.
- ☐ No `misc/other/old` folders.
- ☐ Every important doc has an owner.
- ☐ Draft/deprecated docs are visibly marked.
- ☐ There's one source of truth per doc type.
- ☐ A link checker runs in CI.
- ☐ Business rules have stable IDs in one place.
- ☐ AI agents have an entry file pointing to the docs map.

8–10: healthy. 5–7: fix the gaps one by one. 0–4: plan a restructure ([Evolving the Structure](#) (p. 129)).

RELATED: [Evolving the Structure](#) (p. 129) · [Principles of Good Documentation: Common anti-patterns](#) (p. 15) · [Maintenance](#) (p. 152)

PART IV

Choosing a Structure

Which structure fits your project

[choosing-structure/](#)

| | |
|-------------------------------|-----|
| Overview | 120 |
| Decision Guide | 121 |
| Scenarios: Recommended Setups | 126 |
| Evolving the Structure | 129 |

PART IV

Overview

What: How to pick the right doc types, organizing model and storage for *your* project, and how to change course later. **Read when:** you're starting docs for a project, or you suspect your current structure doesn't fit.

■ Pages

| Page | Use it to | Time |
|--|---|--------------------|
| Decision Guide (p. 121) | Answer 8 questions and get a recommended structure (decision tree + scoring matrix) | 10 min |
| Scenarios: Recommended Setups (p. 126) | Find your project shape and copy the recommended setup | 5 min per scenario |
| Evolving the Structure (p. 129) | Know when to restructure and do it without breaking links | 8 min |

■ The short version

1. List your audiences ([Documentation Audiences](#) (p. 16)).
2. List the doc types you actually need now ([doc-types catalog](#) (p. 26)). Not "someday".
3. Pick the primary dimension by how readers search ([The Six Dimensions of Doc Organization](#) (p. 63)).
4. Pick storage per doc type ([Storage Locations](#) (p. 99)).
5. Copy the matching tree ([Folder Structures: Ready-Made Trees](#) (p. 104)).
6. Write the rules in the docs index, and revisit them every 6–12 months.

CHAPTER 4.1

Decision Guide

What: A step-by-step way to choose doc types, an organizing model and storage for a project, using questions, a decision tree and a scoring matrix. **Read when:** setting up or reorganizing documentation. AI agents should apply this before creating a doc structure.

TL;DR

- Answer the **8 questions** below and write the answers down. They become the "How these docs are organized" section.
- Follow the **decision tree** to get a primary model.
- If two models come out close, use the **scoring matrix**.
- When in doubt: **by doc type**, upgraded later to **type → domain** ([Model: Hybrid \(Combining Models\)](#) (p. 96)).

■ Step 1: Answer 8 questions

| # | Question | Why it matters | Typical answers → effect |
|----|---|--|---|
| Q1 | How many docs will exist in 6 months? | Size decides how much structure you need | <20 → flat · 20–150 → one-level folders · 150+ → two levels / hybrid |
| Q2 | Who are the readers? (list them) | Distinct audiences may need separate areas | Mostly developers → type · Users + devs → audience split · Many → audience → Diátaxis |
| Q3 | Is any of it public? | Visibility needs a hard split | Yes → <code>public/</code> vs <code>internal/</code> or separate repo (Internal vs Public Docs (p. 102)) |
| Q4 | How many teams, and do they own separate domains? | Ownership should match folders | 1 team → type · 3+ teams with domains → type → domain or domain → type |
| Q5 | Repo shape? | Decides co-location | Single repo → docs folder · Monorepo → central + co-located · Polyrepo → per-repo + portal |
| Q6 | Who writes the docs? | Decides storage | Engineers → repo · Mixed → repo + wiki for business · Non-technical → wiki/CMS (source) linked from repo |
| Q7 | Are there several live versions? | Versioned docs are expensive | No → Git only · Yes, public → versioned site |
| Q8 | How formal must requirements be? | Decides requirement doc types and IDs | Startup → PRD + stories · Client/regulated → BRD + SRS + traceability |

Bonus questions:

- **Will AI agents work in the repo?** → add `AGENTS.md`, stable IDs and a separate agent working folder ([AI Agent Docs](#) (p. 56)).
- **Is there production on-call?** → you need runbooks, named by alert.

■ Step 2: Decision tree

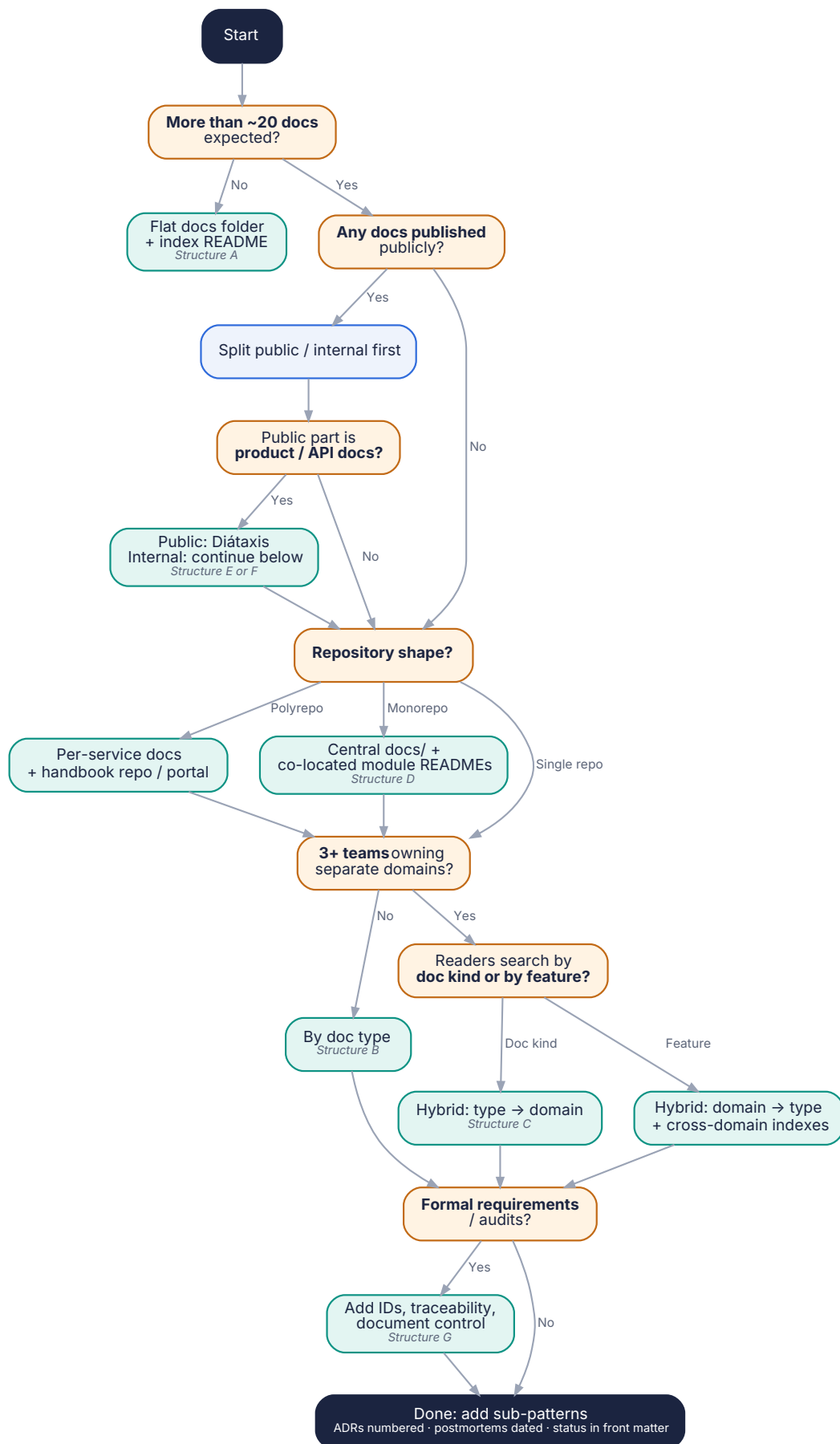


Figure 4.1 The decision tree. Orange boxes are questions, green boxes are recommended structures.

The structure letters refer to the trees in [Folder Structures: Ready-Made Trees](#) (p. 104).

■ Step 3: Scoring matrix (when two options are close)

Weight each criterion 1–3 by importance to *your* project, score each model 1–3, then multiply and sum.

| Criterion | Weight (you) | By type | By audience | By domain | Diátaxis | Type → domain | Co-located + central |
|------------------------------|--------------|---------|-------------|-----------|----------|---------------|----------------------|
| Newcomers find things fast | | 3 | 3 | 2 | 3 | 3 | 2 |
| Clear team ownership | | 1 | 2 | 3 | 1 | 3 | 3 |
| Scales past 150 docs | | 2 | 2 | 3 | 3 | 3 | 3 |
| Docs stay in sync with code | | 2 | 2 | 2 | 2 | 2 | 3 |
| Easy to start today | | 3 | 2 | 2 | 1 | 2 | 2 |
| Serves non-technical readers | | 2 | 3 | 3 | 3 | 2 | 1 |
| Safe public/internal split | | 1 | 3 | 1 | 2 | 1 | 1 |

Example: weights [3, 2, 1, 2, 3, 1, 1]. "By type" = $3 \cdot 3 + 2 \cdot 1 + 1 \cdot 2 + 2 \cdot 2 + 3 \cdot 3 + 1 \cdot 2 + 1 \cdot 1 = 29$. "Type → domain" = $9 + 6 + 3 + 4 + 6 + 2 + 1 = 31$. Close, so start with by type, and plan domain subfolders for when folders grow.

■ Step 4: Decide storage per doc type

Fill in this table (defaults in [Storage Locations: Source of truth per doc type \(recommended defaults\)](#) (p. 100)):

```

Doc type	Source of truth	Published to	Owner
Architecture & ADRs	repo `docs/architecture/`	internal site	tech lead
PRDs	repo `docs/requirements/`	-	product
User stories	Jira	-	product
Runbooks	repo `docs/operations/runbooks/`	-	platform team
User guide	repo `docs/public/`	help site	docs team

```

MARKDOWN

■ Step 5: Record the decision

Put the results in the docs index (and optionally an ADR such as "ADR-0002: Documentation structure"):

```
## How these docs are organized
```

MARKDOWN

- **Model:** by doc type; domain subfolders when a folder exceeds 15 files.
- **Public docs:** none yet.
- **Storage:** Markdown in this repo is the source of truth, except user stories (Jira) and meeting notes (Notion).
- **Records:** ADRs numbered in `architecture/decisions/`; postmortems dated.
- **Metadata:** owner, status, last_reviewed in front matter.
- **Agents:** AGENTS.md at root; agent working files in `.agents/` (not in docs).
- **Review:** structure revisited every 6 months.

■ For AI agents: applying this guide

When asked to create documentation for a project:

1. **Inspect** the repo: languages, monorepo or not, existing docs, `.github/`, existing agent files, API specs.

2. **Infer** answers to Q1–Q8 from the evidence. Ask the human only about what can't be inferred (usually Q2, Q3, Q6, Q8).
3. **Propose** the tree (from [Folder Structures: Ready-Made Trees](#) (p. 104)) and the "How these docs are organized" text **before** creating files.
4. **Create** only folders that will contain content now, each with a `README.md` index.
5. **Use** [Templates overview](#) (p. 156) and fill in real project facts. Never leave a template's placeholder text in a "finished" doc.
6. **Keep** your own working notes and plans **out of** the human docs root.

RELATED: [Scenarios: Recommended Setups](#) (p. 126) · [The Six Dimensions of Doc Organization](#) (p. 63) · [organizing overview](#) (p. 60)

CHAPTER 4.2

Scenarios: Recommended Setups

What: Recommended documentation setups for common project types: which docs to write first, which model to use, and where to store each doc type. **Read when:** you recognize your project in one of the scenarios and want a proven starting point.

TL;DR

| # | Scenario | Model | Tree |
|----|---------------------------------|--|---|
| 1 | Solo developer / prototype | Flat | A (p. 104) |
| 2 | Startup, one product team | By doc type | B (p. 104) |
| 3 | Scale-up, several product teams | Type → domain | C (p. 105) |
| 4 | Monorepo platform | Central + co-located | D (p. 106) |
| 5 | Microservices in many repos | Per-service + handbook/portal | Model: Per-Service Repos + Central Portal (p. 85) |
| 6 | Open-source library | Diátaxis + community files | E (p. 106) |
| 7 | Public API / developer platform | Public/internal → Diátaxis, versioned | F (p. 107) |
| 8 | Agency / client project | Type + formal requirements | G (p. 107) (lighter) |
| 9 | Enterprise / regulated | Type + IDs + traceability + document control | G (p. 107) |
| 10 | AI-agent-driven development | Any + agent layer | H (p. 108) |

1. Solo developer / prototype

- **Write first:** README (what + how to run), `.env.example`, a short decision log.
- **Skip for now:** PRDs, runbooks, CONTRIBUTING.
- **Model:** flat `docs/` with <10 files.
- **Watch for:** "I'll remember why I did that". You won't. Keep the decision log.

2. Startup, one product team

- **Write first:** README, getting started, architecture overview (1 page + diagram), ADRs, lightweight PRDs per feature, glossary, deployment.
- **Model:** [by doc type](#) (p. 68).
- **Storage:** everything in the repo; user stories in the tracker.
- **Watch for:** PRDs piling up. Mark them `shipped` and move business rules into a living rules doc.

■ 3. Scale-up, several product teams

- **Write first:** everything from #2, plus a domain map (domain → team → services), `CODEOWNERS` for docs, a business rules catalog per domain, runbooks per domain, and RFCs for cross-team changes.
- **Model:** [hybrid type → domain](#) (p. 96).
- **Watch for:** each team inventing its own layout. Publish domain-folder templates.

■ 4. Monorepo platform

- **Write first:** root README with a package map, module README template, central architecture + conventions, nested `AGENTS.md` per app.
- **Model:** [central + co-located](#) (p. 82).
- **Tooling:** a CI check that each package has a README; optional aggregated docs site.
- **Watch for:** central docs duplicating module READMEs. Apply the "one module vs many" rule.

■ 5. Microservices in many repos

- **Write first:** repo template with a standard docs layout, handbook repo (landscape, standards, onboarding, glossary), service catalog.
- **Model:** [per-service repos + portal](#) (p. 85).
- **Watch for:** cross-service flows documented nowhere. Put sequence diagrams of key flows in the handbook.

■ 6. Open-source library

- **Write first:** README with a 30-second example, getting-started tutorial, API reference (generated), CHANGELOG, CONTRIBUTING, SECURITY, LICENSE, CODE_OF_CONDUCT.
- **Model:** [Diátaxis](#) (p. 76); maintainer docs in a separate `contributing/` section.
- **Publishing:** docs site with versioning once there are multiple major versions in use.
- **Watch for:** README trying to be the whole manual.

■ 7. Public API / developer platform

- **Write first:** quick start (first call in 5 minutes), authentication, error catalog, generated reference, webhooks, changelog, versioning policy.
- **Model:** public/internal split; public is Diátaxis; reference [versioned](#) (p. 91).
- **Extras:** `llms.txt`, SDK examples, Postman/Bruno collection, status page link.
- **Watch for:** internal hostnames in examples. Apply the [Internal vs Public Docs](#) (p. 102) safeguards.

■ 8. Agency / client project

- **Write first:** BRD or signed scope, PRD/SRS with requirement IDs, change request log, handover docs (deployment, admin guide, runbooks), user manual.
- **Model:** by doc type with formal requirements; optionally numbered top folders for the deliverable set.
- **Storage:** repo as source; export to PDF for sign-off (with version + approval table).
- **Watch for:** requirement changes agreed in email and never documented. Keep a `requirements/change-log.md`.

■ 9. Enterprise / regulated

- **Write first:** document control (versions, approvals, review cycle), BRD, SRS with IDs, traceability matrix, security/compliance docs (data flows, threat model, controls), test strategy, DR plan.
- **Model:** [structure G](#) (p. 107).
- **Tooling:** front-matter validation, required reviewers via CODEOWNERS, generated traceability.
- **Watch for:** docs written for auditors that engineers never read. Keep engineer-facing docs (setup, conventions) separate and lean.

■ 10. AI-agent-driven development

- **Write first:** `AGENTS.md` (commands, boundaries, docs map), glossary, business rules with stable IDs, architecture overview, conventions with / examples.
- **Model:** any of the above plus a separate agent working area (`.agents/` or similar).
- **Practices:** specs/plans written before implementation; reviewed plans promoted to ADRs/design docs; agent output marked as such.
- **Watch for:** agents writing straight into `docs/` and creating noise. Review before promoting. See [AI Agent Docs](#) (p. 56).

RELATED: [Decision Guide](#) (p. 121) · [Folder Structures: Ready-Made Trees](#) (p. 104) · [Evolving the Structure](#) (p. 129)

CHAPTER 4.3

Evolving the Structure

What: When to change your documentation structure, and how to migrate without losing content or breaking links. **Read when:** your current structure has outgrown the project, or you're merging docs from several places.

TL;DR

- Structures should **evolve with the project**. Start small and add levels when the signals appear.
- **Restructure incrementally**, one folder at a time, in separate PRs.
- **Inventory first, move second, fix links third, delete last**.
- Preserve history with `git mv`, and add redirects if docs are published.

■ Typical evolution path

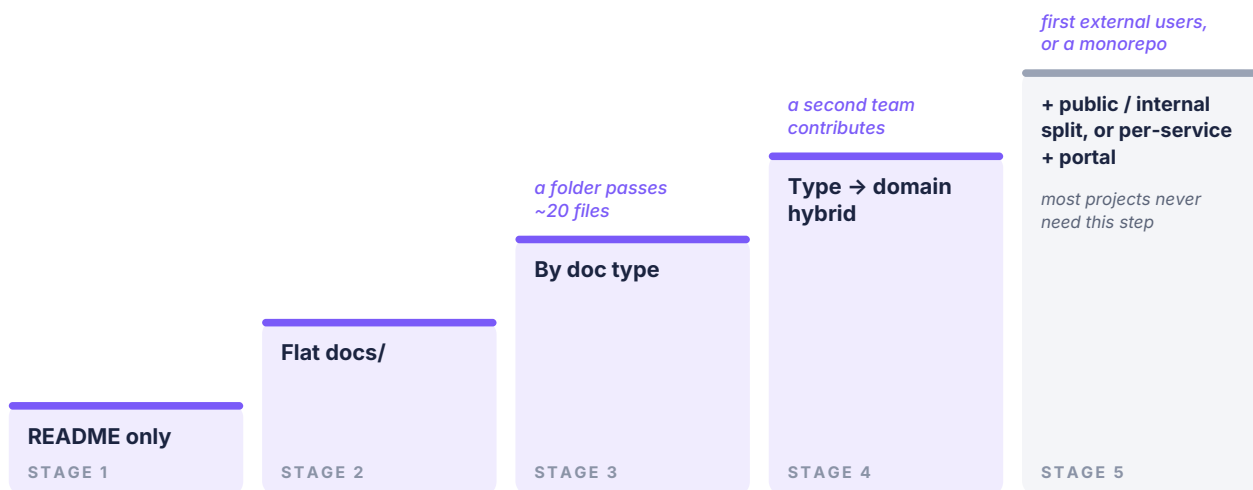


Figure 4.2 The typical evolution path. Each step is annotated with a typical trigger from the signals table below.

Most projects never need the last step, and that's fine.

■ Signals that it's time to change

| Signal | Suggested change |
|--|--|
| A folder has more than ~20 files | Add secondary-dimension subfolders |
| "Where does this go?" asked repeatedly | Write rules; maybe change primary dimension |
| A second team starts contributing | Ownership via domain subfolders + CODEOWNERS |
| First external users / public docs | Public/internal split |

| Signal | Suggested change |
|---|------------------------------|
| Repo becomes a monorepo | Co-locate module docs |
| Docs in two tools disagree | Consolidate source of truth |
| Search returns mostly outdated pages | Lifecycle metadata + archive |
| Health check score below 5 (smells (p. 118)) | Planned restructure |

■ Migration procedure

1. Inventory

List every doc, including those in wikis and personal drives, in a table:

```
MARKDOWN
Current path	Type	Audience	Domain	Status	Owner	Action	New path
wiki/Payments Spec	PRD	product	billing	shipped	@anna	move+archive	docs/archive/prd-payments.md
docs/setup.md	guide	devs	-	active	@li	move	docs/getting-started/README.md
docs/notes-old.md	?	?	?	stale	-	delete	-
```

The **Action** column is one of: keep, move, merge, split, archive, delete.

2. Agree on the target

Write the new structure and rules (see [decision-guide step 5](#) (p. 124)) and get team agreement **before** moving files.

3. Move in small batches

```
SHELL
git mv docs/setup.md docs/getting-started/README.md
```

- One folder or doc type per PR. Reviewers can follow it, and conflicts stay small.
- Don't edit content and move files in the same commit, so Git can track the rename.

4. Fix links

- Run a link checker (lychee, markdown-link-check) after each batch.
- Search for old paths: `rg "docs/setup.md"`. Check code comments, AGENTS.md and CI configs too.
- For **published sites**, add redirects from old URLs (most generators support a redirects plugin or config).

5. Add indexes and metadata

Create `README.md` indexes in new folders and add front matter (owner, status) as you go.

6. Delete or archive

Remove what the inventory marked `delete`. Git keeps the history. Archive what has historical value, with a banner.

7. Announce

Tell the team, update onboarding docs and agent instruction files, and pin the "How these docs are organized" section.

■ Consolidating from a wiki into the repo

1. Export pages to Markdown (most tools support it, or use a converter such as pandoc).
2. Clean up: fix headings, remove tool-specific macros, re-link images.
3. Leave a **stub** in the wiki: "Moved to `<repo link>`. This page is no longer maintained."
4. Restrict editing on the old pages to prevent forks.

■ Splitting a giant document

1. Identify sections with different audiences or lifecycles ([one doc or two?](#) (p. 28)).
2. Create the new docs and move sections over.
3. Turn the original into a short **index page** linking to the parts, so existing links keep working.

RELATED: [Organization Smells & Fixes](#) (p. 117) · [Decision Guide](#) (p. 121) · [Maintenance](#) (p. 152)

PART V

Writing

How to write, illustrate and maintain docs

writing/

| | |
|-----------------------|-----|
| Overview | 133 |
| Writing Process | 134 |
| Style Guide | 137 |
| Page Patterns | 141 |
| Diagrams & Visuals | 146 |
| Writing for AI Agents | 149 |
| Maintenance | 152 |

PART V

Overview

What: How to plan, write, illustrate, review and maintain documentation that humans and AI agents can both use. **Read when:** you're about to write or review a doc.

■ Pages

| Page | What you'll learn | Time |
|---|---|--------|
| Writing Process (p. 134) | Plan → outline → draft → review → publish → maintain | 6 min |
| Style Guide (p. 137) | Voice, sentences, headings, lists, tables, code, terminology | 10 min |
| Page Patterns (p. 141) | The skeleton of each kind of page (tutorial, how-to, reference, explanation, decision, overview, index) | 10 min |
| Diagrams & Visuals (p. 146) | When and how to use diagrams, screenshots and tables | 7 min |
| Writing for AI Agents (p. 149) | Making docs reliable input for LLMs and coding agents | 8 min |
| Maintenance (p. 152) | Ownership, review cycles, linting, link checks, CI, deprecation | 8 min |

■ The five rules if you read nothing else

1. **Start with the reader's goal.** First line: what this page helps them do.
2. **Answer first, details after.** TL;DR, then steps, then background.
3. **One idea per paragraph, one action per step.**
4. **Show real examples** that work when copy-pasted.
5. **Name things consistently.** The same term for the same concept, everywhere ([Glossary](#) (p. 23)).

CHAPTER 5.1

Writing Process

What: A repeatable process for producing a document: plan, outline, draft, review, publish, maintain. **Read when:** before writing any doc longer than one screen.

TL;DR

- **Plan (5 min):** audience, goal, doc type, location. Write them down.
- **Outline before drafting.** Headings only, and get early feedback on those.
- **Draft fast, then edit hard.** Cut about 30%.
- **Test it:** follow your own steps on a clean environment, or have a newcomer do it.
- **Review in a PR,** with a checklist.
- **Assign an owner** and a review date when you publish.

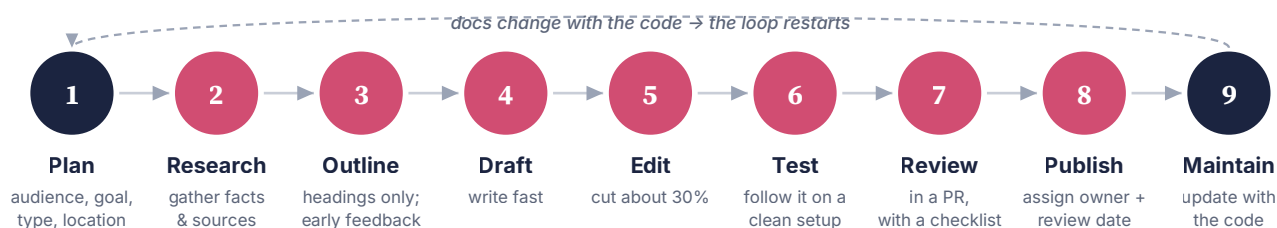


Figure 5.1 The nine steps of the writing process. Maintenance feeds back into planning.

1. Plan

Fill in this card at the top of your draft (delete it or keep it as front matter later):

```

<!--
Audience:  new backend developers
Goal:       run the billing service locally and process a test payment
Doc type:   how-to guide           (see doc-types/README.md)
Location:   docs/getting-started/billing-local.md
Owner:      team-billing
Success:    a new dev completes it in <20 min without asking for help
-->
  
```

MARKDOWN

If you can't fill in **Goal** in one sentence, the doc isn't ready to be written.

2. Research

- Collect facts from code, specs, existing docs, tickets and experts.
- Find what **already exists**. Link to it instead of rewriting it (single source of truth).
- Note open questions, and don't guess. Write `TODO(@owner): confirm timeout value` and resolve it before publishing.

■ 3. Outline

Write the headings first, using the [page pattern](#) (p. 141) for your doc type:

```
# Run billing locally
## Prerequisites
## Start dependencies
## Configure Stripe test keys
## Run the service
## Process a test payment
## Troubleshooting
```

MARKDOWN

Share the outline for a 2-minute sanity check. That's the cheapest moment to catch a wrong scope.

■ 4. Draft

- Write quickly, and don't polish yet.
- Fill each section with: **what to do** → **example** → **expected result**.
- Use real commands, real requests and real field names.
- Mark gaps with `TODO`.

■ 5. Edit

Make one pass for each of these:

| Pass | Look for |
|-------------|--|
| Structure | Does the order match the reader's path? Can a scanner get the point from headings + TL;DR alone? |
| Cut | Remove history, hedging, repetition, "simply", "just", "obviously" |
| Clarity | Short sentences, active voice, one idea per paragraph (Style Guide (p. 137)) |
| Accuracy | Every command/version/path checked |
| Links | Relative, working, descriptive text |
| Consistency | Terms match the glossary; formatting matches the rest of the docs |

■ 6. Test

| Doc type | How to test |
|------------------|--|
| Tutorial / setup | Follow it on a clean machine or container, or watch a newcomer follow it |
| How-to / runbook | Execute the steps in staging; drill runbooks |
| Reference | Compare with the spec/code; generate it if possible |
| Requirements | QA can derive test cases; stakeholders confirm in review |
| API examples | Run them (ideally in CI) |

■ 7. Review

Open a PR. Reviewers use this checklist:

MARKDOWN

- [] Audience and goal are clear in the first lines
- [] Correct doc type and location
- [] Facts are accurate (subject-matter expert reviewed)
- [] Steps tested
- [] No duplication of existing docs (links instead)
- [] Follows style guide and page pattern
- [] Links work; images have alt text
- [] Front matter: owner, status, last_reviewed
- [] Linked from the folder index
- [] No secrets / internal-only info in public docs

Get reviewers in two roles if you can: a **subject expert** (is it correct?) and a **target reader** (is it usable?).

■ 8. Publish

- Merge, add the doc to the folder index, and announce it if relevant.
- Set `status: active` and `last_reviewed`.

■ 9. Maintain

- Update it in the same PR as related code changes.
- Re-review on the schedule ([Maintenance](#) (p. 152)).
- When readers ask a question the doc should answer, **fix the doc** as well as answering.

RELATED: [Style Guide](#) (p. 137) · [Page Patterns](#) (p. 141) · [Maintenance](#) (p. 152)

CHAPTER 5.2

Style Guide

What: Writing rules for clear, consistent, scannable documentation: voice, sentences, headings, lists, tables, code, links and terminology. **Read when:** writing or reviewing any doc. Adopt it (or adapt it) as your project's style guide.

TL;DR

- **Plain, direct, active.** "Run the migration", not "The migration should be run".
- **Short:** sentences under ~25 words, paragraphs of 3–4 sentences.
- **Descriptive headings** in sentence case, which act as a table of contents.
- **Lists for sequences and sets, tables for comparisons, code blocks for anything copyable.**
- **One term per concept.** Define it in the glossary and use it everywhere.

Voice and tone

| ✓ Do | ✗ Don't |
|--|---|
| Address the reader as "you" | "The user should..." / "One must..." |
| Use imperative for instructions: "Click Save ." | "You might want to click save." |
| Use active voice: "The service sends an email." | "An email is sent by the service." |
| Be confident and specific: "Requests time out after 30 s." | "Requests may possibly time out at some point." |
| Use present tense: "The API returns..." | "The API will return..." |
| Be neutral and inclusive | Idioms, jokes, culture-specific references |

Avoid words that make readers feel bad: *simply, just, easy, obviously, of course*. If it were obvious, they wouldn't be reading the docs.

Sentences and paragraphs

- One idea per sentence, one topic per paragraph.
- Put the **condition first**: "If the build fails, run `make clean`." This lets readers skip what doesn't apply.
- Put the **most important word early** in headings and list items.
- Expand acronyms on first use: "Architecture Decision Record (ADR)".
- Use numbers, not words, for quantities: "3 retries", "30 seconds".

Headings

| Rule | ✓ | ✗ |
|-------------------------|------------------------|-------------|
| One H1 per file = title | # Deploy to production | Several H1s |

| Rule | ✓ | ✗ |
|--------------------------------|---------------------------|--|
| Don't skip levels | H2 → H3 | H2 → H4 |
| Sentence case | ## Configure the database | ## Configure The Database |
| Descriptive, task-based | ## Rotate the API key | ## Procedure, ## Misc |
| Keep them unique within a page | | Two ## Example sections (breaks anchors) |
| No punctuation at the end | ## Prerequisites | ## Prerequisites: |

Test: read only the headings. You should be able to tell what the page covers and in what order.

■ Lists

- **Numbered** for sequential steps, **bullets** for unordered items.
- **One action per step.** Put sub-actions in nested lists.
- Use parallel grammar (all verbs, or all nouns).
- Lists of more than about 7 items become hard to scan, so group them or use a table.
- Punctuation: no period for fragments, periods for full sentences. Be consistent within a list.

■ Steps (procedures)

```
1. Open Settings → Billing.
2. Select Issue refund.
3. Enter the amount and select Confirm.
```

MARKDOWN

The refund appears in **History** with the status *Pending*.

- Put the UI location before the action.
- Use **bold** for UI elements and `code` for things to type.
- Show the **expected result** after important steps.
- Mark dangerous steps:  This deletes all data in the database.

■ Tables

Use tables for **comparisons, options, parameters and mappings**. Don't use them for narrative.

- Keep cells short (a few words or one sentence).
- The first column holds the thing being compared.
- Include units in headers: Timeout (s).
- Avoid empty cells. Use – for "not applicable".

■ Code and commands

```
bash
php artisan migrate --force
```

MARKDOWN

| Rule | Why |
|---|--|
| Always set the language (<code>bash</code> , <code>json</code> , <code>yaml</code> , <code>php</code> , <code>http</code> , <code>sql</code> , <code>text</code>) | Syntax highlighting; tells AI what it is |
| No <code>\$</code> prompt in copyable commands | Clean copy-paste |
| Show output in a separate <code>text</code> block | Readers know what to expect |
| Use realistic, working values | Fake-looking examples confuse |
| Mark placeholders clearly: <code><your-api-key></code> or <code>YOUR_API_KEY</code> | Readers know what to replace |
| Keep examples minimal but complete | Should run as-is |
| Never include real secrets | Use obviously fake values |

Inline code for: file names (`config.yaml`), paths, commands, function/class names, field names, values, env vars.

■ Links

- Use descriptive link text: `see the [refund rules](...)`, not `click [here](...)`.
- Use relative paths for internal docs ([Markdown Essentials: Linking rules \(p. 21\)](#)).
- Link the **first** mention of a concept on a page, not every mention.

■ Terminology

- Keep a **glossary** ([example \(p. 23\)](#)) and use its terms exactly.
- Don't use synonyms for variety: "order" in one paragraph and "purchase" in the next makes readers wonder whether they're different things.
- Match **UI labels** and **code names** exactly, with the same capitalization.
- Prefer common words: "use", not "utilize"; "start", not "initiate".

■ Emphasis and callouts

- **Bold** for UI elements and key terms on first use. Don't bold whole sentences.
- *Italic* sparingly, for emphasis or new terms.
- Use callouts (`> [!NOTE]`, `> [!WARNING]`) only for information that genuinely needs attention. If everything is a warning, nothing is.

■ Numbers, dates, units

- ISO dates: `2026-10-09` .
- Units with a space: `30 s`, `512 MB` . Be consistent about binary and decimal units.
- Time zones are explicit: `14:00 UTC` .

■ Accessibility

- Give images alt text that describes the content, not "image".
- Don't convey meaning by color alone ("the red items").
- Write meaningful link text.

- Use real heading structure, not bold text pretending to be a heading.

RELATED: [Page Patterns](#) (p. 141) · [Markdown Essentials](#) (p. 19) · [Writing for AI Agents](#) (p. 149)

CHAPTER 5.3

Page Patterns

What: Ready skeletons for each kind of page: tutorial, how-to guide, reference, explanation, overview, decision record, troubleshooting and index. **Read when:** you know which doc type you're writing and need its shape. Full copyable files are in [Templates overview](#) (p. 156).

TL;DR

- Every page starts with the **universal header**: title, one-line purpose, audience, and optionally a TL;DR.
- Each page kind has **one job**. Don't mix a tutorial with a reference.
- Use the patterns below as outlines and fill them with real content.

■ Universal page header

```
---
owner: team-billing
status: active
last_reviewed: 2026-10-01
---

# <Title that names the topic or task>

> <One sentence: what this page helps the reader do or understand.>
> **Audience:** <who> · **Prerequisites:** <links>

## TL;DR                ← optional for short pages; recommended for long ones
- ...
```

MARKDOWN

■ Tutorial

Job: teach a beginner by guiding them to a working result. Learning-oriented.

```
# Build your first <thing>

In this tutorial you'll build <result>. It takes about <N> minutes.

## What you'll build
<screenshot or description of the end result>

## Before you start
- <tool + version>
- <account>

## Step 1: <action>
<instruction>
<code>
<what you should see>

## Step 2: <action>
...
```

MARKDOWN

What you've learned

- <concept 1>
- <concept 2>

Next steps

- <link to how-to guides>
- <link to explanation>

✔ One path, no options. Every step produces visible progress. Test it end to end. ✖ No "alternatively...", no deep explanations (link out instead), no assumed knowledge.

■ How-to guide

Job: help a competent reader complete one specific task. Goal-oriented.

How to <do task>

(or a verb title: "Rotate API keys")

<One sentence on when you'd need this.>

Prerequisites

- <permissions, versions, prior setup>

Steps

1. <action>

2. <action>

```
`` bash
```

```
<command>
```

```
```
```

3. <action>

### ## Verify

<how to check it worked>

### ## Troubleshooting

| Problem | Solution |
|---------|----------|
|---------|----------|

|     |     |
|-----|-----|
| --- | --- |
|-----|-----|

### ## Related

- <links>

MARKDOWN

✔ Starts from the goal. Assumes basics. Can mention options briefly ("To do X instead, see..."). ✖ No teaching from scratch, no history or theory.

## ■ Reference page

**Job:** describe facts completely and consistently, for looking things up. Information-oriented.

### # <Thing> reference

<One sentence: what this is.>

### ## <Item 1>

<Short description.>

| Property | Type | Required | Default | Description |
|----------|------|----------|---------|-------------|
|----------|------|----------|---------|-------------|

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| --- | --- | --- | --- | --- |
|-----|-----|-----|-----|-----|

### \*\*Example\*\*

```
```<lang>
```

```
...
```

MARKDOWN

```

...
## <Item 2>
... (identical structure for every item)

```

✔ Same structure for every entry. Complete. Alphabetical or mirrors the product's structure. Generate it if possible. ✖ No instructions or opinions. Link to how-tos and explanations instead.

■ Explanation (concept) page

Job: build understanding: the why, how it works, and trade-offs. Understanding-oriented.

```

# <Concept / "How X works" / "About X">

<Summary paragraph: what it is and why it matters.>

## How it works
<narrative + diagram>

## Why it's designed this way
<context, constraints, decisions (link ADRs)>

## Trade-offs and alternatives

## Related

```

MARKDOWN

✔ Discursive prose is fine. Diagrams help. Connect to other concepts. ✖ No step-by-step instructions (link to a how-to).

■ Overview page (system, domain, module)

Job: orient the reader. What this is, how it fits in, where to go next.

```

# <System / domain / module name>

<What it does in 2-3 sentences.>

- **Owner:** <team>
- **Code:** <paths>
- **Depends on / used by:** <links>

## Context
<diagram>

## Key concepts
<3-7 bullets, linked to glossary>

## Where to go next
| I want to... | Read |
|---|---|

```

MARKDOWN

■ Decision record (ADR)

```

# ADR-<NNNN>: <Decision in active voice>

- **Status:** proposed | accepted | superseded by ADR-XXXX
- **Date:** YYYY-MM-DD
- **Deciders:** <names>

```

MARKDOWN

```
## Context
## Decision
## Alternatives considered
## Consequences
```

Details: [Architecture Docs: Architecture Decision Records \(ADRs\) \(p. 45\)](#) · Template: [Architecture Decision Record \(Nygard style + alternatives\) template \(p. 166\)](#)

■ Troubleshooting page

Job: map symptoms to fixes, fast.

```
# Troubleshooting <area>

## <Exact error message or symptom>
**Cause:** <one sentence>
**Fix:**
1. <step>

## <Next symptom>
...
```

MARKDOWN

✔ Use the **exact error text** as headings, which makes them searchable for humans and AI. ✔ Put the most common problems first.

■ Index page (folder README)

Job: a menu for the folder.

```
# <Folder topic>

> <What this folder contains.> **Owner:** <team>

| Page | Description |
|---|---|
| [page.md](page.md) | <one line> |
```

MARKDOWN

See [Navigation & Metadata: Folder index \(README.md\) pattern \(p. 113\)](#).

■ Matching doc types to patterns

Doc type	Pattern(s)
README (root)	Overview + quick start (mini tutorial)
Getting started	Tutorial
Runbook	How-to (with mitigation first)
API endpoint docs	Reference
API quick start	Tutorial
Architecture overview	Overview + explanation
ADR	Decision record
PRD / BRD	Own templates (Templates overview (p. 156))

Doc type	Pattern(s)
Business rules	Reference (table with IDs)
FAQ	Troubleshooting-like Q → A

RELATED: [diataxis model](#) (p. 76) · [Templates overview](#) (p. 156) · [Style Guide](#) (p. 137)

CHAPTER 5.4

Diagrams & Visuals

What: When to use diagrams, screenshots and tables, which diagram type fits which question, and how to keep visuals maintainable. **Read when:** a doc describes structure, flow, relationships or a UI.

TL;DR

- Use a diagram when the reader needs to see **structure or flow**. Use text when they need **steps or facts**.
- Prefer **diagrams as code** (Mermaid first). They're diffable, reviewable and readable by AI agents.
- **One diagram, one message**. Give it a title and a legend, and keep it under about 15 boxes.
- Screenshots go stale fast. Crop tightly, annotate, and use them only where the UI is the point.
- Always pair a visual with **text that states its key point**, for accessibility and for AI agents.

■ Which visual for which question

Reader's question	Visual	Mermaid type
What talks to what?	Context / container diagram	<code>flowchart</code> or <code>C4Context</code>
In what order do things happen across components?	Sequence diagram	<code>sequenceDiagram</code>
What are the steps and decisions?	Flowchart	<code>flowchart</code>
What states can this be in?	State diagram	<code>stateDiagram-v2</code>
How is data related?	ER diagram	<code>erDiagram</code>
What's the class/module structure?	Class diagram	<code>classDiagram</code>
What's the timeline / plan?	Gantt / timeline	<code>gantt</code> , <code>timeline</code>
How do options compare?	Table , not a diagram	—
What does the screen look like?	Screenshot (annotated)	—

■ Mermaid examples

Sequence diagram

```
sequenceDiagram
    actor Agent as Support agent
    participant UI as Admin UI
    participant API as Billing API
    participant S as Stripe
    Agent->>UI: Request refund €600
    UI->>API: POST /v2/refunds
    API->>API: Check BR-PAY-04 (> €500?)
    API-->>UI: 202 pending_approval
    Note over API: Manager approves later
```

MERMAID

```
API->>S: Create refund
S-->>API: refund.succeeded
```

State diagram

```
stateDiagram-v2
    [*] --> Requested
    Requested --> PendingApproval: amount > €500
    Requested --> Processing: amount ≤ €500
    PendingApproval --> Processing: approved
    PendingApproval --> Rejected: rejected
    Processing --> Succeeded
    Processing --> Failed
    Succeeded --> [*]
```

MERMAID

Diagram tools

Tool	Format	Best for
Mermaid	Text in Markdown	Default choice; renders on GitHub/GitLab/most doc sites
PlantUML	Text	Complex UML; needs a renderer
Structurizr DSL	Text (C4 model)	Many C4 views from one model
D2	Text	Nicer layouts, modern syntax
draw.io / diagrams.net	<code>.drawio.svg</code> (editable SVG)	Freeform diagrams that stay editable in the repo
Excalidraw	<code>.excalidraw</code> / <code>.svg</code>	Sketchy whiteboard style
Figma / Miro	External	Design and workshops. Link to them, export a PNG for docs.

Rule: if the diagram **can't be edited from the repo**, it'll go stale. Store the source next to the export.

Diagram rules

- ✓ Give it a **title** or a caption sentence above it: "Figure: refund flow with approval".
- ✓ Show **one level of abstraction** per diagram ([C4](#) (p. 45)).
- ✓ **Label arrows** with what flows (`JSON/HTTPS`, `events`, `charges`).
- ✓ Use the same **names as in code and the glossary**.
- ✓ Keep it under about **15 elements**. Split bigger ones.
- ✗ Don't rely on color alone to carry meaning.
- ✗ Don't put critical information *only* in the diagram. State it in text too.

Screenshots

✓ Do

✗ Don't

Crop to the relevant area

Full-screen captures

Annotate (box/arrow) the element to click

Leave the reader searching

Use test data

Show real customer data

Name files meaningfully: `refund-dialog.png`

Screenshot 2026-10-09 at 14.02.png

✓ Do	✗ Don't
Store next to the doc: <code>images/</code>	Hotlink from external hosts
Write alt text: <code>![Refund dialog with Amount field highlighted](images/refund-dialog.png)</code>	<code>![image](...)</code>

Use screenshots only where the visual adds value. For a "click **Settings** → **Billing**" step, text is enough.

■ Tables as visuals

Tables are often the best "diagram" for:

- comparisons (options × criteria),
- mappings (alert → runbook, rule → code),
- parameters (name, type, default, description).

■ Asset organization

```
architecture/
├── README.md
├── images/
│   ├── c4-context.drawio.svg    editable source + render
│   └── checkout-sequence.png
```

Or a shared `_assets/` folder at the docs root for images used across sections.

RELATED: [Architecture Docs](#) (p. 44) · [Markdown Essentials: Diagrams as code: Mermaid](#) (p. 21) · [Style Guide](#) (p. 137)

CHAPTER 5.5

Writing for AI Agents

What: How to write and organize docs so LLMs and AI coding agents read them correctly and act on them reliably, without making them worse for humans. **Read when:** your team uses AI assistants, or your docs feed a chatbot, search or RAG system.

TL;DR

- **What's good for humans is good for agents:** clear structure, explicit statements, consistent terms, working examples.
- Agents have **no tribal knowledge**. Anything implied must be **written down**.
- Agents navigate by **entry files, links and search**. Predictable paths and index files matter more than ever.
- Make every page **self-contained enough** to be understood when retrieved alone (RAG chunks).
- Use **stable IDs** (rules, requirements, ADRs) so agents can cite and cross-reference precisely.

■ How agents consume docs

Mechanism	Implication for your docs
Load an instruction file at start (<code>AGENTS.md</code> , <code>CLAUDE.md</code>)	Keep it short; make it the map to everything else
Follow links from file to file	Relative links, descriptive link text, indexes in each folder
Search (grep, semantic search)	Use the words people would search for; consistent terms; IDs
Chunking (RAG): split pages by heading	Each section understandable alone; headings that name the topic
Limited context window	Concise pages; summaries first; no giant files mixing topics
Literal interpretation	Explicit rules ("always", "never"); no sarcasm or ambiguity

■ Rules

1. Be explicit, not implicit

✗ Implicit	✓ Explicit
"Use the usual approach for money."	"Store money as integer minor units (cents) in <code>*_cents</code> columns. Never use floats."
"Don't touch the legacy stuff."	"Do not modify files in <code>src/Legacy/</code> . They are frozen until migration RFC-0031 completes."
"Run the tests."	"Run <code>composer test</code> . All tests must pass before committing."

2. Make every section stand alone

A retrieved chunk may be seen **without** the rest of the page.

- ❌ "As mentioned above, it requires approval."
- ✅ "Refunds over €500 require finance manager approval (BR-PAY-04)."

Repeat the **subject** in headings: `## Refund approval rules`, not `## Rules`.

3. Structure over prose

- Use headings, lists and tables. Agents extract structured content more reliably than long paragraphs.
- Put **key facts in predictable places**: front matter, TL;DR, tables.
- One topic per file. Mixed files produce mixed answers.

4. Consistent terminology and stable IDs

- One term per concept, defined in the [Glossary](#) (p. 23). Agents treat synonyms as different things.
- IDs for rules, requirements and decisions: `BR-PAY-04`, `FR-PAY-012`, `ADR-0012`.
- Reference IDs in code comments and tests, so agents can trace rule → code → test with a single search.

5. Show correct and incorrect examples

Agents learn patterns from examples. Pair each convention with ✅ and ❌ code from *this* codebase.

6. Mark status and authority

- Front matter `status: deprecated` plus a visible banner, so agents don't follow outdated docs.
- Mark the canonical source: "Canonical: this page. Other mentions link here."
- Put dead docs in an `archive/` folder and tell agents to ignore it in `AGENTS.md`.

7. Give a map

In `AGENTS.md` or the docs index, include a short **docs map**:

```
## Where to find things
| Need | Location |
|---|---|
| Business rules | docs/requirements/rules/ (IDs BR-*) |
| Architecture decisions | docs/architecture/decisions/ (ADR-*) |
| API contract | api/openapi.yaml (source of truth) |
| Local setup | docs/getting-started/README.md |
| Do NOT use | docs/archive/ (historical only) |
```

MARKDOWN

8. State boundaries and procedures as commands

```
## Rules for AI agents
- Never edit applied migrations in `database/migrations/`; create a new one.
- Never commit `.env` or files in `storage/`.
- Before claiming a task is done, run `composer test && composer lint`.
- When changing an API endpoint, update `api/openapi.yaml` in the same change.
```

MARKDOWN

■ Agents as doc writers

Agents can draft docs well, provided you give them structure:

Give the agent	So that
This guide's Decision Guide (p. 121) and Templates overview (p. 156)	Output fits your structure
The style guide	Consistent tone and formatting
The rule: "Write working notes to <code>.agents/</code> , final docs to <code>docs/</code> only after review"	Docs root stays curated
A requirement to mark output: <code>> Draft generated by AI on <date>; needs review</code>	Reviewers know what to check
An instruction to cite sources (files, ADRs) for claims	Easier fact-checking

Review checklist for agent-written docs:

- ☐ Facts verified against code and specs, with no invented flags, endpoints or config keys
- ☐ No placeholder text left from templates
- ☐ Links resolve
- ☐ Doesn't duplicate an existing doc
- ☐ Correct location per the organizing rules

■ **Formats beyond Markdown for agents**

Format	Use
<code>AGENTS.md</code> / <code>CLAUDE.md</code>	Repo instructions (AI Agent Docs (p. 56))
<code>llms.txt</code>	Curated index of a public docs site
OpenAPI / JSON Schema	Machine-precise contracts; agents read them well
Front matter	Machine-readable metadata (status, owner, IDs)
Markdown tables	Rules, mappings, parameters: easy to parse

RELATED: [AI Agent Docs](#) (p. 56) · [Style Guide](#) (p. 137) · [Navigation & Metadata](#) (p. 113)

CHAPTER 5.6

Maintenance

What: How to keep docs correct over time: ownership, update triggers, review cycles, automation (linting, link checking, CI), deprecation and archiving. **Read when:** setting up a docs process, or when docs keep going stale.

TL;DR

- **Ownership:** every doc or folder has an owner team. Enforce it with `CODEOWNERS`.
- **Docs change with code:** same PR, with a checkbox in the PR template.
- **Review cycle:** a `last_reviewed` date, and a reminder for anything older than 6–12 months.
- **Automate** what you can: markdownlint, a link checker, spell/style checks, front-matter validation, secret scanning.
- **Deprecate visibly, archive or delete decisively.**

Ownership

```
# .github/CODEOWNERS
/docs/architecture/      @org/tech-leads
/docs/requirements/billing/ @org/team-billing @org/product-billing
/docs/operations/        @org/platform
/docs/public/             @org/docs-team
/AGENTS.md                @org/tech-leads
```

TEXT

The owner doesn't write everything. They make sure it **stays correct** and they review changes.

Update triggers

When this happens	Update
Public API changes	API spec + guide + CHANGELOG
New env var / config	<code>.env.example</code> + configuration doc
Architectural decision	New ADR (+ overview if the big picture changed)
Business rule changes	Rules catalog (new ID or versioned rule) + affected PRDs/tests
Incident	Postmortem + runbook improvements
Onboarding friction	Getting-started fix by the new person
Release	CHANGELOG, release notes
Repeated question in chat	FAQ or the relevant doc

Add this to the PR template:

Docs

MARKDOWN

- [] Docs updated (or not needed because: ___)

■ Review cycles

Doc kind	Review
Getting started, runbooks	Every 3–6 months, or on every failure
Architecture overview, conventions	Every 6 months
Reference (generated)	Automatic
ADRs, postmortems	Never edited (immutable); status updates only
Requirements (shipped)	Archive after ship; rules stay living

Automate a staleness report: a script that lists files whose `last_reviewed` is older than N months and opens an issue for each owner.

■ Automation

Check	Tools (examples)	Catches
Markdown lint	<code>markdownlint-cli2</code> , <code>remark-lint</code>	Heading levels, list style, trailing spaces
Prose / style lint	Vale (custom style rules), <code>alex</code>	Banned words ("simply"), terminology, passive voice
Spelling	<code>cspell</code> , <code>codespell</code>	Typos, with project dictionary
Link check	<code>lychee</code> , <code>markdown-link-check</code>	Broken internal and external links, bad anchors
Front matter validation	Custom script / JSON Schema	Missing owner, invalid status
Orphan check	Custom script	Pages not linked from any index
Secret scanning	<code>gitleaks</code> , <code>trufflehog</code> , platform scanning	Leaked credentials in docs
API spec lint	<code>spectral</code> , <code>redocly lint</code>	Missing descriptions/examples
Code samples	Extract-and-run scripts, doctest-style tools	Broken examples

Example CI job (GitHub Actions)

```

name: docs
on:
  pull_request:
    paths: ['**/*.md', 'api/**']
jobs:
  docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Markdown lint
        uses: DavidAnson/markdownlint-cli2-action@v16
        with:
          globs: '**/*.md'
      - name: Link check
        uses: lycheeverse/lychee-action@v2
        with:
          args: --offline --include-fragments '**/*.md'

```

YAML

Pin action versions to what's current when you set this up.

Example `.markdownlint.jsonc`

```
{
  "default": true,
  "MD013": false,           // line length: off (no hard wrap)
  "MD033": { "allowed_elements": ["details", "summary", "br"] },
  "MD041": true             // first line must be a top-level heading
}
```

JSONC

Note: MD041 conflicts with front matter in some setups. Configure `front_matter_title` or disable it if needed.

■ Deprecation and archiving

Step	Action
1. Deprecate	<code>status: deprecated</code> , banner with replacement link and removal date
2. Redirect	Link old → new; on doc sites, add a redirect
3. Archive	Move to <code>archive/</code> (if it has historical value) with an "Archived" banner, excluded from site search
4. Delete	Remove; Git keeps history

Banner example:

```
> [!WARNING]
> Deprecated (2026-10). Use [Deployment v2](deployment.md) instead. This page will be deleted after
2027-01.
```

MARKDOWN

■ Metrics (optional)

- Page views and search terms with no results (docs site analytics) show you what's missing.
- Time-to-first-commit for new hires shows onboarding quality.
- Percentage of docs with `last_reviewed` under 12 months shows freshness.
- Repeated questions in support or chat channels show doc gaps.

RELATED: [Writing Process](#) (p. 134) · [Navigation & Metadata](#) (p. 113) · [Organization Smells & Fixes](#) (p. 117)

PART VI

Templates

Copy-paste starter files for common documents

templates/

Overview	156
Root README	159
Module / package / service README	161
CONTRIBUTING	162
CHANGELOG	164
Docs root index	165
Architecture Decision Record (Nygard style + alternatives)	166
RFC / design doc	167
Business Requirements Document	169
Product Requirements Document	171
User story with acceptance criteria	173
Use case	175
Business rules catalog for one domain	176
Hand-written API endpoint reference	177
Runbook	179
Blameless incident postmortem	181
Tutorial (learning-oriented)	183
How-to guide (task-oriented)	185
AGENTS	186

PART VI

Overview

What: Copy-paste starter files for the most common project documents. **Read when:** you're creating a new document. Copy the template, rename it, and replace every `<placeholder>`.

How to use

1. Find the doc type in the table below. Not sure which type you need? See the [doc-types catalog](#) (p. 26).
2. Copy the file to its location in your project ([Folder Structures: Ready-Made Trees](#) (p. 104)).
3. Delete the `<!-- TEMPLATE ... -->` comment at the top.
4. Replace every `<placeholder>`, and **delete sections that don't apply**. Don't leave them empty.
5. Add the doc to its folder's `README.md` index.

Each template is a raw Markdown file. Open it in an editor (not the rendered view) to copy it exactly.

DEVELOPER

README

module README

CONTRIBUTING

CHANGELOG

docs index

ARCHITECTURE

ADR

RFC / design doc

USER / GUIDES

tutorial

how-to

AI AGENT

AGENTS.md

5 BUSINESS / REQUIREMENTS

BRD

PRD

user story

use case

business rules

2 OPERATIONS

runbook

postmortem

2 API

API endpoint

1

5

2

1

Figure 6.1 The 18 templates grouped by documentation family.

Index

Developer

Template	Copy to	Guide
Root README template (p. 159)	<code>README.md</code> (repo root)	Developer Docs: Root README.md (p. 29)
Module / package / service README template (p. 161)	<code><module>/README.md</code>	Model: Co-located with Code (p. 82)

Template	Copy to	Guide
CONTRIBUTING template (p. 162)	<code>CONTRIBUTING.md</code>	Developer Docs: CONTRIBUTING.md (p. 30)
CHANGELOG template (p. 164)	<code>CHANGELOG.md</code>	Developer Docs: CHANGELOG.md (p. 32)
Docs root index template (p. 165)	<code><docs-root>/README.md</code>	Navigation & Metadata: The docs root index (p. 114)

Architecture

Template	Copy to	Guide
Architecture Decision Record (Nygard style + alternatives) template (p. 166)	<code>architecture/decisions/NNNN-title.md</code>	Architecture Docs: Architecture Decision Records (ADRs) (p. 45)
RFC / design doc template (p. 167)	<code>rfcs/NNNN-title.md</code>	Architecture Docs: RFCs and design docs (p. 46)

Business / requirements

Template	Copy to	Guide
Business Requirements Document template (p. 169)	<code>requirements/brd-<initiative>.md</code>	Business & Requirements Docs: BRD (Business Requirements Document) (p. 40)
Product Requirements Document template (p. 171)	<code>requirements/<domain>/<feature>.md</code>	Business & Requirements Docs: PRD (Product Requirements Document) (p. 40)
User story with acceptance criteria template (p. 173)	issue tracker or <code>requirements/<feature>/stories/</code>	Business & Requirements Docs: User stories (p. 41)
Use case template (p. 175)	<code>requirements/use-cases/UC-NN-title.md</code>	Business & Requirements Docs: Use cases (p. 42)
Business rules catalog for one domain template (p. 176)	<code>requirements/rules/<domain>.md</code>	Business & Requirements Docs: Business rules catalog (p. 42)

API

Template	Copy to	Guide
Hand-written API endpoint reference template (p. 177)	<code>api/reference/<resource>.md</code> (if not generated)	API Docs: What every endpoint page must include (p. 35)

Operations

Template	Copy to	Guide
Runbook template (p. 179)	<code>operations/runbooks/<alert-name>.md</code>	Operations Docs: Runbooks (p. 48)
Blameless incident postmortem template (p. 181)	<code>operations/postmortems/YYYY-MM-DD-title.md</code>	Operations Docs: Incident postmortems (p. 49)

User / guides

Template	Copy to	Guide
Tutorial (learning-oriented) template (p. 183)	<code>tutorials/<title>.md</code>	Page Patterns: Tutorial (p. 141)
How-to guide (task-oriented) template (p. 185)	<code>how-to/<task>.md</code>	Page Patterns: How-to guide (p. 142)

AI agents

Template	Copy to	Guide
AGENTS template (p. 186)	<code>AGENTS.md</code> (repo root)	AI Agent Docs (p. 56)

TEMPLATE 1

Root README

readme-template.md

64 lines

Copy to README.md at the repository root. Keep it under ~3 screens; link out for details

MARKDOWN

```

# <Project name>

<One sentence: what this project is and who it's for.>

<!-- Optional badges: build, coverage, version, license -->

## Overview

<2-5 sentences or bullets: the problem, the solution, key features.>

- <Key feature 1>
- <Key feature 2>
- <Key feature 3>

## Quick start

**Requirements:** <runtime + version>, <tool + version>, <service/account>

```bash
git clone <repo-url>
cd <project>
<install command>
cp .env.example .env
<run command>
```

Open <http://localhost:PORT>. You should see <expected result>.

Full setup and troubleshooting: [Getting started](<docs-root>/getting-started/README.md).

## Usage

<!-- For libraries/CLIs: 1-3 minimal examples. For apps: delete or replace with key workflows. -->

```<language>
<minimal example>
```

## Project structure

```text
<top-level-folder>/ <one-line purpose>
<top-level-folder>/ <one-line purpose>
<docs-root>/ documentation
```

## Documentation

Topic	Link
Docs index	[<docs-root>/README.md](<docs-root>/README.md)
Architecture	[<docs-root>/architecture/](<docs-root>/architecture/README.md)
API	[<docs-root>/api/](<docs-root>/api/README.md)
Requirements	[<docs-root>/requirements/](<docs-root>/requirements/README.md)
Operations	[<docs-root>/operations/](<docs-root>/operations/README.md)
Changelog	[CHANGELOG.md](CHANGELOG.md)

```

Contributing

See CONTRIBUTING.md.

License

<License name>. See LICENSE.

TEMPLATE 2

Module / package / service README

module-readme-template.md

40 lines

Copy to <module>/README.md

MARKDOWN

```

# <module-name>

<One sentence: what this module does.>

- **Owner:** <team> (<chat channel>)
- **Code:** `<path>`
- **Depends on:** <modules, services, databases, external APIs>
- **Used by:** <consumers>
- **Status:** <active | experimental | deprecated>

## Responsibilities

- <What it is responsible for>
- <What it explicitly does NOT do>

## Run locally

```bash
<command>
```

## Configuration

Variable	Required	Default	Description
`<VAR>`	✓	-	<description>

## Public interface

<Main classes / endpoints / events / functions others use. Link to API reference if generated.>

## Key concepts

- **<Term>:** <definition or link to glossary>

## Related docs

- [<Business rules>](<path>)
- [<ADR-NNNN: decision>](<path>)
- [<Runbooks>](<path>)

```

TEMPLATE 3

CONTRIBUTING

contributing-template.md

53 lines

md. Copy to the repository root (or .github/)

MARKDOWN

Contributing to <project>

Thanks for contributing! This guide explains how to set up, make changes, and get them merged.

Development setup

Follow [Getting started](<docs-root>/getting-started/README.md). In short:

```
```bash
<install command>
<run command>
<test command>
```
```

Workflow

1. Create a branch from ``<main>``: ``<type>/<short-description>`` (e.g. ``feat/partial-refunds``, ``fix/invoice-rounding``).
2. Make small, focused commits. Format: `<Conventional Commits / team format>`, e.g. ``feat(billing): add partial refunds``.
3. Open a pull request and fill in the template.

Code style

- Formatting and linting are automated: ``<lint command>``, ``<format command>``.
- Project conventions (patterns, naming, error handling): [conventions](<docs-root>/development/conventions.md).

Tests

- Run all tests: ``<test command>``
- <Coverage or test-type expectations, e.g. "New business rules need unit tests referencing the rule ID">

Pull requests

- Keep PRs under <~400> changed lines where possible.
- CI must pass: <list checks>.
- Required reviewers: <rule, or "see CODEOWNERS">.

Documentation changes

Update docs **in the same PR** when you:

- change a public API → ``<path to API spec/docs>``
- add or change configuration → ``.env.example`` + ``<config doc>``
- make a significant technical decision → new ADR in ``<docs-root>/architecture/decisions/`
- change a business rule → ``<docs-root>/requirements/rules/`
- change user-visible behavior → ``CHANGELOG.md`` under **UnreLeased**

Reporting bugs and requesting features

<Where and how: issue templates, labels, security issues → SECURITY.md.>

Getting help

<Channel, maintainers, office hours.>

TEMPLATE 4

CHANGELOG

changelog-template.md

35 lines

md (Keep a Changelog format). Copy to the repository root and the example entries.

Changelog

MARKDOWN

All notable changes to this project are documented in this file.

The format is based on [Keep a Changelog](https://keepachangelog.com/en/1.1.0/), and this project adheres to [Semantic Versioning](https://semver.org/spec/v2.0.0.html).

[Unreleased]

Added

- <New feature, user-facing wording> (<#issue or PR>)

Changed

Deprecated

Removed

Fixed

Security

[<X.Y.Z>] - <YYYY-MM-DD>

Added

- <Example: Partial refunds via `POST /v2/refunds` (#512)>

Fixed

- <Example: Duplicate password-reset emails (#482)>

[Unreleased]: <compare-url>/v<X.Y.Z>...HEAD

[<X.Y.Z>]: <compare-url>/v<previous>...v<X.Y.Z>

TEMPLATE 5

Docs root index

docs-index-template.md

45 lines

Copy to <docs-root>/README.md

MARKDOWN

```
# <Project> documentation

> Everything about building, running and using <project>. Start with the table below.

## Who these docs are for

I am...	Start here
A new developer	[Getting started](getting-started/README.md)
Working on the architecture	[Architecture](architecture/README.md)
Integrating with the API	[API](api/README.md)
Product / business / QA	[Requirements](requirements/README.md)
On call	[Runbooks](operations/runbooks/README.md)

## Map

Folder	Contains	Owner
[getting-started/] (getting-started/README.md)	Local setup, configuration, troubleshooting	<team>
[architecture/] (architecture/README.md)	Overview, diagrams, decisions (ADRs)	<team>
[api/] (api/README.md)	API reference and guides	<team>
[requirements/] (requirements/README.md)	PRDs, business rules, NFRs	<team>
[operations/] (operations/README.md)	Deployment, runbooks, postmortems	<team>
[glossary.md] (glossary.md)	Shared vocabulary	everyone

## How these docs are organized

- **Top level** = <dimension, e.g. doc type>. **<one line>
- **Second level** = <dimension, e.g. domain> **when a folder has more than <15> files.
- **ADRs** are numbered `NNNN-title.md`; **postmortems** are dated `YYYY-MM-DD-title.md`.
- **Module docs** live next to the code: `<path>/README.md`.
- **Metadata:** every doc has `owner`, `status`, `last_reviewed` in front matter.
- **Source of truth:** Markdown in this repo, except <user stories (tracker), meeting notes (wiki)>.
- **Archived** docs are in [archive/] (archive/README.md) and are not maintained.

## Contributing to docs

- Templates: <link>
- Style guide: <link>
- Docs change in the same PR as the code they describe.

## Most-used pages

- [<page>](<path>)
- [<page>](<path>)
```

TEMPLATE 6

Architecture Decision Record (Nygard style + alternatives)

adr-template.md

38 lines

Copy to architecture/decisions/NNNN-short-title.md. Keep it to 1–2 pages. Once accepted, never rewrite. Supersede with a new ADR instead

MARKDOWN

```
---
id: ADR-<NNNN>
status: proposed           # proposed | accepted | rejected | deprecated | superseded
date: <YYYY-MM-DD>
deciders: [<name>, <name>]
supersedes:                # ADR-XXXX (optional)
superseded_by:             # ADR-XXXX (fill when superseded)
---

# ADR-<NNNN>: <Decision stated in active voice, e.g. "Use PostgreSQL for the order store">

## Context

<What situation forces a decision? Requirements, constraints, forces, facts. Link to RFCs, requirements, incidents. Neutral tone.>

## Decision

We will <decision>.

<Key details of the decision: scope, what it applies to, what it doesn't.>

## Alternatives considered

Option	Pros	Cons	Why not chosen
<Option A>			
<Option B>			

## Consequences

- ✓ <Positive consequence>
- ✗ <Negative consequence / trade-off accepted>
- → <Follow-up work required, with ticket links>

## References

- <Links to RFC, benchmarks, docs, discussions>
```

TEMPLATE 7

RFC / design doc

rfc-design-doc-template.md

89 lines

Copy to rfcs/NNNN-short-title.md (or architecture/design/). Write it BEFORE implementing. Typical length 3–15 pages

MARKDOWN

```
---
id: RFC-<NNNN>
title: <Title>
status: draft # draft | review | accepted | rejected | withdrawn | implemented
author: <name>
reviewers: [<name>, <name>]
created: <YYYY-MM-DD>
updated: <YYYY-MM-DD>
decision_date:
related: [<PRD link>, <ADR links>]
---

# RFC-<NNNN>: <Title>

> Status: ● Draft. Comments welcome until <YYYY-MM-DD>.

## Summary

<One paragraph: what you propose and why.>

## Context and problem

<Current state, the problem, evidence (data, incidents, user feedback). Link requirements.>

## Goals

- <Measurable goal>

## Non-goals

- <Explicitly out of scope>

## Proposed design

### Overview

<Diagram + description of the approach.>



```

mermaid
graph LR
 A[<Component>] --> B[<Component>]

```



### Details

<APIs, data model changes, algorithms, interfaces. Use subsections per component.>

### Data model changes

<Tables/entities added or changed, migrations.>

### API changes

<New/changed endpoints, events; breaking changes flagged.>
```

Alternatives considered

| Alternative | Pros | Cons | Why not |
|-------------|------|------|---------|
| --- | --- | --- | --- |

Risks and mitigations

| Risk | Likelihood | Impact | Mitigation |
|------|------------|--------|------------|
| --- | --- | --- | --- |

Security, privacy, compliance

<Data handled, access, threats, regulatory impact.>

Performance and scalability

<Expected load, limits, benchmarks.>

Rollout plan

<Phases, feature flags, migration, backward compatibility, rollback plan.>

Testing and observability

<Test strategy; metrics, logs, alerts, dashboards to add.>

Open questions

- [] <Question> (owner: <name>, due: <date>)

Decision

<Filled in when decided: outcome, date, who decided, resulting ADRs.>

TEMPLATE 8

Business Requirements Document

brd-template.md

86 lines

Copy to requirements/brd-<initiative>.md. Business language only, with no technical solutions

MARKDOWN

```

---
id: BRD-<NN>
title: <Initiative name>
status: draft # draft | review | approved | superseded
owner: <business sponsor / BA>
version: 0.1
created: <YYYY-MM-DD>
approved:
---

# BRD: <Initiative name>

## Document control

Version	Date	Author	Change
0.1	<YYYY-MM-DD>	<name>	Initial draft

## 1. Executive summary

<3-5 sentences: the problem, the proposed initiative, the expected business outcome.>

## 2. Background and problem

<Current situation and pain points, with evidence: numbers, costs, complaints, risks.>

## 3. Business objectives

ID	Objective	Metric	Current	Target	By
OBJ-1	<e.g. Faster refund handling>	<avg handling time>	<3 days>	<4 hours>	<date>

## 4. Stakeholders

Stakeholder	Role / interest	Involvement (RACI)
<Finance>	<approves refunds>	<A>

## 5. Scope

### In scope

- <item>

### Out of scope

- <item, explicitly excluded>

## 6. Business requirements

ID	Requirement	Priority	Source
BR-01	<The business needs to ...>	Must	<stakeholder / policy>
BR-02		Should	

```

7. Business rules

<Link to the rules catalog, or list new rules with IDs.>

8. Constraints and assumptions

- ****Constraints:**** <budget, deadlines, regulation, systems>
- ****Assumptions:**** <what we believe is true; validate later>

9. Dependencies

- <Other initiatives, vendors, teams>

10. Risks

| Risk | Impact | Likelihood | Mitigation | Owner |
|------|--------|------------|------------|-------|
| --- | --- | --- | --- | --- |

11. Success measures

<How and when the objectives will be measured after delivery.>

12. Glossary

<Link to glossary or define initiative-specific terms.>

13. Approval

| Name | Role | Decision | Date |
|------|------|----------|------|
| --- | --- | --- | --- |

TEMPLATE 9

Product Requirements Document

prd-template.md

98 lines

Copy to requirements/<domain>/<feature>.md (or product/features/<feature>.md). Aim for 2–6 pages

MARKDOWN

```

---
id: PRD-<NN>
title: <Feature name>
status: draft # draft | review | approved | in-development | shipped | archived
owner: <product manager>
reviewers: [<eng lead>, <design>, <qa>]
target_release: <version / quarter>
created: <YYYY-MM-DD>
updated: <YYYY-MM-DD>
links:
  brd: <link>
  designs: <figma link>
  epic: <tracker link>
---

# PRD: <Feature name>

> **Status:** ● Draft · **Owner:** <name> · **Target:** <release>

## 1. Problem

<Who has what problem, how we know (data, research, feedback), and why now.>

## 2. Goals and non-goals

**Goals**

- <Outcome, ideally measurable>

**Non-goals**

- <What this feature deliberately does not do>

## 3. Users and personas

Persona	Need	Frequency
<Support agent>	<issue partial refunds quickly>	<daily>

## 4. User journeys

<Narrative of the main flow(s), step by step from the user's perspective. Add a diagram if useful.>

## 5. Requirements

ID	Requirement	Priority	Notes
FR-<DOM>-01	<The system lets a support agent ...>	P0	
FR-<DOM>-02		P1	

Priority: P0 = must for launch, P1 = should, P2 = nice to have.

User stories with acceptance criteria: <link to tracker epic or stories file>.

## 6. Business rules

```

```
Rule	Link
<Refunds > €500 need approval>	[BR-PAY-04](<path to rules catalog>#br-pay-04)
```

<Link to existing rules; define new rules in the rules catalog, not here.>

7. Non-functional requirements

```
Category	Requirement
Performance	<p95 < 300 ms>
Security / privacy	
Accessibility	<WCAG 2.2 AA>
Audit / compliance	
```

8. Design

<Links to designs; key screens; states (empty, loading, error).>

9. Analytics and success metrics

```
Metric	Baseline	Target	How measured
```

10. Release plan

<Phases, feature flag, rollout %, migration, communication (release notes, support training).>

11. Dependencies and risks

```
Item	Type	Owner	Mitigation
```

12. Open questions

- [] <Question> (owner: <name>, due: <date>)

13. Changelog

```
Date	Change	By
```

TEMPLATE 10

User story with acceptance criteria

user-story-template.md

55 lines

Usually goes in the issue tracker; use this file when stories live in the repo (requirements/<feature>/stories/US-NN-title.md)

MARKDOWN

```
---
id: US-<NN>
feature: <PRD-NN link>
status: ready          # draft | ready | in-progress | done
priority: <P0 | P1 | P2>
estimate: <points>
---

# US-<NN>: <Short title>

**As a** <role>
**I want** <capability>
**so that** <benefit>.

## Context

<Optional: why this matters, links to PRD section, designs.>

## Acceptance criteria

```gherkin
Scenario: <happy path>
 Given <precondition>
 When <action>
 Then <expected outcome>

Scenario: <alternative / edge case>
 Given <precondition>
 When <action>
 Then <expected outcome>

Scenario: <error case>
 Given <precondition>
 When <action>
 Then <error behavior>
```

## Business rules

- [<BR-ID>](<path>): <one-line summary>

## Out of scope

- <What this story does not cover>

## Notes

- <Design link, technical notes, dependencies>

## Definition of Done

- [ ] Acceptance criteria pass (automated tests reference this story ID)
```

- [] Docs / CHANGELOG updated
- [] Reviewed and merged

TEMPLATE 11

Use case

use-case-template.md

46 lines

Copy to requirements/use-cases/UC-NN-short-title.md. Use for flows with several actors or many alternative paths

MARKDOWN

```
---
id: UC-<NN>
title: <Goal of the actor, e.g. "Issue refund">
status: draft
owner: <BA / PM>
---

# UC-<NN>: <Title>

Field	Value
Primary actor	<role>
Secondary actors	<other roles / systems>
Goal	<what the actor wants to achieve>
Trigger	<event that starts the use case>
Preconditions	<what must be true before>
Postconditions (success)	<state after success>
Postconditions (failure)	<state after failure>
Business rules	<BR-IDs>
Related	<PRD, stories, screens>

## Main success scenario

1. <Actor> <does something>.
2. System <responds>.
3. <Actor> <does something>.
4. System <responds>. Use case ends.

## Alternative flows

3a. <Condition, e.g. amount > €500>

1. System <does something different>.
2. Continue at step <N> / use case ends.

## Exceptions

E1. <Error condition, e.g. payment provider unavailable>

1. System <shows/does>.
2. <Recovery>.

## Notes

- <Frequency, performance expectations, open questions>
```

TEMPLATE 12

Business rules catalog for one domain

business-rules-template.md

50 lines

Copy to requirements/rules/<domain>.md. IDs are permanent: never reuse or renumber; deprecate instead

MARKDOWN

```

---
domain: <domain>
owner: <business owner team>
status: active
last_reviewed: <YYYY-MM-DD>
---

# Business rules: <Domain>

> Canonical source for <domain> business rules. PRDs, stories, code and tests reference these IDs.
> **ID format:** `BR-<DOMAIN CODE>-<NN>`

## Summary

ID	Rule	Status
[BR-<DOM>-01](#br-dom-01)	<short rule>	active
[BR-<DOM>-02](#br-dom-02)	<short rule>	deprecated

---

## BR-<DOM>-01

**Rule:** <One precise, testable statement. e.g. "Refunds above €500.00 require approval by a finance manager before processing.">

Field	Value
Source	<policy, regulation, contract, stakeholder>
Rationale	<why the rule exists>
Since	<YYYY-MM-DD>
Applies to	<channels, products, regions>
Exceptions	<none / list>
Implemented in	`<path or module>`
Tests	`<test file or test IDs>`

**Examples**

Input	Expected
<Refund €500.00>	<Processed immediately>
<Refund €500.01>	<Pending approval>

---

## BR-<DOM>-02

> ● **Deprecated** since <date>. Replaced by [BR-<DOM>-NN](#br-dom-nn).

**Rule:** <...>

```

TEMPLATE 13

Hand-written API endpoint reference

api-endpoint-template.md

76 lines

Prefer generating reference from OpenAPI; use this when that's not possible, or as a guide page around a generated reference. Copy to api/reference/<resource>.md

MARKDOWN

```
# <Resource name>

<One sentence: what this resource represents.>

---

## <Verb phrase, e.g. "Create a refund">

`<METHOD> /<version>/<path>`

<One or two sentences: what it does, important side effects.>

**Auth:** `<Bearer token>` · **Scopes:** `<scope:write>` · **Idempotent:** <yes, via `Idempotency-Key` header / no> · **Rate limit:** <default / N per minute>

### Path parameters

Name	Type	Description
`<id>`	string	<description>

### Query parameters

Name	Type	Required	Default	Description
`<param>`	integer		`<20>`	<description, constraints>

### Request body

Field	Type	Required	Description
`<field>`	string	✓	<description, format, constraints>
`<field>`	integer		<description>

### Example request

```bash
curl -X <METHOD> https://api.example.com/<version>/<path> \
 -H "Authorization: Bearer <token>" \
 -H "Content-Type: application/json" \
 -d '{
 "<field>": "<value>"
 }'
```

### Example response `<201 Created>`

```json
{
 "id": "<id>",
 "<field>": "<value>",
 "createdAt": "2026-10-09T12:00:00Z"
}
```

### ### Errors

HTTP	Code	Meaning	What to do
400	`<validation_error>`	<invalid input>	<fix the field named in `error.field`>
404	`<not_found>`	<resource doesn't exist>	<check the ID>
409	`<conflict_code>`	<state conflict>	<how to resolve>

### ### Side effects

- <Emits `<event.name>` webhook>
- <Sends email to customer>

### ### Business rules

- [<BR-ID>]([<path>](#)): <summary>

---

## <Next operation>

...

## TEMPLATE 14

# Runbook

runbook-template.md

71 lines

Copy to operations/runbooks/<alert-name>.md and link it from the alert's runbook\_url. Mitigation first, diagnosis second

MARKDOWN

```

alert: <AlertName exactly as in the alerting tool>
service: <service>
severity: <SEV-1 | SEV-2 | SEV-3>
owner: <team>
last_reviewed: <YYYY-MM-DD>
last_tested: <YYYY-MM-DD>

Alert: <AlertName>

What users experience: <e.g. checkout fails with an error page>
Severity: <SEV-x> · **Escalate after:** <15 min without mitigation>

1. Quick mitigation (do this first)

1. <Check whether a deploy happened in the last 30 min: link to deploy log>
2. If yes, roll back:

    ```bash
    <rollback command>
    ```

3. <Other fast mitigation: toggle feature flag, scale up, fail over>

2. Diagnosis

| Check | Where / command | Healthy looks like |
|---|---|---|
| <Error rate> | <dashboard link> | << 1%> |
| <Logs> | <`<log query>`> | <no `ERROR` from service X> |
| <Dependency status> | <status page link> | <operational> |

3. Resolution

Cause A: <e.g. payment provider outage>

1. <step>
2. <step>

Cause B: <e.g. database connection pool exhausted>

1. <step>

    ```bash
    <command>
    ```

 ! <Warning for destructive steps>

4. Verify

- [] <Error rate back below threshold for 10 min>
- [] <Synthetic check green>
```

**## 5. Escalation**

```
| After | Contact |
|---|---|
| <15 min> | <secondary on-call / team channel> |
| <30 min or data loss risk> | <incident commander, engineering manager> |
```

**## 6. After the incident**

- Open a postmortem if SEV-1/2: [template]([path to postmortem template](#))
- Update this runbook with anything that was missing.

**## Background**

<Optional: why this alert fires, architecture link, past incidents.>

## TEMPLATE 15

## Blameless incident postmortem

postmortem-template.md

61 lines

Copy to operations/postmortems/YYYY-MM-DD-short-title.md. Focus on systems and processes, not individuals.  
Immutable once published

MARKDOWN

```

date: <YYYY-MM-DD>
severity: <SEV-1 | SEV-2 | SEV-3>
status: draft # draft | reviewed | published
authors: [<name>]
services: [<service>]

Postmortem: <Short title> (<YYYY-MM-DD>)

Summary

<3 sentences: what happened, impact, how it was resolved.>

Impact

| Metric | Value |
|---|---|
| Duration | <HH:MM> (<start> - <end> UTC) |
| Users affected | <number / %> |
| Failed requests / orders | <number> |
| SLO budget consumed | <%> |
| Revenue / contractual impact | <if known> |

Timeline (UTC)

| Time | Event |
|---|---|
| <14:02> | <Deploy of v2.3.1 started> |
| <14:09> | <Alert CheckoutHigh5xxRate fired> |
| <14:15> | <On-call acknowledged, started runbook> |
| <14:21> | <Rollback completed; errors dropping> |
| <14:30> | <Incident resolved> |

Root cause

<Technical cause, explained clearly. Use 5 Whys or similar.>

Contributing factors

- <e.g. Missing test for currency rounding>
- <e.g. Alert threshold too high, delayed detection>

Detection and response

- **What went well:** <...>
- **What went badly:** <...>
- **Where we got lucky:** <...>

Action items

| Action | Type | Owner | Ticket | Due |
|---|---|---|---|---|
| <Add rounding test> | prevent | <name> | <link> | <date> |

```

```
| <Lower alert threshold> | detect | <name> | <link> | <date> |
| <Update runbook step 3> | mitigate | <name> | <link> | <date> |
```

## ## Lessons Learned

- <Short, general lessons for the team>

## TEMPLATE 16

# Tutorial (learning-oriented)

tutorial-template.md

59 lines

Copy to tutorials/<title>.md. One path, no options, every step must work. Test it end to end on a clean environment

MARKDOWN

```
<Build / Create your first ...>

In this tutorial you'll <build X>. By the end you'll have <concrete result> and understand <1-2 concepts>.

Time: ~<N> minutes · **Level:** beginner

What you'll build

<Screenshot, diagram or short description of the end result.>

Before you start

You need:

- <Tool> <version>: [install guide](<link>)
- <Account / access>

Step 1: <Action>

<Short instruction.>

```bash
<command>
```

You should see:

```text
<expected output>
```

Step 2: <Action>

<Short instruction.>

```<language>
<code>
```

<One sentence on what just happened, with no deep theory. Link to explanation instead.>

Step 3: <Action>

...

Check your result

<How the learner confirms everything works.>

What you've learned

- <Concept 1>
- <Concept 2>

Next steps
```

- <How-to guide for a common next task>
- <Explanation page for deeper understanding>
- <Reference for all options>

## TEMPLATE 17

# How-to guide (task-oriented)

how-to-template.md

37 lines

Copy to how-to/&lt;task&gt;.md. Assumes the reader knows the basics. One task per page

MARKDOWN

```
How to <do the task>

<One sentence: when and why you'd do this.>

Prerequisites

- <Permission / role>
- <Version / prior setup>: [link](<path>)

Steps

1. <Action, with UI location first: "In Settings → Billing, select Refunds.">
2. <Action>

    ```bash
    <command>
    ```

3. <Action>

 <Expected result after an important step.>

Verify

<How to confirm it worked: command, UI state, log line.>

Troubleshooting

Problem	Cause	Solution
<exact error message>	<cause>	<fix>

Related

- <Explanation: how this works>
- <Reference: all options>
- <Other how-tos>
```

## TEMPLATE 18

## AGENTS

agents-md-template.md

59 lines

md, instructions for AI coding agents. Copy to the repository root. Keep it short (<200 lines). Link to docs instead of copying them. For Claude Code, CLAUDE.md can contain just "@AGENTS.md" (an import) plus Claude-specific notes

# AGENTS.md

MARKDOWN

## ## Project

<2-3 sentences: what this project is, main stack, architecture style.>

## ## Commands

```
```bash
<install>      # install dependencies
<dev>          # run locally
<test>         # run all tests
<test-single>  # run one test file: <example>
<lint>         # lint + static analysis
<format>       # format code
<build>        # production build
```
```

## ## Where things are

| Need                               | Location                          |
|------------------------------------|-----------------------------------|
| Domain / business logic            | `<path>`                          |
| HTTP / controllers                 | `<path>`                          |
| Tests                              | `<path>`                          |
| Business rules (IDs `BR-*`)        | `<docs-root>/requirements/rules/` |
| Architecture & decisions (`ADR-*`) | `<docs-root>/architecture/`       |
| API contract (source of truth)     | `<path>/openapi.yaml`             |
| Glossary                           | `<docs-root>/glossary.md`         |
| × Do not use (historical)          | `<docs-root>/archive/`            |

## ## Conventions

- <Non-obvious rule, e.g. "Money is integer cents in `\*\_cents` fields; never floats.">
- <Rule, e.g. "Controllers stay thin; logic goes in `src/Domain/\*/Actions`.">
- <Naming / patterns. Link to the full conventions doc: `<docs-root>/development/conventions.md`>

## ## Boundaries

- Never edit: `<generated dirs>`, `<vendor dirs>`, applied migrations in `<path>`.
- Never commit: `.env`, secrets, `<local files>`.
- Ask before: <destructive DB operations, dependency upgrades, deleting files>.

## ## Workflow

- Before finishing a task, run `<test>` and `<lint>`; both must pass.
- When changing an API endpoint, update `<openapi path>` in the same change.
- When changing user-visible behavior, add an entry under **UnreleAsed** in `CHANGELOG.md`.
- When making a significant technical decision, propose an ADR in `<docs-root>/architecture/decisions/`.

## ## Agent working files

- Put plans, notes and reports in `<.agents/>`, **not** in `<docs-root>/`.
- Mark generated docs: `> Draft generated by AI on <date>; needs review.`

**## More context**

- Contributing: [CONTRIBUTING.md](CONTRIBUTING.md)
- Docs index: [<docs-root>/README.md](<docs-root>/README.md)

## APPENDIX

# Further Reading

**What:** Well-known, widely cited articles, guides, specs, books and examples on every topic this guide covers, grouped by topic. **Read when:** you want to go deeper than this guide, or you want the original source behind a recommendation.

## ■ TL;DR: start with these 10

| #  | Resource                                                             | Why                                                            |
|----|----------------------------------------------------------------------|----------------------------------------------------------------|
| 1  | <a href="#">Diátaxis</a>                                             | The framework for tutorials / how-to / reference / explanation |
| 2  | <a href="#">Write the Docs: Documentation Guide</a>                  | Community knowledge base on all things docs                    |
| 3  | <a href="#">Google Technical Writing courses</a>                     | Free, short, practical writing training for engineers          |
| 4  | <a href="#">Google developer documentation style guide</a>           | The most-used style reference for technical docs               |
| 5  | <a href="#">Documenting Architecture Decisions</a> by Michael Nygard | The original ADR article                                       |
| 6  | <a href="#">The C4 model</a>                                         | How to draw architecture diagrams at the right zoom level      |
| 7  | <a href="#">Design Docs at Google</a>                                | How and why to write design docs before building               |
| 8  | <a href="#">Keep a Changelog</a>                                     | The standard changelog format                                  |
| 9  | <a href="#">Docs as Code</a> (Write the Docs)                        | Why docs belong in Git with the code                           |
| 10 | <a href="#">AGENTS.md</a>                                            | The open format for AI coding agent instructions               |

Links checked on 2026-10-09. External sites change, so if a link breaks, search for the title.

## ■ 1. Documentation frameworks and principles

| Resource                                                    | Source             | Why read it                                                            | Guide page                                                                |
|-------------------------------------------------------------|--------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <a href="#">Diátaxis</a>                                    | Daniele Procida    | The four-quadrant model behind most modern product docs                | <a href="#">Model: Diátaxis (by User Need)</a> (p. 76)                    |
| <a href="#">The Documentation System</a>                    | Divio              | The earlier, shorter write-up of the same idea, a good 15-minute intro | <a href="#">Model: Diátaxis (by User Need)</a> (p. 76)                    |
| <a href="#">Docs as Code</a>                                | Write the Docs     | Rationale and practices for treating docs like code                    | <a href="#">Principles of Good Documentation: 3. Docs-as-code</a> (p. 14) |
| <a href="#">A beginner's guide to writing documentation</a> | Write the Docs     | Why to write docs and what a minimal set looks like                    | <a href="#">Developer Docs</a> (p. 29)                                    |
| <a href="#">Readme Driven Development</a>                   | Tom Preston-Werner | Classic essay: write the README before the code                        | <a href="#">Developer Docs: Root README.md</a> (p. 29)                    |
| <a href="#">The Good Docs Project</a>                       | Community          | Open-source templates and guides for many doc types                    | <a href="#">Templates overview</a> (p. 156)                               |

## ■ 2. Writing and style

| Resource                                            | Source                 | Why read it                                                            | Guide page                                                                      |
|-----------------------------------------------------|------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <a href="#">Technical Writing One &amp; Two</a>     | Google                 | Short free courses: sentences, lists, paragraphs, audience, organizing | <a href="#">Style Guide</a> (p. 137)                                            |
| <a href="#">Developer documentation style guide</a> | Google                 | Detailed rules for voice, formatting, code, links, accessibility       | <a href="#">Style Guide</a> (p. 137)                                            |
| <a href="#">Word list</a>                           | Google                 | Which terms to use and avoid                                           | <a href="#">Style Guide: Terminology</a> (p. 139)                               |
| <a href="#">Microsoft Writing Style Guide</a>       | Microsoft              | Second major style reference, strong on tone and global English        | <a href="#">Style Guide</a> (p. 137)                                            |
| <a href="#">GitLab documentation style guide</a>    | GitLab                 | A real-world docs-as-code style guide used by hundreds of contributors | <a href="#">Style Guide</a> (p. 137)                                            |
| <a href="#">How users read on the web</a>           | Nielsen Norman Group   | The research behind "write for scanning"                               | <a href="#">Principles of Good Documentation: 5. Write for scanning</a> (p. 14) |
| <a href="#">F-shaped pattern of reading</a>         | Nielsen Norman Group   | Why the first words of headings and paragraphs matter                  | <a href="#">Style Guide: Headings</a> (p. 137)                                  |
| <a href="#">Plain language guidelines</a>           | U.S. plainlanguage.gov | Clear-writing rules, especially useful for business and user docs      | <a href="#">User Docs: Writing for end users</a> (p. 52)                        |

## ■ 3. Markdown and diagrams

| Resource                                                  | Source      | Why read it                                                | Guide page                                                                         |
|-----------------------------------------------------------|-------------|------------------------------------------------------------|------------------------------------------------------------------------------------|
| <a href="#">Markdown Guide</a>                            | Matt Cone   | Friendly reference for basic and extended syntax           | <a href="#">Markdown Essentials</a> (p. 19)                                        |
| <a href="#">CommonMark tutorial</a>                       | CommonMark  | 10-minute interactive intro to standard Markdown           | <a href="#">Markdown Essentials</a> (p. 19)                                        |
| <a href="#">GitHub Flavored Markdown spec</a>             | GitHub      | Exact behavior of tables, task lists, autolinks            | <a href="#">Markdown Essentials: GitHub-Flavored Markdown (GFM) extras</a> (p. 20) |
| <a href="#">Basic writing and formatting syntax</a>       | GitHub Docs | Alerts, anchors, footnotes and more as GitHub renders them | <a href="#">Markdown Essentials</a> (p. 19)                                        |
| <a href="#">Mermaid documentation</a>                     | Mermaid     | All diagram types and syntax                               | <a href="#">Diagrams &amp; Visuals</a> (p. 146)                                    |
| <a href="#">Include diagrams in Markdown with Mermaid</a> | GitHub Blog | How Mermaid works on GitHub                                | <a href="#">Diagrams &amp; Visuals</a> (p. 146)                                    |

## ■ 4. README, changelog and repository files

| Resource                         | Source        | Why read it                                     | Guide page                                             |
|----------------------------------|---------------|-------------------------------------------------|--------------------------------------------------------|
| <a href="#">Make a README</a>    | Danny Guo     | What a README should contain, with a template   | <a href="#">Developer Docs: Root README.md</a> (p. 29) |
| <a href="#">Awesome README</a>   | Community     | Curated list of excellent READMEs to learn from | <a href="#">Root README template</a> (p. 159)          |
| <a href="#">Keep a Changelog</a> | Olivier Lacan | Changelog format and principles                 | <a href="#">Developer Docs: CHANGELOG.md</a> (p. 32)   |

| Resource                                       | Source             | Why read it                                                         | Guide page                                                             |
|------------------------------------------------|--------------------|---------------------------------------------------------------------|------------------------------------------------------------------------|
| <a href="#">Semantic Versioning</a>            | Tom Preston-Werner | What version numbers communicate                                    | Developer Docs: <a href="#">CHANGELOG.md</a> (p. 32)                   |
| <a href="#">Conventional Commits</a>           | Community          | Commit format that enables generated changelogs                     | <a href="#">CONTRIBUTING template</a> (p. 162)                         |
| <a href="#">Default community health files</a> | GitHub Docs        | CONTRIBUTING, SECURITY, CODE_OF_CONDUCT and where GitHub finds them | Process & Team Docs: <a href="#">Standard repository files</a> (p. 54) |

## ■ 5. API documentation

| Resource                                       | Source                              | Why read it                                            | Guide page                                            |
|------------------------------------------------|-------------------------------------|--------------------------------------------------------|-------------------------------------------------------|
| <a href="#">OpenAPI Specification (latest)</a> | OpenAPI Initiative                  | The REST API contract format                           | API Docs: <a href="#">Spec-first: OpenAPI</a> (p. 35) |
| <a href="#">Learn OpenAPI</a>                  | OpenAPI Initiative                  | Beginner-friendly guide to writing OpenAPI             | API Docs: <a href="#">Spec-first: OpenAPI</a> (p. 35) |
| <a href="#">AsyncAPI docs</a>                  | AsyncAPI Initiative                 | Documenting event-driven and messaging APIs            | API Docs: <a href="#">Event-driven APIs</a> (p. 36)   |
| <a href="#">Documenting APIs course</a>        | Tom Johnson (I'd Rather Be Writing) | The most complete free course on API documentation     | API Docs (p. 34)                                      |
| <a href="#">Google API Design Guide</a>        | Google Cloud                        | API design conventions, including how to document them | API Docs (p. 34)                                      |
| <a href="#">Microsoft REST API Guidelines</a>  | Microsoft                           | Errors, versioning, pagination conventions             | API Docs: <a href="#">Errors</a> (p. 36)              |
| <a href="#">Zalando RESTful API Guidelines</a> | Zalando                             | Detailed, widely copied API guidelines                 | API Docs (p. 34)                                      |
| <a href="#">Spectral / Redocly CLI</a>         | Stoplight / Redocly                 | Linting and rendering OpenAPI specs                    | Maintenance: <a href="#">Automation</a> (p. 153)      |

## ■ 6. Architecture and decisions

| Resource                                                               | Source                               | Why read it                                                | Guide page                                                                                   |
|------------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <a href="#">Documenting Architecture Decisions</a>                     | Michael Nygard                       | The original ADR proposal                                  | Architecture Docs: <a href="#">Architecture Decision Records (ADRs)</a> (p. 45)              |
| <a href="#">ADR GitHub organization</a>                                | ADR community                        | Overview of ADR formats and tools                          | Architecture Docs: <a href="#">Architecture Decision Records (ADRs)</a> (p. 45)              |
| <a href="#">MADR</a>                                                   | ADR community                        | Markdown ADR template with options and pros/cons           | <a href="#">Architecture Decision Record (Nygard style + alternatives) template</a> (p. 166) |
| <a href="#">Architecture decision record examples</a>                  | Joel Parker Henderson                | Many ADR templates and real examples                       | <a href="#">Architecture Decision Record (Nygard style + alternatives) template</a> (p. 166) |
| <a href="#">adr-tools</a>                                              | Nat Pryce                            | CLI for creating numbered ADRs                             | Model: <a href="#">Sequential &amp; Dated Records</a> (p. 88)                                |
| <a href="#">Scaling the Practice of Architecture, Conversationally</a> | Andrew Harmel-Law (martinfowler.com) | ADRs plus the "advice process" for decentralized decisions | Architecture Docs (p. 44)                                                                    |

| Resource                                       | Source                        | Why read it                                     | Guide page                                                       |
|------------------------------------------------|-------------------------------|-------------------------------------------------|------------------------------------------------------------------|
| <a href="#">The C4 model</a>                   | Simon Brown                   | Context, container, component and code diagrams | <a href="#">Architecture Docs: C4 model</a> (p. 45)              |
| <a href="#">Structurizr</a>                    | Simon Brown                   | C4 diagrams as code                             | <a href="#">Diagrams &amp; Visuals: Diagram tools</a> (p. 147)   |
| <a href="#">arc42</a>                          | Gernot Starke, Peter Hruschka | Full architecture documentation template        | <a href="#">Architecture Docs: Architecture overview</a> (p. 44) |
| <a href="#">Design Docs at Google</a>          | Malte Ubl                     | Structure, value and lifecycle of design docs   | <a href="#">Architecture Docs: RFCs and design docs</a> (p. 46)  |
| <a href="#">Software design document guide</a> | Atlassian                     | Practical design-doc outline                    | <a href="#">RFC / design doc template</a> (p. 167)               |

Real RFC processes to study: [Rust RFCs](#) · [Python PEP 1](#) (the PEP process) · [Kubernetes KEPs](#)

## ■ 7. Requirements and product docs

| Resource                                           | Source                 | Why read it                                                                   | Guide page                                                                                    |
|----------------------------------------------------|------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <a href="#">Painless Functional Specifications</a> | Joel Spolsky           | Classic 4-part series on why and how to write specs                           | <a href="#">Business &amp; Requirements Docs</a> (p. 38)                                      |
| <a href="#">Product requirements</a>               | Atlassian              | Agile PRD structure                                                           | <a href="#">Product Requirements Document template</a> (p. 171)                               |
| <a href="#">PRDs and 1-pagers: examples</a>        | Lenny Rachitsky        | Real PRD templates from well-known product companies                          | <a href="#">Product Requirements Document template</a> (p. 171)                               |
| <a href="#">Shape Up: Write the Pitch</a>          | Basecamp (Ryan Singer) | A lean alternative to PRDs: problem, appetite, solution, rabbit holes, no-gos | <a href="#">Business &amp; Requirements Docs: PRD (Product Requirements Document)</a> (p. 40) |
| <a href="#">User stories</a>                       | Atlassian              | User stories with examples                                                    | <a href="#">User story with acceptance criteria template</a> (p. 173)                         |
| <a href="#">INVEST in Good Stories</a>             | Bill Wake              | Origin of the INVEST checklist                                                | <a href="#">Business &amp; Requirements Docs: User stories</a> (p. 41)                        |
| <a href="#">Gherkin reference</a>                  | Cucumber               | Given/When/Then syntax for acceptance criteria                                | <a href="#">Business &amp; Requirements Docs: Acceptance criteria</a> (p. 42)                 |
| <a href="#">Ubiquitous Language</a>                | Martin Fowler          | Why a shared domain vocabulary (glossary) matters                             | <a href="#">Business &amp; Requirements Docs: Domain glossary</a> (p. 43)                     |

## ■ 8. Operations and incidents

| Resource                                                  | Source          | Why read it                                                           | Guide page                                                      |
|-----------------------------------------------------------|-----------------|-----------------------------------------------------------------------|-----------------------------------------------------------------|
| <a href="#">Postmortem Culture: Learning from Failure</a> | Google SRE Book | Blameless postmortems explained                                       | <a href="#">Operations Docs: Incident postmortems</a> (p. 49)   |
| <a href="#">Example Postmortem</a>                        | Google SRE Book | A complete real-format postmortem                                     | <a href="#">Blameless incident postmortem template</a> (p. 181) |
| <a href="#">Incident postmortems</a>                      | Atlassian       | Postmortem process, templates and meeting guide                       | <a href="#">Blameless incident postmortem template</a> (p. 181) |
| <a href="#">PagerDuty Incident Response</a>               | PagerDuty       | Open-source incident response docs, a model for runbook-style writing | <a href="#">Runbook template</a> (p. 179)                       |

## ■ 9. Organizing knowledge and handbooks

| Resource                            | Source                                           | Why read it                                                                            | Guide page                                                                                                         |
|-------------------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <a href="#">GitLab Handbook</a>     | GitLab                                           | The best-known public company handbook, thousands of Markdown pages organized at scale | <a href="#">Storage Locations: Handbook repo (organization-level)</a> (p. 101)                                     |
| <a href="#">The PARA Method</a>     | Tiago Forte                                      | Projects / Areas / Resources / Archive for knowledge bases                             | <a href="#">Model: Flat Structure + Metadata (Wiki-style): PARA (for team or personal knowledge bases)</a> (p. 94) |
| <a href="#">Backstage TechDocs</a>  | Spotify / CNCF                                   | Docs-like-code aggregated into a developer portal                                      | <a href="#">Model: Per-Service Repos + Central Portal</a> (p. 85)                                                  |
| <a href="#">Docs for Developers</a> | Bhatti, Corleissen, Lambourne, Nunez, Waterhouse | Book companion site, covering planning, writing, organizing and maintaining docs       | <a href="#">Writing Process</a> (p. 134)                                                                           |

## ■ 10. AI agents and LLM-friendly docs

| Resource                                       | Source                                   | Why read it                                                                           | Guide page                                                     |
|------------------------------------------------|------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <a href="#">AGENTS.md</a>                      | Agentic AI Foundation (Linux Foundation) | The format and the list of supporting tools                                           | <a href="#">AI Agent Docs</a> (p. 56)                          |
| <a href="#">Claude Code memory (CLAUDE.md)</a> | Anthropic                                | How CLAUDE.md files, nesting and <code>@imports</code> work                           | <a href="#">AI Agent Docs: Agent instruction files</a> (p. 56) |
| <a href="#">Copilot custom instructions</a>    | GitHub Docs                              | <code>copilot-instructions.md</code> and path-specific <code>*.instructions.md</code> | <a href="#">AI Agent Docs: Agent instruction files</a> (p. 56) |
| <a href="#">llms.txt</a>                       | Jeremy Howard (proposal)                 | The llms.txt proposal for LLM-readable site indexes                                   | <a href="#">AI Agent Docs: llms.txt</a> (p. 57)                |

## ■ 11. Docs-as-code tooling

| Tool                                            | What it does                             | Guide page                                         |
|-------------------------------------------------|------------------------------------------|----------------------------------------------------|
| <a href="#">Vale</a>                            | Prose linter with custom style rules     | <a href="#">Maintenance: Automation</a> (p. 153)   |
| <a href="#">markdownlint</a>                    | Markdown style and structure linter      | <a href="#">Maintenance: Automation</a> (p. 153)   |
| <a href="#">lychee</a>                          | Fast link checker for Markdown and sites | <a href="#">Maintenance: Automation</a> (p. 153)   |
| <a href="#">Material for MkDocs</a>             | Popular docs site generator for Markdown | <a href="#">User Docs: Tooling</a> (p. 52)         |
| <a href="#">Docusaurus</a>                      | Docs site generator with versioning      | <a href="#">Model: Organize by Version</a> (p. 91) |
| <a href="#">The Good Docs Project templates</a> | Community doc templates                  | <a href="#">Templates overview</a> (p. 156)        |

## ■ 12. Books

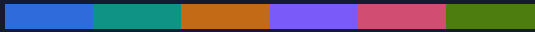
| Book                                               | Author(s)                                                                            | Why read it                                           |
|----------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------|
| <a href="#">Docs for Developers</a> (Apress, 2021) | Jared Bhatti, Zachary Sarah Corleissen, Jen Lambourne, David Nunez, Heidi Waterhouse | The best end-to-end book on engineering documentation |
| <a href="#">Docs Like Code</a>                     | Anne Gentle                                                                          | Practical docs-as-code workflows with Git and CI      |

| Book                                                                   | Author(s)         | Why read it                                                                                               |
|------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------|
| <i>Living Documentation</i><br>(Addison-Wesley, 2019)                  | Cyrille Martraire | Docs generated from and kept in sync with code, DDD-friendly                                              |
| <i>Every Page Is Page One</i><br>(XML Press, 2014)                     | Mark Baker        | Writing self-contained topics for readers arriving from search, which is just as relevant to AI retrieval |
| <i>Site Reliability Engineering</i><br>( <a href="#">free online</a> ) | Google            | Postmortems, on-call and operational docs culture                                                         |

## ■ 13. Great documentation to study

Learn by example. Pick one or two that match your project type and look at how they're organized.

| Docs                                        | Study it for                                                       |
|---------------------------------------------|--------------------------------------------------------------------|
| <a href="#">Stripe API reference</a>        | Reference layout, examples next to every field, error docs         |
| <a href="#">Django documentation</a>        | Clear separation of tutorials, topic guides, reference and how-tos |
| <a href="#">Kubernetes documentation</a>    | Concepts / tasks / tutorials / reference at very large scale       |
| <a href="#">GitLab Handbook</a>             | Organizing a company's knowledge in Markdown                       |
| <a href="#">Rust RFCs</a>                   | A mature, numbered proposal process                                |
| <a href="#">PagerDuty Incident Response</a> | Operational docs written for use under stress                      |
| <a href="#">Awesome README</a>              | README patterns from many projects                                 |



# Wrong docs are worse than no docs.

Which documents should a project have? Where should they live? How should they be named, linked, reviewed and retired? This handbook answers those questions with comparison matrices, decision trees, ready-made folder structures and 18 copy-paste templates.

**Kyrylo Butskyi**

`kyrylo.butskyi@gmail.com`